04820KA001 Mono Canada Slitter 903 1029 V9

Author: Michou
Created: 10/30/2026 13:41:49
Last Modified: 10/30/2026 13:41:49
Comment: Modified to Connect 2nd Diameter Sens

Sysmac Studio Module Version: 1.60.0.64010

Table of Contents

04820KA001 Mono Canada Slitter 903 1029 V9	3
1.PLC	3
1-8.POUs	3
1-8-1.Programs	3
1-8-1-1.TaglMonoTRM	3
1-8-1-1-1.Variables	3
1-8-1-1-2.General	14
1-8-1-1-3.Motor_Manager	20
1-8-1-1-4.OverRider_Manager	34
1-8-1-1-5.Console	37
1-8-1-1-6.Alarms	40
1-8-1-1-7.Output_Manager	45
1-8-1-1-8.AutoStop_Control	51
1-9.Data	72
1-9-1.Data Types	72
1-9-2.Global Variables	73

1.PLC

1-8.POUs

1-8-1.Programs

1-8-1-1.TaglMonoTRM

1-8-1-1-1.Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Comment
VAR						
SpdDelta	REAL			False	False	Minimum or maximum speed SpdDeltaerence
UwSlowDiamCalc	REAL			False	False	V7c: speed-dependent unwinder slow-down diameter (mm)
UwSlowDiamLatch	BOOL			False	False	V7c: unwinder diameter slow-down reached (frozen)
FFRate	REAL			False	False	V7f: ramp rate (reference per 4 ms scan)
FFRateSet	REAL			False	False	V7f: rate allowed to still stop at the target
FFJerk	REAL			False	False	V7f: max rate change per scan
SpdScanCnt	INT			False	False	V9: scan counter for the 100 ms speed window
HornRun	BOOL			False	False	V9: slow-down / final speed running
HornRunMem	BOOL			False	False	V9: edge memory
HornStartMem	BOOL			False	False	V9: start edge memory
HornTicks	INT			False	False	V9: scans since slow-down start
HornCnt	INT			False	False	V9: scans in the beep period
HornPer	INT			False	False	V9: beep period in scans
HornStartScans	INT			False	False	V9: start beep in scans
HornLenScans	INT			False	False	V9: short beep in scans
HornSlowBeep	BOOL			False	False	V9: slow-down beep
HornStopPend	BOOL			False	False	V9: stopped after a slow-down, waiting for standstill
HornStopTicks	INT			False	False	V9: stop beep counter
LongActive	BOOL			False	False	V9: long-baseline learning running
LongValid	BOOL			False	False	V9: long baseline >= 20 mm growth, replaces the windows
LongStartPos	DINT			False	False	V9: encoder at the start of the long baseline
LongStartD	REAL			False	False	V9: rewinder diameter at the start (mm)
LongGrow_mm	REAL			False	False	V9: growth since the start (mm)
LongPls	REAL			False	False	V9: pulses since the start
LongK	REAL			False	False	V9: K from the long baseline
RevW	REAL			False	False	V9: encoder pulses summed in the current revolution
RevDP	DINT			False	False	V9: encoder pulses this scan
RevPrevPos	DINT			False	False	V9: encoder last scan
RevSum	REAL			False	False	V9: sum of raw DT35 distance x pulses
RevMin	REAL			False	False	V9: min raw distance in the revolution
RevMax	REAL			False	False	V9: max raw distance in the revolution
RwDiamRevOk	BOOL			False	False	V9: one full revolution averaged since the last fault
UwEmptyTicks	INT			False	False	V9: unwinder empty time (100 m

						s ticks)
UwFullTicks	INT			False	False	V9: new roll seen time (100 ms ticks)
GsmArmed	BOOL			False	False	V9: roll was empty -> next new roll asks the GSM question
T_JogStartBlk	TOF			False	False	V9: START blocked while JOG held + 2 s
StartHeldBlk	BOOL			False	False	V9: START pressed during the block, wait for release
StartPB_OK	BOOL			False	False	V9: START push-button accepted (replaces I_Start_PB in General 7/8/10)
FFSoftMem	BOOL			False	False	V9: soft stop last scan (rung B)
FFSlowAcc	BOOL			False	False	V9: gentle ramp-up after a cancelled soft stop
FFStartEdge	BOOL			False	False	V9: START push-button edge (rung B)
FFStartMem	BOOL			False	False	V9: START last scan (rung B)
SpdDeltaMin	REAL			False	False	Minimum speed difference
FFSqrtArg	REAL			False	False	SQRT argument (REAL)
SpdDeltaMax	REAL			False	False	Maximum speed SpdDeltaerence
SoftStopMem	BOOL			False	False	v6r: soft stop edge memory
SoftStopRatio	REAL			False	False	v6r: InvRef counts per m/min, frozen at the soft stop start
Clock100ms	BOOL			False	False	100 ms clock
MemPos	DINT			False	False	Previous encoder count
MatChangeCnt	INT			False	False	v6s: consecutive learning windows far from RwMat_K
DeltaPulse	DINT			False	False	Encoder count difference
SlowMem	BOOL			False	False	SlowDown edge memory
DecelDiam	REAL			False	False	Rewinder diameter used by the decel model (mm, min 100)
TeachSwap	REAL			False	False	Temporary for UNDO swap
SlowStartDiam	REAL			False	False	Diameter at slow-down start (mm)
SoftStartTicks	INT			False	False	S-curve start: 4 ms ticks since start
SoftStartK	REAL			False	False	S-curve start: accel factor 0.05..1
BrakeHold	BOOL			False	False	Keep T-one in auto during a normal stop (O_Brake)
StopNow	BOOL			False	False	Immediate stop request (replaces I_Stop_PB in General rung 9)
StopPB_Mem	BOOL			False	False	STOP button edge memory
T_StopLong	TON			False	False	STOP held 1 s = immediate stop
T_SoftStop	TON			False	False	Soft stop timeout (120 s)
BrakeHoldTicks	INT			False	False	BrakeHold duration after stop (100 ms ticks, max 150)
StopStartDiam	REAL			False	False	Diameter at stop command (mm)
DecelK_Meas	REAL			False	False	Measured decel per mm of diameter
SlowMeasActive	BOOL			False	False	Slow-down measurement running
TeachTicks	INT			False	False	TEACH CORE stable time (100 ms ticks)
SlowStartPos	DINT			False	False	Encoder at slow-down start
SlowStartSpd_Pps	REAL			False	False	Speed at slow-down start (pulses/s)
SlowMeas_Pls	REAL			False	False	Measured slow-down length (pulses)
SlowDecelMeas	REAL			False	False	Measured slow-down decel (mm/

_mm						s2)
SlowOneRamp	BOOL			False	False	Slow-down uses one ramp (manual or learned)
Velocity	REAL			False	False	Speed in [mm/s]
Alarm_Array	ARRAY[0..32] OF BOOL			False	False	Alarm
T_PlcOn	TON			False	False	PLC active timeout
ControlVoltage	BOOL			False	False	Auxiliaries OK
StartMotor_Cmd	BOOL			False	False	Motor start command
HubExtractEnable	BOOL			False	False	Hub extraction enable command
Brake_Cmd	BOOL			False	False	Brake command
T_StopVent	TOF			False	False	Fans stop timer
TimStopVen	TIME			False	False	Fans stop timeout
MotorCooling	BOOL			False	False	Fans command
OverRider_Auto	BOOL			False	False	OverRider in automatic
OverRider_Cont	BOOL			False	False	OverRider in continuous
OverRider_Lock	BOOL			False	False	OverRider lock command
GSM_LearnFault	BOOL			False	False	Paper [Grams / Square Meter] LearnFailCnt ≥ RwLearnFailMax
TickCnt	INT			False	False	100 ms tick scan counter (25 x 4 ms)
RunWasJog	BOOL			False	False	Run used jog / manual / slow button (2nd ramp)
StopLearnValid	BOOL			False	False	This stop may be learned
SpdBand_Pps	REAL			False	False	Speed band of the trim ramp (pulses/s)
TrimDecel_Pps2	REAL			False	False	Trim ramp decel (pulses/s2)
SpdKnee_Pps	REAL			False	False	Speed where coarse ramp hands over to trim ramp (pulses/s)
OverRider_RaiseUp	BOOL			False	False	OverRider raise command
Pot1_Mem	INT			False	False	Work potentiometer value
Clock10ms	BOOL			False	False	10 ms clock
VelSet_mms	REAL			False	False	Set speed in mm/s
Vfd_SP_Modif	REAL			False	False	Inverter speed modifier
RwAutoLimAct	BOOL			False	False	v6y: rewinder AUTO speed limit active (taken over at standstill)
UwTeachTicks	INT			False	False	Unwinder TEACH CORE samples (100 ms)
T_PosPresOff	TON			False	False	OverRider photocell OFF timer
T_PosPresOn	TON			False	False	OverRider photocell ON timer
TimPosPresOff	TIME			False	False	OverRider photocell OFF timeout
TimPosPresOn	TIME			False	False	OverRider photocell ON timeout
RollEnd_Active	BOOL			False	False	Roll diameter photocell intercepted
Output_Ratio	REAL			False	False	Analog output ratio
Input_Ratio	REAL			False	False	Analog input ratio
T_StartLow	TON			False	False	Slow start timer
T_UwDiamDebounce	TON			False	False	Roll diameter photocell Debounce timer 500mS
UwDiamReached	BOOL			False	False	Roll diameter reached
Index_A	INT			False	False	
AlarmStrobe	BOOL			False	False	Alarm message display strobe
T_Display	TON			False	False	Alarm display timer
RangeOK	BOOL			False	False	Range reached
Vel_Max_mms	REAL			False	False	Maximum speed in mm/s
Slow_Inhibit	BOOL			False	False	Inhibit slow button

Decel_Ramp	REAL			False	False	Deceleration ramp
Accel_Ramp	REAL			False	False	Acceleration ramp
Pot2_Mem	INT			False	False	Final potentiometer value
Current_mm	REAL			False	False	Current millimetres
EncoderPulse_to_mm	REAL			False	False	mm per encoder pulse
SlowDown	BOOL			False	False	Slow down
Stop	BOOL			False	False	Stop
T_EncoderStill	TON			False	False	(?) Encoder stopped timer
Analog_Res	UINT			True	False	Analog board resolution
Mt_To_mm	REAL			False	False	m/min to mm/s factor
Sec_to_Min	REAL			False	False	mm/s to m/min factor
Pot1_Word	WORD			False	False	(?) Pot 1 raw word
Pot1_WordSW	WORD			False	False	(?) Pot 1 raw word, byte-swapped
Pot2_Word	WORD			False	False	(?) Pot 2 raw word
Pot2_WordSW	WORD			False	False	(?) Pot 2 raw word, byte-swapped
InvRef2	UINT			False	False	(?) Inverter reference 2
InvRefSW	WORD			False	False	(?) Inverter reference, byte-swapped
InvRefWW	WORD			False	False	(?) Inverter reference, word
Init_Ramp	BOOL			True	False	(?) Ramp initialisation
SlowByDiam	BOOL			False	False	(?) Slow down by diameter
UwStopByDiam	BOOL			False	False	Stop by diameter
T_Screen_Delay	TON			False	False	(?) Start page timer
T_PulStop	TON			False	False	(?) Stop button timer
Reading_Max	REAL	2000		True	False	Maximum sensor reading (full scale)
DeadZone	REAL	60		True	False	Sensor dead zone
In_UwDiameter	REAL			True	False	UnWinder Diameter Analog input voltage
T_Sample	TON			False	False	100 ms tick timer
Tick	BOOL			False	False	100 ms tick
MmPerPulse	REAL			False	False	mm per encoder pulse
RwDistRaw_mm	REAL			False	False	Rewinder Unfiltered distance (mm)
RwFiltInit	BOOL			False	False	Rewinder Filter initialised
SpdInit	BOOL			False	False	Speed init
EncPrev	DINT			False	False	Encoder at previous tick
SpdAct_Pps	REAL			False	False	Actual Speed (pulses/s)
LearnInit	BOOL			False	False	Learn initialised
LearnWinRunning	BOOL			False	False	Motor ran whole window
LearnWinTicks	INT			False	False	Learn window counter
Encoder_Last	DINT			False	False	Encoder at window start
Diameter_Last	REAL			False	False	Diameter at window start
LearnWin_Pls	DINT			False	False	Pulses in window
LearnWinDiamDelta	REAL			False	False	Diameter change in window
MatK_Window	REAL			False	False	Material factor this window
MotorRunMem	BOOL			False	False	Motor start edge memory
StopMeasActive	BOOL			False	False	Stop measurement running
StopStartPos	DINT			False	False	Encoder at stop command
StopStartSpd_Pps	REAL			False	False	Speed at stop command (pulses/s)
StandstillTicks	INT			False	False	Standstill tick counter
LearnWinOk	BOOL			False	False	Learning window gave a valid result
ThickWin_mm	REAL			False	False	Paper thickness this window (m

						m)
LearnFailCnt	INT			False	False	Consecutive failed learning windows
StopLearnOk	BOOL			False	False	Stop measurement gave a valid result
StopDecelMeas_mm	REAL			False	False	Measured stop decel (mm/s2)
StopMeas_Pls	REAL			False	False	Measured stop length (pulses)
StopDecelMeas	REAL			False	False	Measured stop decel (pulses/s2)
StopLen_Pls	REAL			False	False	Stop length (pulses)
StopLenFinal_Pls	REAL			False	False	Stop length from slow speed (pulses)
RwSlowLen_Pls	REAL			False	False	Rewinder Slow-down length (pulses)
SlowDecel_Pps2	REAL			False	False	Slow-down decel (pulses/s2)
SpdFinal_Pps	REAL			False	False	Final speed (pulses/s)
RwRemain_Pls	REAL			False	False	Rewinder Remaining length to target (pulses)
LearnWinStartPos	DINT			False	False	Rewinder Position Window 0
LearnWinStartDiam	REAL			False	False	Rewinder Diameter Window 0
T_ByPassProtOff	TOF			False	False	Delay Off Timer
UwEnMem	BOOL			False	False	Edge memory UwStopEnable
RwEnMem	BOOL			False	False	Edge memory RwEnable
SpdRef_Pps	REAL			False	False	Reference speed MAX (actual,setpoint) (pulses/s)
SpdWork_Pps	REAL			False	False	Work speed setpoint (pulses/s)
SlowLenBase_Pls	REAL			False	False	Slow-down length without margin (pulses)
UwSlowLen_Pls	REAL			False	False	Unwinder slow-down length (pulses)
UwMarginUsed_m	REAL			False	False	Unwinder margin at slow speed (m)
SqrtArg	REAL			False	False	Square root argument
AutoSlowCmd	BOOL			False	False	Automatic Slow Command
AutoStopCmd	BOOL			False	False	Automatic Stop Command
RwLastValidDiam	REAL			False	False	Rewinder Last valid diameter (mm)
RwRemainMax_Pls	REAL			False	False	Rewinder Remaining length to max diameter (pulses)
UnWinder_Empty	BOOL			False	False	UnWinder Empty
StopFailCnt	INT			False	False	Consecutive failed stop measurements
UwTeachSum	REAL			False	False	Unwinder TEACH CORE sonar sum (mm)
T_MatReset	TON			False	False	v6s: RwMatReset stays ON 3 s
RwAutoMaxSpdAct	REAL			False	False	v6y: active rewinder AUTO max speed (taken over at standstill)
UwEmptyMem	BOOL			False	False	v6x: UnWinder_Empty edge memory
LearnWinLen_Pls	REAL			False	False	v6w: learning window length in encoder pulses
WebRatio	REAL			False	False	V7: drive counts per m/min of web at the current diameter
FFActive	BOOL			False	False	V7: feed-forward loop active
FFWasActive	BOOL			False	False	V7: edge memory
FFTarget	REAL			False	False	V7: reference for the set speed (

						mm/s of the 0-10 V scale)
RampRef	REAL			False	False	V7: ramped reference (mm/s of the 0-10 V scale)
StepAcc	REAL			False	False	V7: accel step per scan
StepDec	REAL			False	False	V7: decel step per scan
FFSteadyTicks	INT			False	False	V7: scans at target
FFErr	REAL			False	False	V7: relative speed error
VAR_EXTERNAL						
VelSet	REAL				False	Set speed
OverSpeed	BOOL				False	Speed too high
SpdDelta_Percent	REAL				False	Speed difference percentage
Encoder_Pos	DINT				False	Encoder counter
Velocity_M_min	REAL				False	Speed in m/min
Current_Metre	REAL				False	Current metres
I_RsetCnt	BOOL				False	Metre counter reset button
WorkSpd_Max_Set	REAL				False	HMI request: work pot full scale (m/min), applied only with motor stopped
FinalSpd_Max_Set	REAL				False	HMI request: final pot full scale (m/min), applied only with motor stopped
SpdMax_Pending	BOOL				False	New pot range waiting for the motor to stop
RwAxisDist_Prev	REAL				False	Rewinder: RwAxisDist before the last TEACH (for UNDO)
UwCoreDist_Prev	REAL				False	Unwinder: UwCoreDistance before the last TEACH (for UNDO)
RwTeachUndo	BOOL				False	HMI: UNDO rewinder TEACH (PLC resets it)
UwTeachUndo	BOOL				False	HMI: UNDO unwinder TEACH (PLC resets it)
RwLearnReset	BOOL				False	HMI / Watch: clear learned decels RwStopDecelK, RwStopDecel, RwSlowDecelK (PLC resets it)
UnderSpeed	BOOL				False	Speed too low
I_ControlVoltage	BOOL				False	Auxiliaries ON
I_OverRider_PH	BOOL				False	Roll diameter photocell
RwCoreDiameter	REAL				False	Rewinder core outside diameter for TEACH CORE (mm)
UwTeachState	INT				False	Unwinder TEACH CORE: 0 idle 1 teaching 2 done 3 refused
SoftStop	BOOL				False	Soft stop in progress (HMI)
SoftStopEnable	BOOL				False	STOP button = soft stop (False = immediate as before)
SoftStopEndSpd	REAL				False	Soft stop: run command drops below this speed (m/min)
SoftStopDecel	REAL				False	Soft stop decel (mm/s2), 0 = DecelCoarse
RwTeachCore	BOOL				False	HMI: TEACH CORE request (PLC resets it)
RwTeachState	INT				False	TEACH CORE: 0 idle 1 teaching 2 done 3 refused
O_MotorStart	BOOL				False	Motor start
RwSlowDecelK	REAL				False	Learned slow-down decel per mm of rewinder diameter (mm/s2 per mm)
RwStopDecelK	REAL				False	Learned stop decel per mm of rewinder diameter (mm/s2 per m

						m)
I_OR_Position_Sens	BOOL				False	Rider roll position sensor
WorkSpd_Max	REAL				False	Work speed pot full scale (m/min), 50..VelMax
FinalSpd_Max	REAL				False	Final speed pot full scale (m/min), 5..WorkSpd_Max
I_OR_Pressure_Sens	BOOL				False	Rider roll pressure switch
I_OR_Auto_OS	BOOL				False	Rider roll in automatic
I_OR_Cont_OS	BOOL				False	Rider roll in continuous
I_Start_PB	BOOL				False	Start button
I_SafeGuard_Closed	BOOL				False	Side doors closed
I_Motor_Thermal	BOOL				False	Motor thermal switch
I_FanTemp_Ok	BOOL				False	Motor (fan) thermal OK
I_Stop_PB	BOOL				False	Stop button
I_Hub_Position	BOOL				False	Hub/chuck in extraction position
I_FrontGuard_Closed	BOOL				False	Front guard closed
RwMaxSlowFallback	REAL				False	Rewinder max: slow-down diameter while K not learned (mm)
RwMaxSlowAtDiam	REAL				False	Rewinder max: slow-down starts at diameter (mm)
O_Alarm_Array	BOOL				False	Alarm lamp
O_RwUnload	BOOL				False	Unloaders enable - opening
I_Prot_ByPass	BOOL				False	Guards-bypassed selector switch
O_Prot_ByPass_Lamp	BOOL				False	Guards bypassed lamp
O_Brake	BOOL				False	Brake command
Fan_Timeout	REAL				False	Fans stop timeout (from panel)
O_CoolingFan	BOOL				False	Brake and motor fans
O_OR_Auto_Lamp	BOOL				False	Rider roll auto/continuous lamp
O_OverRider_Up	BOOL				False	Raise rider roll
O_OverRider_Hold	BOOL				False	Lock rider roll
Fix10Vdc	INT				False	Fixed 10 Vdc generator
InvRef	INT				False	Inverter speed reference
Encoder_On	BOOL				False	Encoder enable
Encoder_Reset	BOOL				False	Encoder reset
OverRider_TimeOFF	REAL				False	Rider roll photocell OFF timeout (panel)
OverRider_TimeON	REAL				False	Rider roll photocell ON timeout (panel)
VelMax	REAL				False	Maximum speed
I_Jog_PB	BOOL				False	Slow speed button
O_Motor_Lamp	BOOL				False	Motor ON lamp
AlarmOn	BOOL				False	Alarm active
BuzzerOn	BOOL				False	Panel buzzer command
I_VFD_Alarm	BOOL				False	Inverter in alarm
Msg	INT				False	Message
Index_B	INT				False	Active alarm index
MotStop	BOOL				False	Motor stopped
Clock1s	BOOL				False	1 s clock
ManMode	BOOL				False	Manual mode
Speed_Man_SP	INT				False	% speed in manual

Counter_Reset	BOOL				False	Panel counter reset button
I_FrontGuard_Open	BOOL				False	Front guard open
RwUnload_Enable	BOOL				False	Unloaders enable - closing
JogSpd_SP	REAL				False	Speed in manual
O_AccDec2	BOOL				False	Second acc/dec ramp
AccelTrim	REAL				False	Acceleration coefficient in run
DecelTrim	REAL				False	Deceleration coefficient in run
DecelCoarse	REAL				False	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL				False	Deceleration coefficient at start (orig. comment; name suggests accel)
Speed_Display	REAL				False	Set speed (displayed)
O_AcousticSignal	BOOL				False	Horn / acoustic signal
Pot1_Work_Velocity	INT				False	Potentiometer 1 value - Work speed -
O_FrontGuard_ByPass	BOOL				False	Front guard bypass
Pot2_Slow_Velocity	INT				False	Potentiometer 2 value - Final speed -
LowSpd_Display	REAL				False	Slow-down speed (displayed)
Encoder_Pulse	INT				False	Encoder pulses
LowSpd_Man_SP	INT				False	% slow-down speed in manual
RwSlowDecelLrn	REAL				False	Learned real slow-down decel (mm/s2) - 0 = not learned
UwSlowMargin_m	REAL				False	Slow-down metres
UwStop_At_Meter	REAL				False	Total metres
I_AutoStop_OS	BOOL				False	Stop on metre counter
I_RollEnd_SlowDown	BOOL				False	Slow down at end of roll
I_SlowSens_PH	BOOL				False	Slow-down photocell
I_WebBreak_Monitor	BOOL				False	Paper break monitoring ON
I_WebBreak_PH	BOOL				False	Paper break photocell
I_Vacuum_OK	BOOL				False	Extractor fan thermal OK
CountReset_Req	BOOL				False	Metre counter reset request
Psw	INT				False	Password
I_RsetAlarm	BOOL				False	Alarm reset
Fix10Vdc_1	UINT				False	Fixed 10 Vdc generator
EncoderWheel_Diam	REAL				False	Encoder wheel diameter
AlarmCodeSV	ARRAY[0..32] OF BOOL				False	Panel alarms
SafeGuard_ByPass	BOOL				False	Guards bypassed
FrontGuard_Closed	BOOL				False	Front guard closed
Core_Measure	BOOL				False	(?) Stored core value
UwDiam_Measure	REAL				False	Measured distance
UwCoreDistance	REAL				False	(?) Core measurement
RwSlowFallbackMm	REAL				False	

AI_Diam_Unwinder	INT				False	Analog value - parent roll measurement
RwMatChangePct	REAL				False	
UwCoreDiameter	REAL				False	(?) Shaft diameter
Measure_ON	BOOL				False	(?) Measurement enabled
UwDiam_Act	REAL				False	(?) Measured roll diameter
UwDiamStop	REAL				False	(?) Stop diameter
UwSlowDown_Diam	REAL				False	(?) Slow-down diameter
Alarm_Reset	BOOL				False	(?) Alarm reset from panel
HMI_Screen	ARRAY[0..1] OF DINT				False	HMI panel page
Psw22	BOOL				False	(?) Password 22
RwMatReset	BOOL				False	
Psw44	BOOL				False	(?) Password 44
I_Roll_OK	BOOL				False	(?) Roll OK
I_RollEnd_PH	BOOL				False	(?) End-of-roll photocell
AI_Diam_Rewinder	INT				False	Rewinder DT35 raw - I/O Map Unit2 Ch2
RwStopEnable	BOOL				False	Rewinder diameter stop enabled
RwTargetDiam	REAL				False	Rewinder final diameter (mm)
RwUseSlow	BOOL				False	Slow down before target
RwSlowMargin_m	REAL				False	Metres at slow speed before stop
RwSlowDecel	REAL				False	Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL				False	Reset requests below target - this (mm)
RwDist0V	REAL				False	DT35 distance at 0 V (mm)
RwDist10V	REAL				False	DT35 distance at 10 V (mm)
RwRawMin	INT				False	Raw below = sensor fault
RwRawMax	INT				False	Raw above = sensor fault
RwStopOnFault	BOOL				False	Stop on sensor fault
RwAxisDist	REAL				False	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL				False	Distance filter per 100 ms (0..1)
RwWinTicks	INT				False	Material learn window (100 = 10 s)
RwMinPulses	DINT				False	Min pulses in window to learn
RwLearn_K	REAL				False	Material factor filter weight
RwMat_K	REAL				False	Learned material factor
RwLearnStop	BOOL				False	Learn stop decel on every stop
RwMinLearnSpd	REAL				False	Min speed to learn a stop (m/min)
RwStopDecel	REAL				False	Learned stop decel (pulses/s2)
RwTreact	REAL				False	Stop reaction time (s)
RwTstopDef	REAL				False	Stop time until decel learned (s)
RwTrim	REAL				False	Stop length trim (>1 = earlier)
RwDist	REAL				False	Filtered distance (mm)
RwDiam	REAL				False	Rewinder diameter (mm)
RwSpeed	REAL				False	Line speed from encoder (m/min)
RwRemain_m	REAL				False	Metres to target diameter
RwStopLen_m	REAL				False	Stopping length now (m)
RwSlowLen_m	REAL				False	Slow-down length now (m)
RwStopReq	BOOL				False	Rewinder diameter STOP request
RwSlowReq	BOOL				False	Rewinder diameter SLOW request
RwSensFault	BOOL				False	Rewinder DT35 sensor fault

UwStopEnable	BOOL				False	Unwinder metre-counter stop ON (HMI)
SlowAtDisp	REAL				False	HMI: slow-down starts at (m or mm, see SlowAtMode)
UwSlowDiamAuto	BOOL				False	
UwSlowDiamAct	REAL				False	
RwJogMaxDiam	REAL				False	
JogInhibit	BOOL				False	
SCurveTime	REAL				False	
TOneDecel	BOOL				False	
HornStartBeep_s	REAL				False	
HornBeepLen_s	REAL				False	
HornBeepPeriod	REAL				False	
HornStopBeep_s	REAL				False	
HornTest	BOOL				False	
RwLockDiam	REAL				False	
RwLockMinGrow_mm	REAL				False	
RwMatLocked	BOOL				False	
LongThick_mm	REAL				False	
RwDiamRev	REAL				False	
RwWave_mm	REAL				False	
UwShowStopLen	BOOL				False	
UwShowSlowDiam	BOOL				False	
SoftCancelAccel	REAL				False	
SpdHold	BOOL				False	
SlowAtMode	INT				False	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL				False	HMI: show slow-down display
UwSlow_at_Metre	REAL				False	Unwinder: slow-down starts at (m)
UwSlowReq	BOOL				False	Unwinder metre slow-down request (latched)
RwSlowAtDiam	REAL				False	Rewinder: slow-down starts at diameter (mm)
RwStopAtDiam	REAL				False	Rewinder: stop command at diameter (mm)
RwMaxDiam	REAL				False	Rewinder max diameter protection (mm)
RwMaxStop	BOOL				False	Rewinder max diameter STOP (latched)
RwMaxSlow	BOOL				False	Rewinder max diameter SLOW (latched)
UwEmptyHyst	REAL				False	
RwThickMin_mm	REAL				False	Learning: min plausible paper thickness (mm)
RwThickMax_mm	REAL				False	Learning: max plausible paper thickness (mm)
RwDecelMin_mm	REAL				False	Learning: min plausible stop decel (mm/s2)
RwDecelMax_mm	REAL				False	Learning: max plausible stop decel (mm/s2)
RwLearnFailMax	INT				False	Learning: failed windows before alarm (12 = 2 min)
RwStopFailMax	INT				False	Learning: failed stops before alarm

	GSM_Learned_OK	BOOL				False	Rewinder learning fault (alarm 16)
	RwThick_mm	REAL				False	Learned paper thickness (mm)
	Psw33	BOOL				False	
	RwAutoMaxSpd	REAL				False	
	RwWinLen_m	REAL				False	
	RwWinGrow_mm	REAL				False	
	GsmAsk	BOOL				False	
	GsmAskYes	BOOL				False	
	DecelWeb	REAL				False	
	AccelWeb	REAL				False	
	RwRatioD	REAL				False	
	FFEnable	BOOL				False	

1-8-1-1-2.General

0 Remote 10.146.233.11 ... 1s clock.



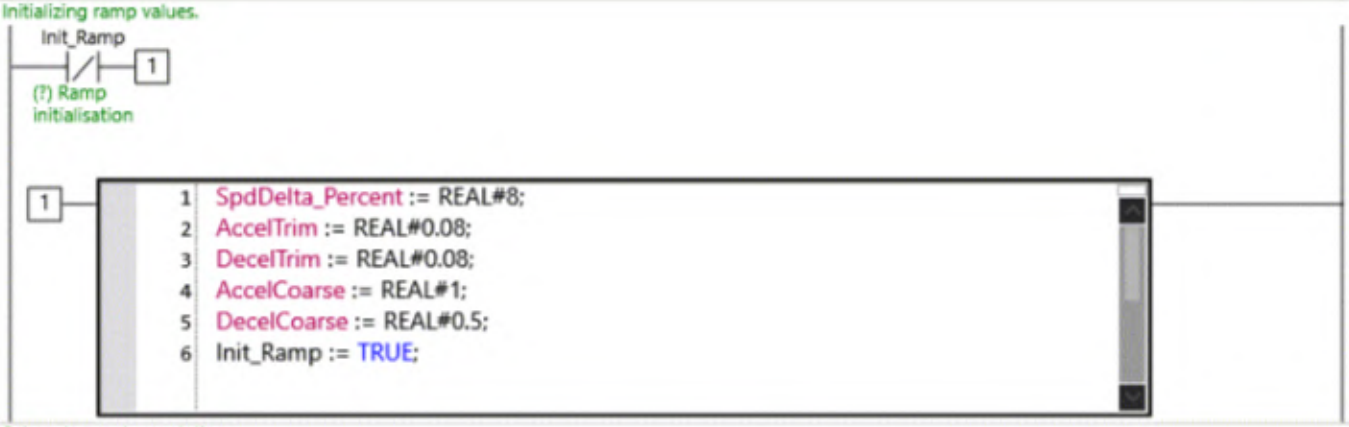
1 100ms clock.



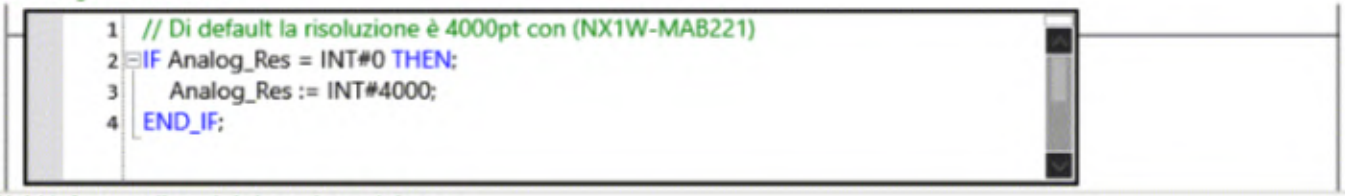
2 10ms clock.



3 Initializing ramp values.



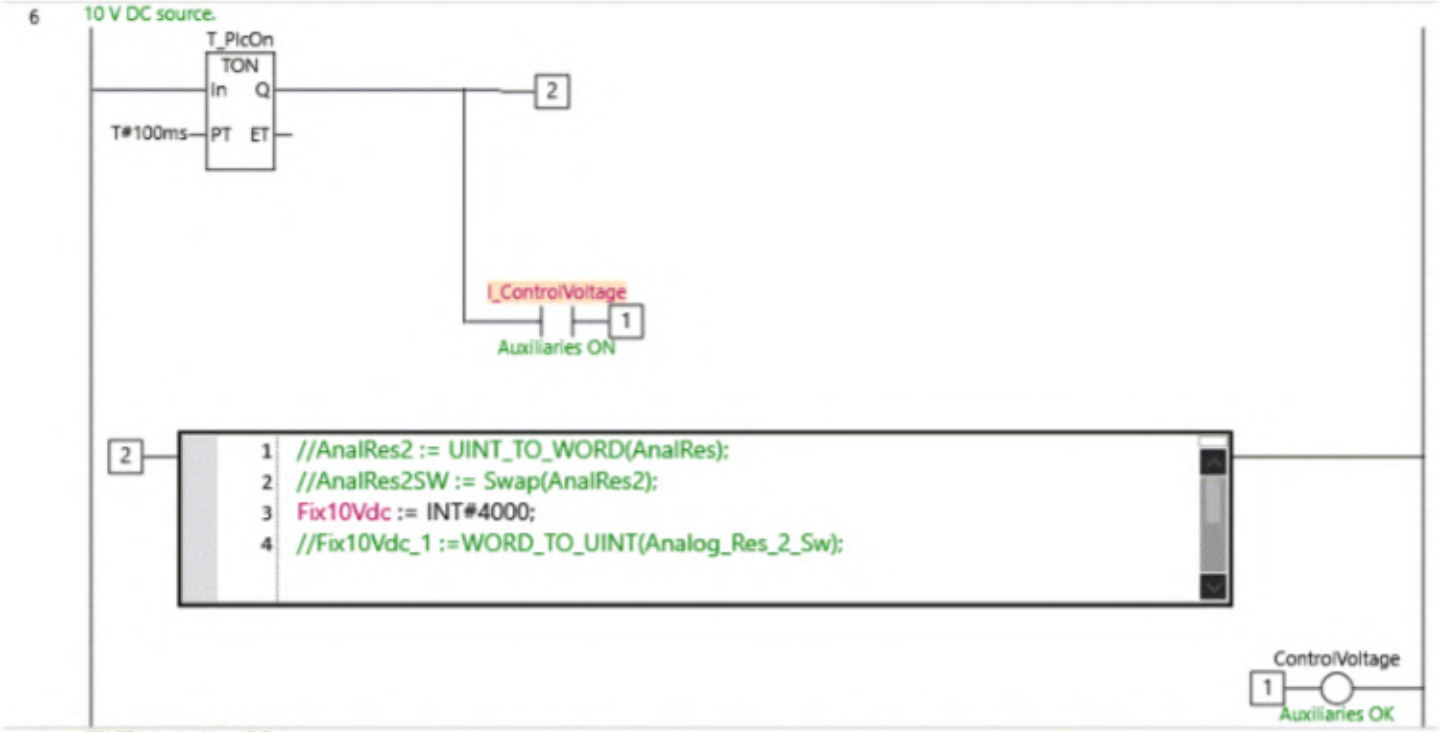
4 Set analog scale resolution.



5 Variable for converting m/min to mm/sec and vice versa.



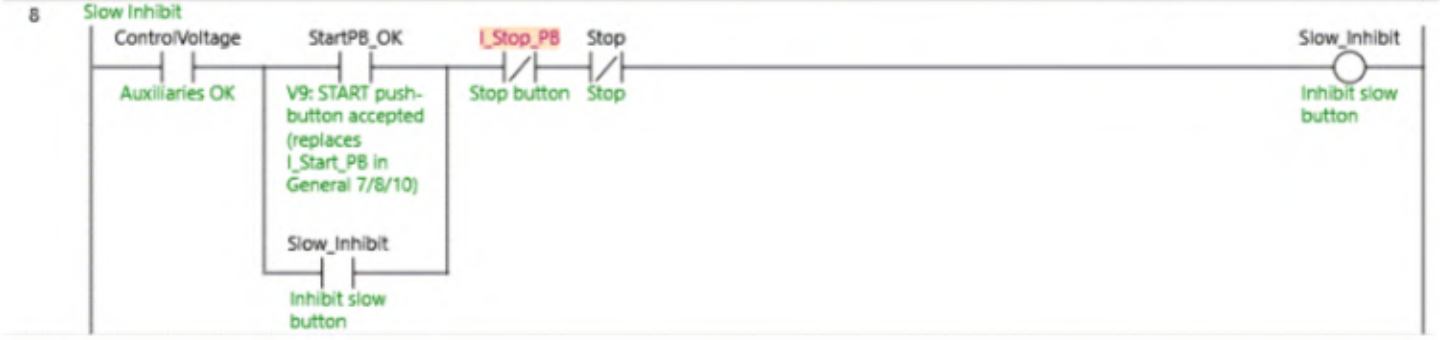
6 10 V DC source.

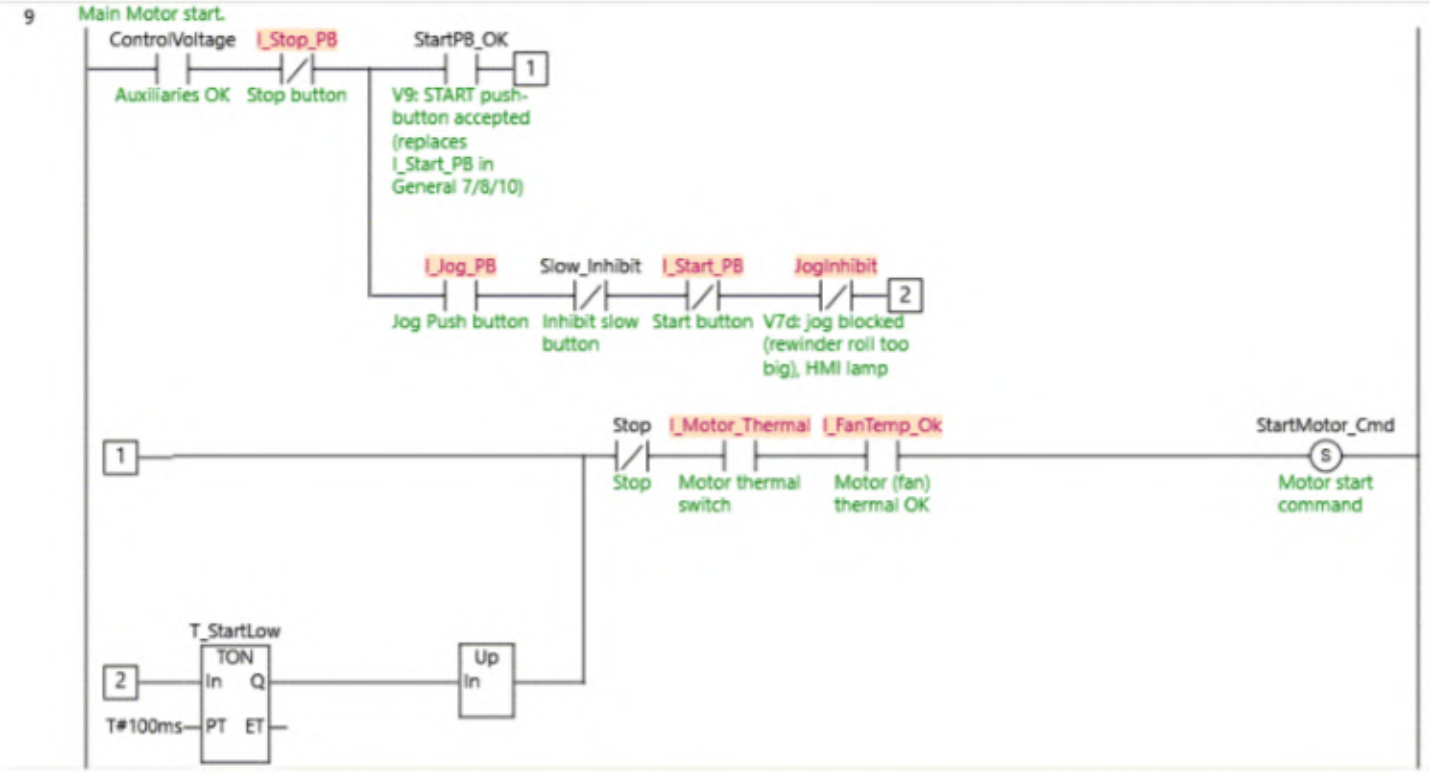


7 START blocked by JOG

```
1  // ==== V9: NEW General rung, INSERT between rung 6 ("10 V DC source") and
   rung 7
2  // ==== ("Slow_Inhibit"). ST rung, no contact in front. Comment: "V9: START blocked
   by JOG".
3  // START push-button is ignored while the JOG (slow speed) button is held and for
4  // 2 s after it is released. A START held during the block must be released and
5  // pressed again (no start by itself when the 2 s are over).
6  // Without this: START pressed during a jog sets Slow_Inhibit -> releasing the jog
7  // no longer stops (General rung 10) and VelSet jumps to the work speed.
8
9  T_JogStartBlk(In := I_Jog_PB, PT := T#2s);      // TOF: ON while held + 2 s
10 StartHeldBlk := (StartHeldBlk OR (I_Start_PB AND T_JogStartBlk.Q)) AND I_Start_PB;
11 StartPB_OK := I_Start_PB AND NOT T_JogStartBlk.Q AND NOT StartHeldBlk;
```

8 Slow Inhibit

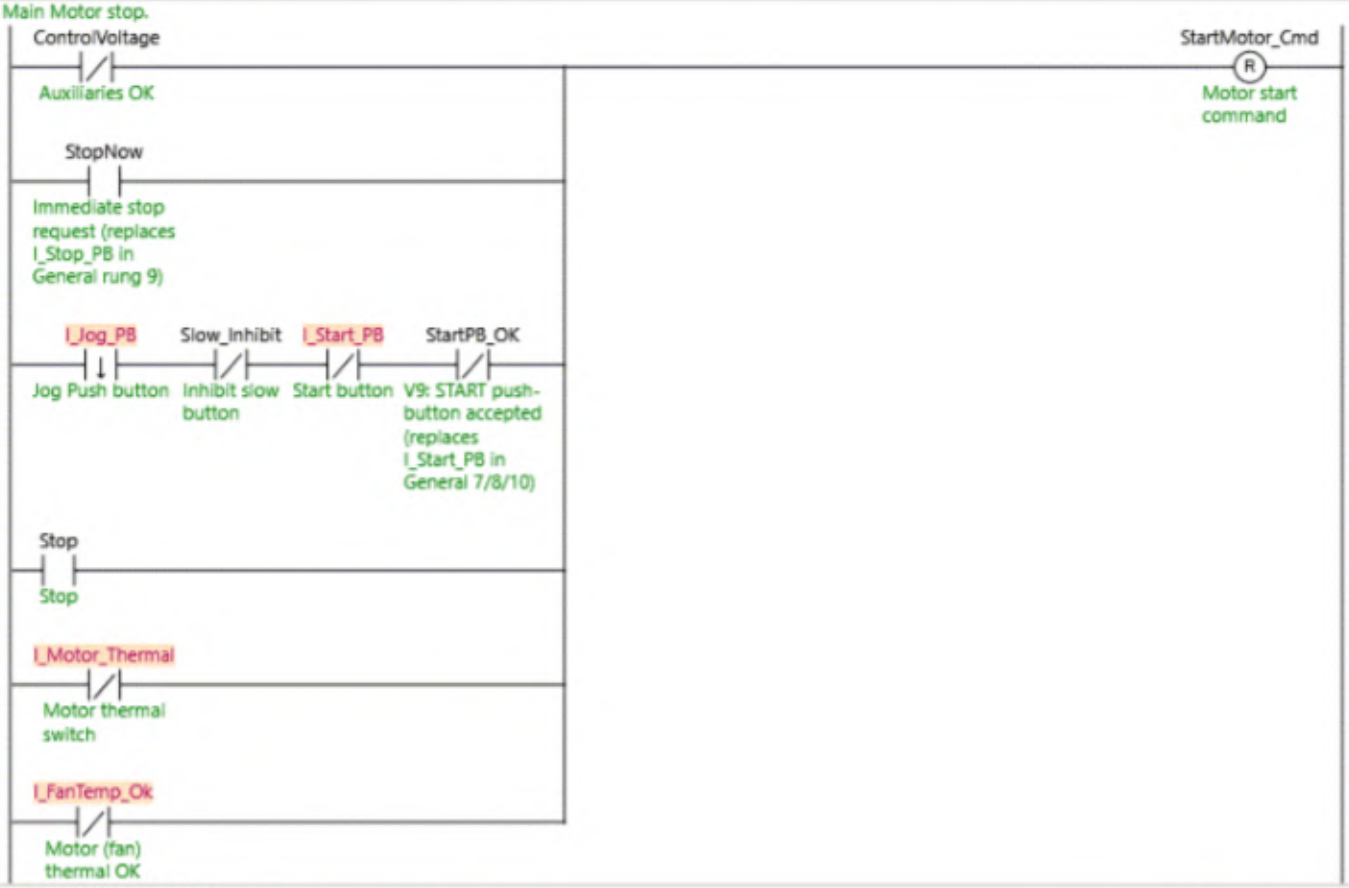



9
Main Motor start.
ControlVoltage
Auxiliaries OK
I_Stop_PB
Stop button
StartPB_OK
1
V9: START push-button accepted (replaces I_Start_PB in General 7/8/10)
I_Jog_PB
Jog Push button
Slow_Inhibit
Inhibit slow button
I_Start_PB
Start button
JogInhibit
2
V7d: jog blocked (rewinder roll too big), HMI lamp
1
Stop
Stop
I_Motor_Thermal
Motor thermal switch
I_FanTemp_Ok
Motor (fan) thermal OK
StartMotor_Cmd
S
Motor start command
T_StartLow
TON
2
In
Q
T#100ms
PT
ET
Up
In

10 Soft Stop Process

```
1  // v6m: STOP button = soft stop. StopNow replaces I_Stop_PB in rung 9.
2
3  StopNow := FALSE;
4  T_StopLong(In := I_Stop_PB, PT := T#1s);
5
6  IF NOT StartMotor_Cmd THEN
7     SoftStop := FALSE;
8  ELSIF I_Stop_PB AND NOT StopPB_Mem THEN          // STOP pressed (edge)
9
10    IF SoftStop OR NOT SoftStopEnable OR I_Jog_PB
11       OR (Velocity_M_min <= SoftStopEndSpd) THEN
12       StopNow := TRUE;                          // 2nd press / jog / slow / disabled
13    ELSE
14       SoftStop := TRUE;                         // start soft stop
15    END_IF;
16 END_IF;
17
18 StopPB_Mem := I_Stop_PB;
19
20 IF T_StopLong.Q THEN
21    StopNow := TRUE;                             // held 1 s = immediate
22 END_IF;
23
24 IF SoftStop AND I_Start_PB THEN
25    SoftStop := FALSE;                           // START cancels the soft stop
26 END_IF;
27
28 T_SoftStop(In := SoftStop, PT := T#120s);
29
30 IF SoftStop AND ((Velocity_M_min <= SoftStopEndSpd) OR T_SoftStop.Q) THEN
31    StopNow := TRUE;                             // almost stopped (or timeout)
32 END_IF;
```

11 Main Motor stop.



12 Hub extraction clearance.



13 Brake control.

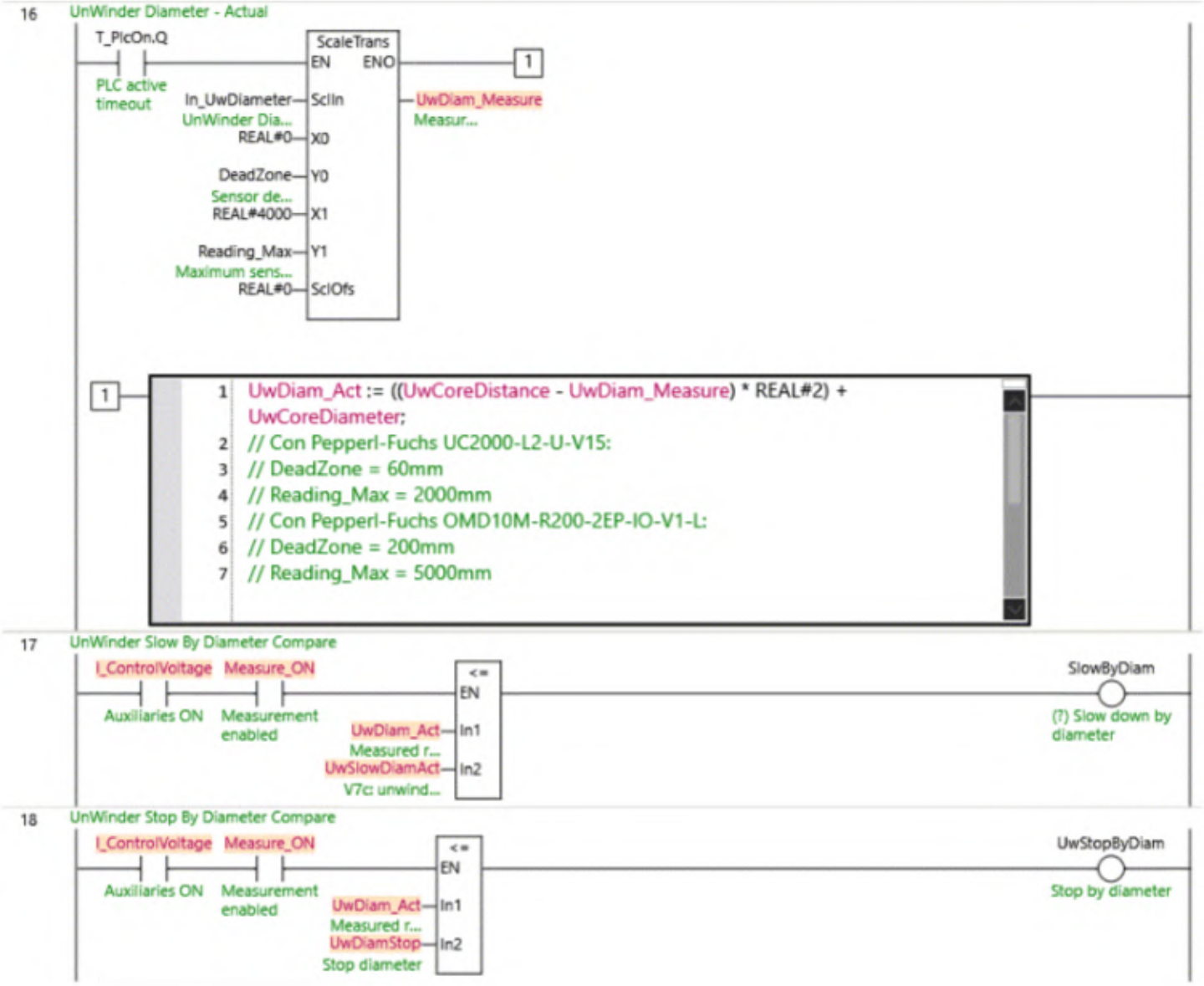


14 Cooling Fan control.



15 UnWinder Diameter - Input

```
1  In_UwDiameter := INT_TO_REAL(AI_Diam_Unwinder)/REAL#1;
```


16
UnWinder Diameter - Actual
T_PlcOn.Q
PLC active timeout
ScaleTrans
EN
ENO
1
In_UwDiameter
UnWinder Dia...
REAL#0
SclIn
X0
DeadZone
Sensor de...
REAL#4000
Y0
X1
Reading_Max
Maximum sens...
REAL#0
Y1
SclOfs
UwDiam_Measure
Measur...
1
1 UwDiam_Act := ((UwCoreDistance - UwDiam_Measure) * REAL#2) +
UwCoreDiameter;
2 // Con Pepperl-Fuchs UC2000-L2-U-V15:
3 // DeadZone = 60mm
4 // Reading_Max = 2000mm
5 // Con Pepperl-Fuchs OMD10M-R200-2EP-IO-V1-L:
6 // DeadZone = 200mm
7 // Reading_Max = 5000mm
17
UnWinder Slow By Diameter Compare
I_ControlVoltage
Auxiliaries ON
Measure_ON
Measurement enabled
<=
EN
UwDiam_Act
Measured r...
In1
UwSlowDiamAct
V7c: unwind...
In2
SlowByDiam
(?) Slow down by diameter
18
UnWinder Stop By Diameter Compare
I_ControlVoltage
Auxiliaries ON
Measure_ON
Measurement enabled
<=
EN
UwDiam_Act
Measured r...
In1
UwDiamStop
Stop diameter
In2
UwStopByDiam
Stop by diameter

1-8-1-1-3.Motor_Manager

0 Enable encoder reading.



1 Calculation of mm per encoder pulse.

```
IF Encoder_Pulse = INT#0 THEN;
    Encoder_Pulse :=INT#1;
END_IF;
// Encoder Standard 400mm
IF  EncoderWheel_Diam = REAL#0 THEN;
    EncoderWheel_Diam := REAL#400;
END_IF;

EncoderPulse_to_mm := EncoderWheel_Diam / (INT_TO_REAL(Encoder_Pulse) *
REAL#4);
```

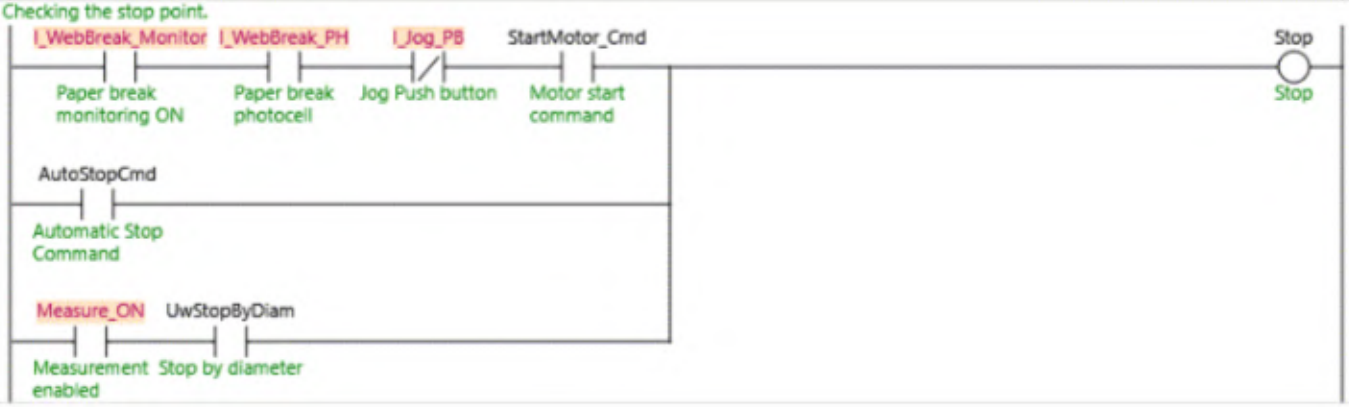
2 Reset encoder counter.



3 Checking the slowdown point.



4 Checking the stop point.



5 Check the maximum and minimum values for working speed and slow speed.

```
IF VelMax < REAL#600 THEN;
      VelMax := REAL#600;
   ELSIF VelMax > REAL#1000 THEN;
      VelMax := REAL#1000;
END_IF;
Vel_Max_mms := VelMax * REAL#1000 / REAL#60;

IF JogSpd_SP < REAL#10 THEN;
      JogSpd_SP := REAL#10;
ELSIF JogSpd_SP > REAL#70 THEN;
   JogSpd_SP := REAL#70;
END_IF;

// v6y: rewinder AUTO max speed, 0 = no limit

IF RwAutoMaxSpd > VelMax THEN
   RwAutoMaxSpd := VelMax;
ELSIF (RwAutoMaxSpd > REAL#0) AND (RwAutoMaxSpd < REAL#50) THEN
   RwAutoMaxSpd := REAL#50;
END_IF;



IF NOT StartMotor_Cmd THEN          // taken over at standstill only
   RwAutoLimAct := RwStopEnable AND (RwAutoMaxSpd > REAL#0);
   RwAutoMaxSpdAct := RwAutoMaxSpd;
END_IF;

// ---- V7: add at the END of Motor_Manager rung 5 ("Check the maximum and
minimum values...")
// ---- after the JogSpd_SP clamp, before Output_Ratio. Same style as the VelMax /
JogSpd clamps.
// V7: limits on settings that stop the machine or break the auto stop if 0 / absurd

IF SpdDelta_Percent < REAL#2 THEN                // fine band [m/min]: 0 = speed
hunting
   SpdDelta_Percent := REAL#2;
ELSIF SpdDelta_Percent > REAL#15 THEN
   SpdDelta_Percent := REAL#15;
END_IF;

IF AccelTrim < REAL#0.02 THEN                  // trim ramps: 0 = no speed correction
   AccelTrim := REAL#0.02;
ELSIF AccelTrim > REAL#0.5 THEN
   AccelTrim := REAL#0.5;
END_IF;

IF DecelTrim < REAL#0.02 THEN
   DecelTrim := REAL#0.02;
ELSIF DecelTrim > REAL#0.5 THEN
   DecelTrim := REAL#0.5;
END_IF;

IF AccelCoarse < REAL#0.2 THEN                  // coarse ramps: 0 = never reaches
speed
   AccelCoarse := REAL#0.2;
ELSIF AccelCoarse > REAL#4 THEN
   AccelCoarse := REAL#4;
```

```
55  END_IF;
56
57  IF DecelCoarse < REAL#0.2 THEN
58      DecelCoarse := REAL#0.2;
59  ELSIF DecelCoarse > REAL#4 THEN
60      DecelCoarse := REAL#4;
61  END_IF;
62
63  IF DecelWeb > REAL#0 THEN                    // V7 web ramps: 0 = old coarse ramp
    (allowed)
64      IF DecelWeb < REAL#50 THEN
65          DecelWeb := REAL#50;
66      ELSIF DecelWeb > REAL#1000 THEN
67          DecelWeb := REAL#1000;
68      END_IF;
69  END_IF;
70
71  IF AccelWeb > REAL#0 THEN
72      IF AccelWeb < REAL#50 THEN
73          AccelWeb := REAL#50;
74      ELSIF AccelWeb > REAL#1000 THEN
75          AccelWeb := REAL#1000;
76      END_IF;
77  END_IF;
78
79  IF SoftStopDecel > REAL#1500 THEN            // 0 = DecelCoarse (allowed)
80      SoftStopDecel := REAL#1500;
81  END_IF;
82
83  IF RwMaxDiam > REAL#1450 THEN                // rewinder MECHANICAL limit
    [mm]
84      RwMaxDiam := REAL#1450;
85  ELSIF RwMaxDiam < REAL#300 THEN
86      RwMaxDiam := REAL#300;
87  END_IF;
88
89  IF RwMaxSlowFallback >= RwMaxDiam THEN       // fallback slow-down below
    the max
90      RwMaxSlowFallback := RwMaxDiam - REAL#30;
91  END_IF;
92
93  // ---- V9 (V7d): add at the END of Motor_Manager rung 5, after the V7 clamps
94  // ---- (last lines now: RwMaxSlowFallback clamp), before Output_Ratio /
    Input_Ratio.
95  // V7d: jog / slow start button blocked on a big rewinder roll.
96  // RwJogMaxDiam (mm, HMI, Retain): 0 = no limit, else min 150.
97  // DT35 fault -> jog allowed (needed to thread / clear the machine).
98
99  IF (RwJogMaxDiam > REAL#0) AND (RwJogMaxDiam < REAL#150) THEN
100     RwJogMaxDiam := REAL#150;
101 END_IF;
102
103 JogInhibit := (RwJogMaxDiam > REAL#0) AND (RwDiam > RwJogMaxDiam) AND
    NOT RwSensFault;
104
105 Output_Ratio := (VelMax * Mt_To_mm) / UINT_TO_REAL(Analog_Res); //REAL#4000;
106 Input_Ratio := UINT_TO_REAL(Analog_Res) / VelMax ;
```

6 Reading the current value of the potentiometers. v6n: requested pot ranges, limited, applied only with the motor stopped !

```
1
2  IF WorkSpd_Max_Set < 50.0 THEN
3      WorkSpd_Max_Set := 50.0;
4  ELSIF WorkSpd_Max_Set > VelMax THEN
5      WorkSpd_Max_Set := VelMax;
6  END_IF;
7
8  IF FinalSpd_Max_Set < 5.0 THEN
9      FinalSpd_Max_Set := 5.0;
10 ELSIF FinalSpd_Max_Set > WorkSpd_Max_Set THEN
11     FinalSpd_Max_Set := WorkSpd_Max_Set;
12 END_IF;
13
14 IF NOT StartMotor_Cmd THEN
15     WorkSpd_Max := WorkSpd_Max_Set;          // motor stopped: apply
16     FinalSpd_Max := FinalSpd_Max_Set;
17 END_IF;
18
19 SpdMax_Pending := (WorkSpd_Max <> WorkSpd_Max_Set)
20             OR (FinalSpd_Max <> FinalSpd_Max_Set)
21             OR (RwAutoLimAct <> (RwStopEnable AND (RwAutoMaxSpd > REAL#0)))
22             OR (RwAutoLimAct AND (RwAutoMaxSpdAct <> RwAutoMaxSpd));
23
24 // v6i: pots scaled to their own full scale (m/min)
25
26 IF Pot1_Work_Velocity > INT#5 THEN
27     Pot1_Mem := REAL_TO_INT(INT_TO_REAL(Pot1_Work_Velocity) * WorkSpd_Max /
   VelMax);
28 ELSE
29     Pot1_Mem := INT#0;
30 END_IF;
31
32 IF Pot2_Slow_Velocity > INT#5 THEN
33     Pot2_Mem := REAL_TO_INT(INT_TO_REAL(Pot2_Slow_Velocity) * FinalSpd_Max /
   VelMax);
34 ELSE
35     Pot2_Mem := INT#0;
36 END_IF;
```

7 Updating the set speed.

```
IF ManMode = FALSE THEN;

    Speed_Display := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;
    LowSpd_Display := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;

  // v6y: rewinder AUTO stop -> work speed limited

  IF RwAutoLimAct AND (Speed_Display > RwAutoMaxSpdAct) THEN
     Speed_Display := RwAutoMaxSpdAct;
  END_IF;

ELSE;
  Speed_Display := Pot1_Work_Velocity / (Analog_Res / UINT#40); // v6i: raw pot, %
  LowSpd_Display := Pot2_Slow_Velocity / (Analog_Res / UINT#40);

END_IF;
```

8 Updating potentiometer value.

ManMode (Manual mode) I_Jog_PB (Jog Push button) 1

```
IF SoftStop THEN
   VelSet := 0.0;                    // v6m: soft stop
ELSIF SlowDown = FALSE  THEN
   VelSet := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;

   // v6y: rewinder AUTO stop -> work speed limited

   IF RwAutoLimAct AND (VelSet > RwAutoMaxSpdAct) THEN
      VelSet := RwAutoMaxSpdAct;
   END_IF;

ELSE
   VelSet := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;
END_IF;

VelSet_mms := VelSet * Mt_To_mm;
```

9 Slow speed.

ManMode (Manual mode) I_Jog_PB (Jog Push button) Slow_Inhibit (Inhibit slow button) 1

```
VelSet := JogSpd_SP;
VelSet_mms := VelSet * Mt_To_mm;
```

10 Check the difference between the set speed and the detected speed.

```
//SpdDelta := VelSet_mms * SpdDelta_Percent / REAL#100;
SpdDelta := SpdDelta_Percent;
SpdDeltaMax := VelSet + SpdDelta;
SpdDeltaMin := VelSet - SpdDelta;
// Too fast
IF Velocity_M_min > SpdDeltaMax THEN;
    OverSpeed := TRUE;
ELSE
    OverSpeed := FALSE;
END_IF;
// Too slow
IF Velocity_M_min < SpdDeltaMin THEN;
    UnderSpeed := TRUE;
ELSE
    UnderSpeed := FALSE;
END_IF;
```

11 Speed calculation.

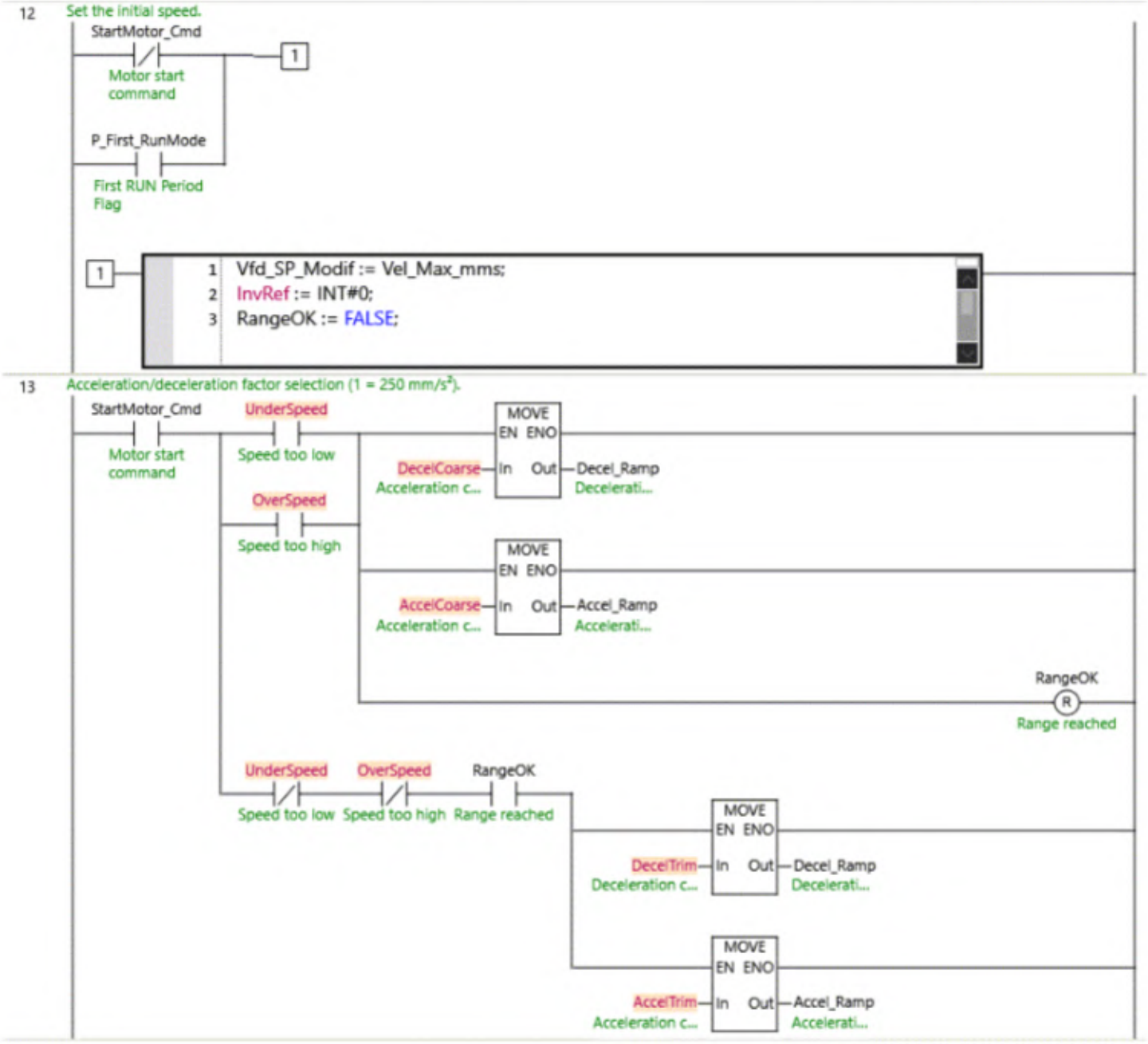
```
// ==== V9: REPLACE Motor_Manager rungs 11 ("Read the current value of the encoder", MOVE on
// ==== Clock100ms rising edge) and 12 ("Speed calculation", Clock100ms falling edge) by ONE
// ==== ST rung with this code (no contact in front). Comment: "V9: speed over exactly 100 ms".
// Old: window = time between the rising and falling edge of Clock100ms, assumed 50 ms,
// but with the 4 ms PrimaryTask it is 12 or 13 scans (48 / 52 ms) -> the speed jumps
// -4 % / +4 % (418 set: display 397 / 431).
// New: exactly 25 scans = 100 ms (same as AutoStop SpdAct_Pps), updated 10 times/s.

SpdScanCnt := SpdScanCnt + 1;

IF SpdScanCnt >= 25 THEN
    SpdScanCnt := 0;
    DeltaPulse := Encoder_Pos - MemPos;
    MemPos := Encoder_Pos;

    IF ABS(DeltaPulse) > 100000 THEN            // counter reset (Encoder_Reset)
        DeltaPulse := 0;
    END_IF;

    Velocity := DINT_TO_REAL(DeltaPulse) * EncoderPulse_to_mm * REAL#10;   // mm/s
    Velocity_M_min := Velocity / Mt_To_mm;
END_IF;
```


12
Set the initial speed.
StartMotor_Cmd
Motor start command
1
P_First_RunMode
First RUN Period Flag
1
1 Vfd_SP_Modif := Vel_Max_mms;
2 InvRef := INT#0;
3 RangeOK := FALSE;
13
Acceleration/deceleration factor selection (1 = 250 mm/s²).
StartMotor_Cmd
Motor start command
UnderSpeed
Speed too low
OverSpeed
Speed too high
MOVE
EN ENO
DecelCoarse
Acceleration c...
In Out
Decel_Ramp
Decelerati...
MOVE
EN ENO
AccelCoarse
Acceleration c...
In Out
Accel_Ramp
Accelerati...
RangeOK
R
Range reached
UnderSpeed
Speed too low
OverSpeed
Speed too high
RangeOK
Range reached
MOVE
EN ENO
DecelTrim
Deceleration c...
In Out
Decel_Ramp
Decelerati...
MOVE
EN ENO
AccelTrim
Acceleration c...
In Out
Accel_Ramp
Accelerati...

14 Ratio drive counts per m/min of web at the current rewinder diameter.

```
// ==== V9: REPLACE the code of Motor_Manager rung 15 ("Ratio drive counts per m/min of web
// ==== at the current rewinder diameter." = V7 rung A) with this file. (V7b jog line inside.)
// Ratio drive counts per m/min of web at the current rewinder diameter.
// The motor drives the rewinder: web speed per Hz grows with the diameter, so
// counts per m/min = RwRatioD / diameter. RwRatioD (Retain) is adapted by rung B.

IF (RwRatioD > 0.0) AND (DecelDiam >= 100.0) THEN
    WebRatio := RwRatioD / DecelDiam;
ELSE
    WebRatio := 0.0;
END_IF;

// Fallback only (FFEnable = FALSE): old integrating loop (rung 16) with the coarse
// ramps in real web mm/s2. With FFEnable = TRUE, rung B drives the reference.
// V7b: never in jog / slow button -> jog keeps the rung 14 ramps exactly as v6y.

IF NOT FFEnable AND NOT I_Jog_PB AND StartMotor_Cmd AND (UnderSpeed OR OverSpeed)
   AND (WebRatio > 0.0) THEN

   IF DecelWeb > 0.0 THEN
      Decel_Ramp := DecelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
      IF Decel_Ramp > 8.0 THEN
         Decel_Ramp := 8.0;
      END_IF;
   END_IF;

   IF AccelWeb > 0.0 THEN
      Accel_Ramp := AccelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
      IF Accel_Ramp > 8.0 THEN
         Accel_Ramp := 8.0;
      END_IF;
   END_IF;
END_IF;
```

15 Soft Stop - SoftStopDecel (mm/s2), same for every roll.

```
// v6r: soft stop at a REAL web decel SoftStopDecel (mm/s2), same for every roll.
// Ratio drive counts / web m/min frozen at the soft stop start (encoder = exact).

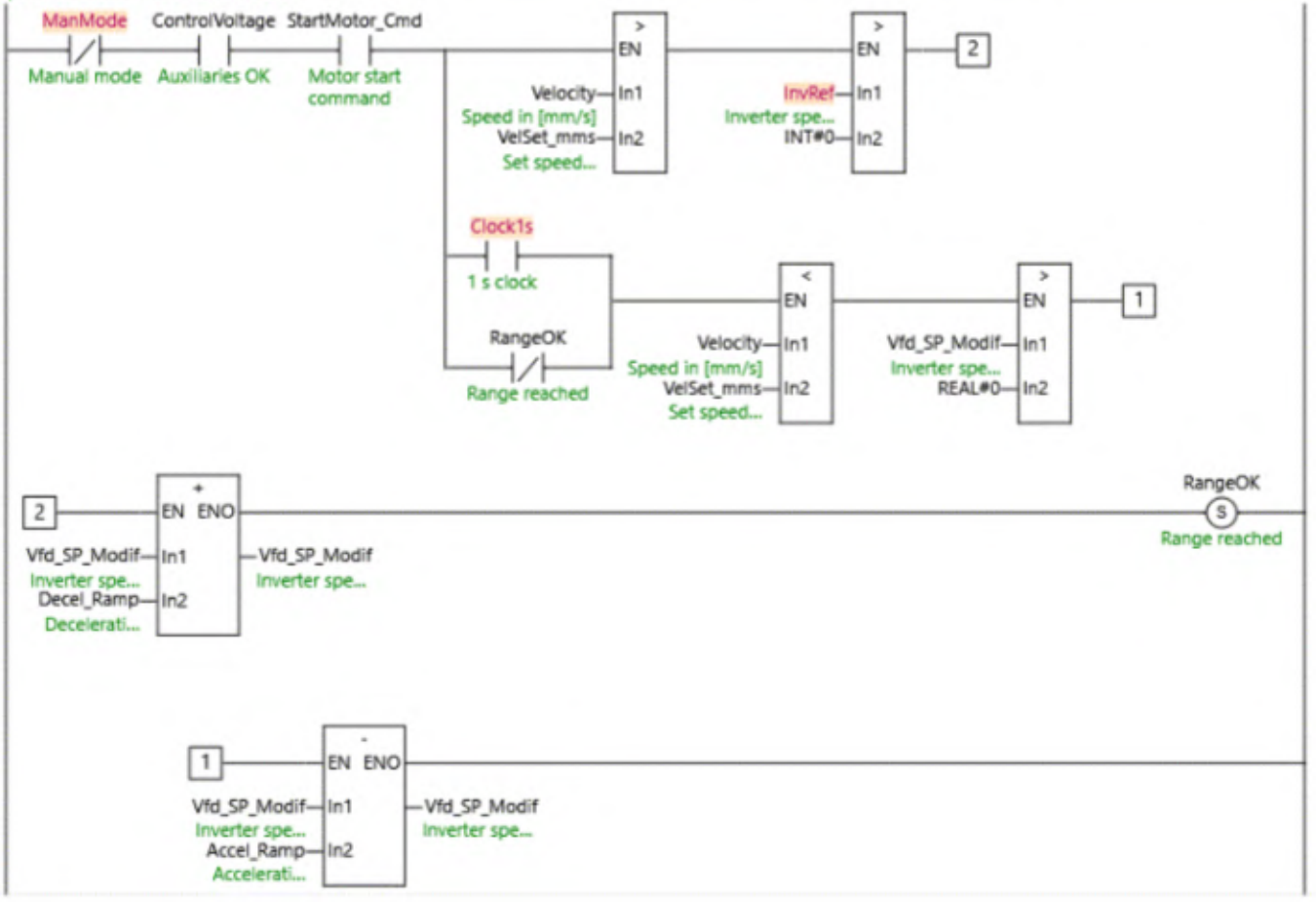
IF SoftStop AND NOT SoftStopMem THEN
    IF Velocity_M_min > 1.0 THEN
        SoftStopRatio := INT_TO_REAL(InvRef) / Velocity_M_min;
    ELSE
        SoftStopRatio := 0.0;
    END_IF;
END_IF;

SoftStopMem := SoftStop;

IF SoftStop AND (SoftStopDecel > 0.0) THEN
    IF SoftStopRatio > 0.0 THEN
        Decel_Ramp := SoftStopDecel * 0.06 * SoftStopRatio * Vel_Max_mms /
1000000.0;
    ELSE
        Decel_Ramp := SoftStopDecel / 250.0;          // fallback (old v6m behaviour)
    END_IF;

    IF Decel_Ramp > 8.0 THEN
        Decel_Ramp := 8.0;                    // max 9 Hz/s asked from the PLC
    END_IF;
END_IF;
```

16 Speed correction factor calculation.



17 Slow adaptation of RwRatioD on the relative speed error.

```
// ==== V9: REPLACE the code of Motor_Manager rung 18 ("Slow adaptation of RwRatioD on the
// ==== relative speed error." = V7 rung B) with this file. (V7a + V7b + V7f + V9 inside.)
// The PLC sends the drive STRAIGHT to the frequency of the set speed
// (VelSet x WebRatio), along a ramp in REAL web mm/s2 that does not depend on the
// measured speed -> no hunting, whatever the roll inertia / web delay.
// The measured speed only adapts RwRatioD slowly (~10 s), once at target.
// It overwrites Vfd_SP_Modif after rung 16, so rung 16 has no effect in AUTO.
// FFEnable = FALSE -> old loop (rung 16) exactly as before.
// V7b: not in jog / slow button (I_Jog_PB): at 10 m/min the feed-forward gives
// only ~0.4 Hz, too little to move the web -> the old loop (rung 16) runs the jog.
FFActive := FFEnable AND NOT ManMode AND NOT I_Jog_PB AND StartMotor_Cmd
        AND ControlVoltage AND (WebRatio > 0.0) AND (Vel_Max_mms > 0.0);

IF NOT StartMotor_Cmd THEN
    RampRef := 0.0;
    FFRate := 0.0;
    FFSteadyTicks := 0;
    FFSlowAcc := FALSE;
END_IF;
// V9: START during a soft stop: the decel flattens and the speed is HELD
// (SpdHold, HMI lamp) until START is pushed AGAIN -> gentle ramp up at
// SoftCancelAccel (mm/s2) to the set speed. The hold only blocks speeding up:
// a lower target (pot down, auto slow-down, RwAutoMaxSpd) still slows down.
// STOP during the hold = new soft stop (General rung 9).
IF (SoftCancelAccel > 0.0) AND (SoftCancelAccel < 20.0) THEN
    SoftCancelAccel := 20.0;
ELSIF SoftCancelAccel > 1000.0 THEN
    SoftCancelAccel := 1000.0;
END_IF;
FFStartEdge := I_Start_PB AND NOT FFStartMem;
FFStartMem := I_Start_PB;
IF FFSoftMem AND NOT SoftStop AND StartMotor_Cmd AND FFActive THEN   // soft stop cancelled
    SpdHold := TRUE;
    FFSlowAcc := FALSE;
ELSIF SpdHold AND FFStartEdge THEN                  // 2nd START: resume
    SpdHold := FALSE;
    FFSlowAcc := TRUE;
END_IF;
IF SoftStop OR NOT StartMotor_Cmd OR NOT FFActive THEN    // FFEnable OFF: old loop
    SpdHold := FALSE;
    FFSlowAcc := FALSE;
END_IF;
FFSoftMem := SoftStop;
// V7f: S-curve time (s), 0 = off, max 2 s
IF SCurveTime < 0.0 THEN
    SCurveTime := 0.0;
ELSIF SCurveTime > 2.0 THEN
    SCurveTime := 2.0;
END_IF;

IF FFActive THEN
    IF NOT FFWasActive THEN
        RampRef := Vel_Max_mms - Vfd_SP_Modif;          // bumpless start / switch-
```

```
over
54          FFRate := 0.0;
55      END_IF;
56
57      // Target reference (mm/s of the 0..10 V scale) for the set speed
58      FFTarget := VelSet * WebRatio * Output_Ratio;
59      IF FFTarget > Vel_Max_mms THEN
60          FFTarget := Vel_Max_mms;
61      ELSIF FFTarget < 0.0 THEN
62          FFTarget := 0.0;
63      END_IF;
64
65      // V9: hold after a cancelled soft stop (the S-curve flattens it smoothly):
66      // never above the present reference, lower targets still allowed
67      IF SpdHold THEN
68          IF FFTarget > RampRef THEN
69              FFTarget := RampRef;
70          END_IF;
71      ELSIF FFSlowAcc AND (ABS(FFTarget - RampRef) <= FFTarget * 0.005 + 2.0) THEN
72          FFSlowAcc := FALSE;                         // back at the set speed
73      END_IF;
74
75      // Ramp steps per 4 ms scan: web mm/s2 -> reference mm/s per scan
76      IF FFSlowAcc AND (SoftCancelAccel > 0.0) THEN
77          StepAcc := SoftCancelAccel * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;   //
    V9
78      ELSIF AccelWeb > 0.0 THEN
79          StepAcc := AccelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
80      ELSE
81          StepAcc := AccelCoarse;
82      END_IF;
83      IF SoftStop AND (SoftStopDecel > 0.0) THEN
84          StepDec := SoftStopDecel * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
85      ELSIF DecelWeb > 0.0 THEN
86          StepDec := DecelWeb * 0.06 * WebRatio * Vel_Max_mms / 1000000.0;
87      ELSE
88          StepDec := DecelCoarse;
89      END_IF;
90      IF StepAcc > 8.0 THEN
91          StepAcc := 8.0;
92      END_IF;
93      IF StepDec > 8.0 THEN
94          StepDec := 8.0;
95      END_IF;
96
97      // Ramp generator
98      // V7f: S-curve. FFRate (reference per scan) changes at most FFJerk per scan,
99      // so the full ramp is reached after SCurveTime (s) and is rounded off the
100     // same way before the target -> the T-one / brake air can follow.
101     // SCurveTime = 0 -> plain linear ramp as V7a.
102     IF SCurveTime > 0.0 THEN
103         IF (FFRate > 0.0) OR ((FFRate = 0.0) AND (FFTarget > RampRef)) THEN
104             FFJerk := StepAcc * 0.004 / SCurveTime;
105         ELSE
106             FFJerk := StepDec * 0.004 / SCurveTime;
107         END_IF;
108
109         // rate allowed so that it can still fall to 0 at the target
110
```

```
111   IF FFTarget >= RampRef THEN
112       FFSqrtArg := REAL#2.0 * FFJerk * (FFTarget - RampRef);
113       FFRateSet := SQRT(FFSqrtArg);
114
115       IF FFRateSet > StepAcc THEN
116         FFRateSet := StepAcc;
117       END_IF;
118     ELSE
119       FFSqrtArg := REAL#2.0 * FFJerk * (RampRef - FFTarget);
120       FFRateSet := SQRT(FFSqrtArg);
121
122       IF FFRateSet > StepDec THEN
123         FFRateSet := StepDec;
124       END_IF;
125
126       FFRateSet := -FFRateSet;
127     END_IF;
128
129     IF FFRateSet > FFRate + FFJerk THEN
130       FFRate := FFRate + FFJerk;
131     ELSIF FFRateSet < FFRate - FFJerk THEN
132       FFRate := FFRate - FFJerk;
133     ELSE
134       FFRate := FFRateSet;
135     END_IF;
136
137     RampRef := RampRef + FFRate;
138
139     IF (ABS(FFTarget - RampRef) <= 3.0) AND (ABS(FFRate) <= 2.0 * FFJerk) THEN
140       RampRef := FFTarget;                    // at target (pot noise)
141       FFRate := 0.0;
142     END_IF;
143   ELSE
144     FFRate := 0.0;
145
146     IF RampRef < FFTarget - StepAcc THEN
147       RampRef := RampRef + StepAcc;
148     ELSIF RampRef > FFTarget + StepDec THEN
149       RampRef := RampRef - StepDec;
150     ELSE
151       RampRef := FFTarget;                    // at target
152     END_IF;
153   END_IF;
154
155   // V7a: "at target" = within 0.5 % of the target (the pot and RwDiam noise
156
157   // move FFTarget by a few counts every scan -> the counter never reached 500)
158
159   IF (ABS(FFTarget - RampRef) <= FFTarget * 0.005 + 2.0)
160     AND NOT SpdHold AND NOT FFSlowAcc THEN          // V9: not during the
      hold
161     IF FFSteadyTicks < 30000 THEN
162       FFSteadyTicks := FFSteadyTicks + 1;
163     END_IF;
164   ELSE
165     FFSteadyTicks := 0;
166   END_IF;
167
168   // Slow adaptation of RwRatioD on the relative speed error (2 s after the
```

```
    // Slow adaptation of RwRatioD on the relative speed error (2 s after the
    // target is reached, not at the 60 Hz limit, not during soft stop / jog)

    IF (FFSteadyTicks >= 500) AND NOT SoftStop AND NOT I_Jog_PB
      AND (VelSet > 15.0) AND (Velocity_M_min > 10.0) AND NOT RwSensFault
      AND (FFTarget < Vel_Max_mms * 0.98) THEN
       FFErr := (VelSet - Velocity_M_min) / VelSet;
       IF FFErr > 0.05 THEN
          FFErr := 0.05;
       ELSIF FFErr < -0.05 THEN
          FFErr := -0.05;
       END_IF;
       IF ABS(FFErr) > 0.003 THEN                  // dead band 0.3 % (encoder noise)
          RwRatioD := RwRatioD * (1.0 + 0.0004 * FFErr);  // ~10 s time constant
       END_IF;
       IF RwRatioD < 1000.0 THEN
          RwRatioD := 1000.0;
       ELSIF RwRatioD > 8000.0 THEN
          RwRatioD := 8000.0;
       END_IF;
    END_IF;

    Vfd_SP_Modif := Vel_Max_mms - RampRef;             // rung 17 -> InvRef
END_IF;
FFWasActive := FFActive;

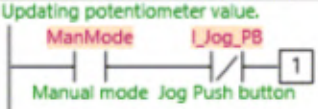
// V7f: T-one decel signal (for a T-one "stop / boost" input, spare PLC output):
// ON while the feed-forward ramps down (more than 1 % above the target, so the
// slow drift of the target with the diameter does not pulse it) or soft stop.
TOneDecel := (FFActive AND (RampRef - FFTarget > FFTarget * 0.01 + 5.0))
```

18 Management of the inverter speed reference in mm/s and conversion to analog points (0–4000 points = 0–10 VDC = 0–10,000 mm/s).




```
1  //InvRef2 := REAL_TO_UINT((Vel_Max_mms - Vfd_SP_Modif ) / Output_Ratio);
2  //InvRefWW := UINT_TO_WORD(InvRef2);
3  //InvRefSW :=Swap(UINT_TO_WORD(InvRef2));
4  //InvRef := WORD_TO_UINT(InvRefSW);
5  InvRef := REAL_TO_INT((Vel_Max_mms - Vfd_SP_Modif ) / Output_Ratio);
```

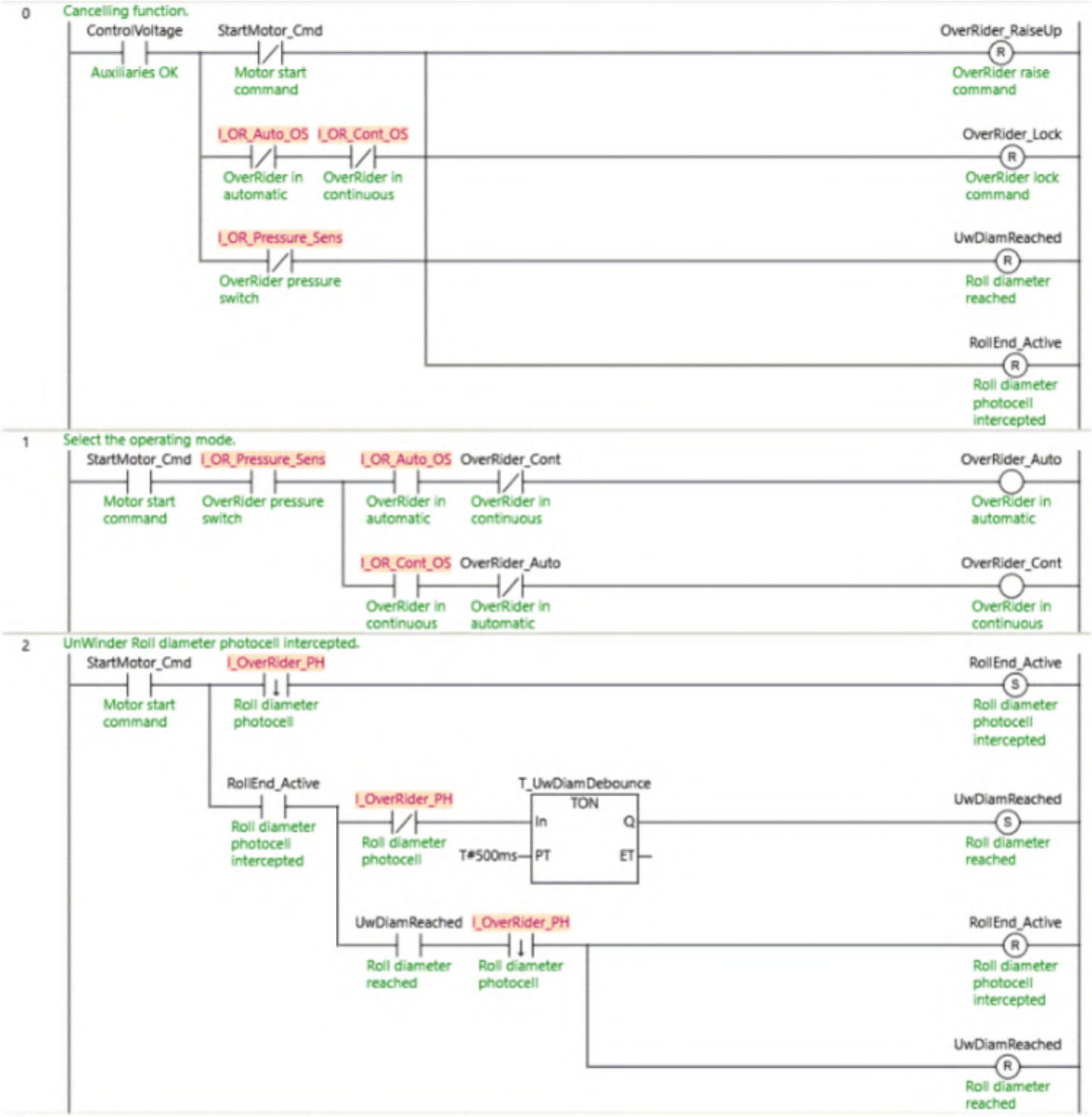
19 Updating potentiometer value.

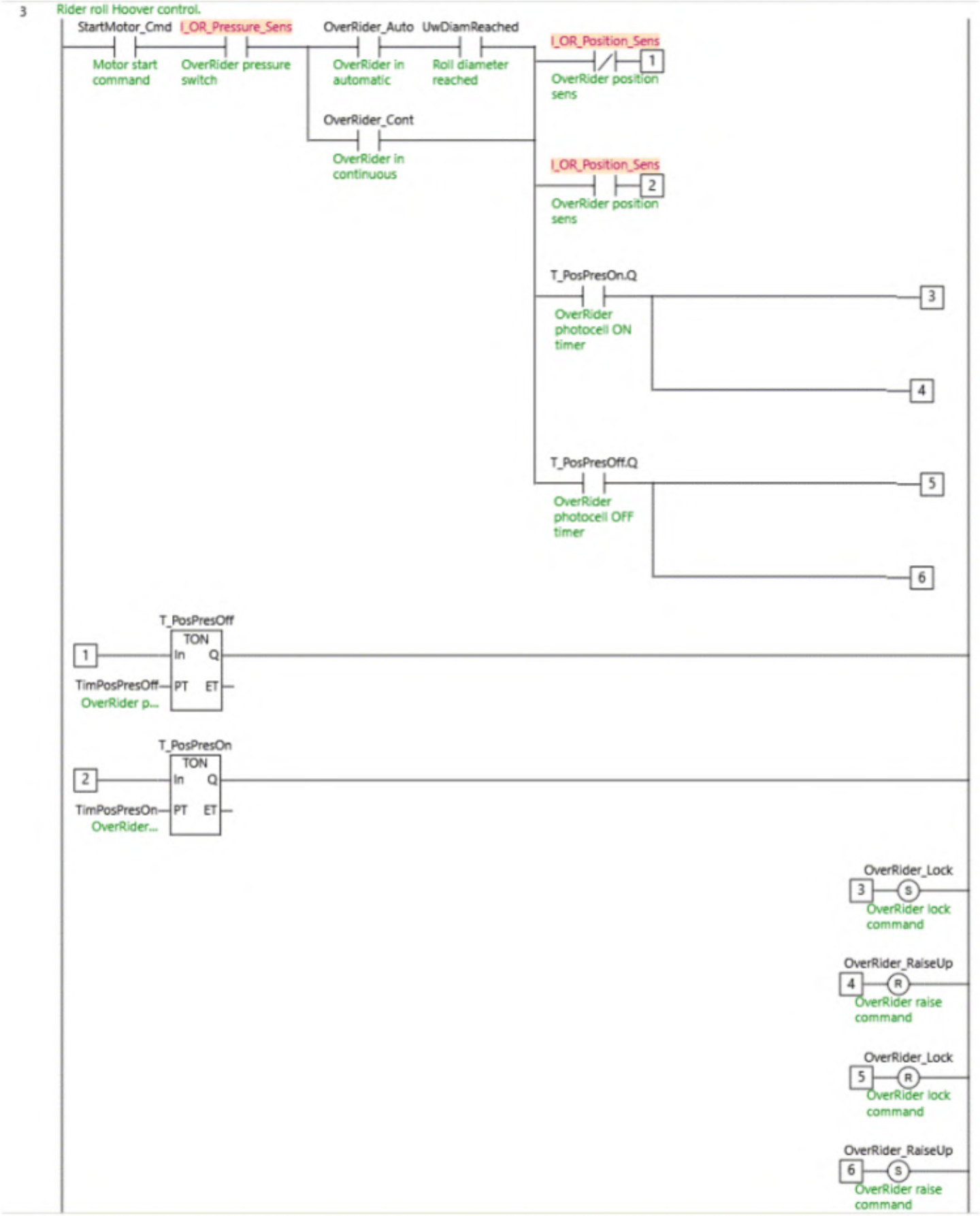



```
1  IF SoftStop THEN
2     InvRef := INT#0;                    // v6m: drive ramps down on its own
3  ELSIF SlowDown = FALSE THEN
4     InvRef := Pot1_Work_Velocity;
5  ELSE
6     InvRef := Pot2_Slow_Velocity;
7  END_IF;
```


20
Slow speed.
ManMode
I_Jog_PB
Slow_Inhibit
1
Manual mode
Jog Push button
Inhibit slow button
1
1 VelSet := JogSpd_SP;
2 VelSet_mms := VelSet * Mt_To_mm;
3 //InvRef2 := REAL_TO_UINT((VelSet_mms) / Output_Ratio);
4 //InvRefWW := UINT_TO_WORD(InvRef2);
5 //InvRefSW :=Swap(UINT_TO_WORD(InvRef2));
6 //InvRef := WORD_TO_UINT(InvRefSW);
7
8 InvRef := REAL_TO_INT(VelSet_mms / Output_Ratio);

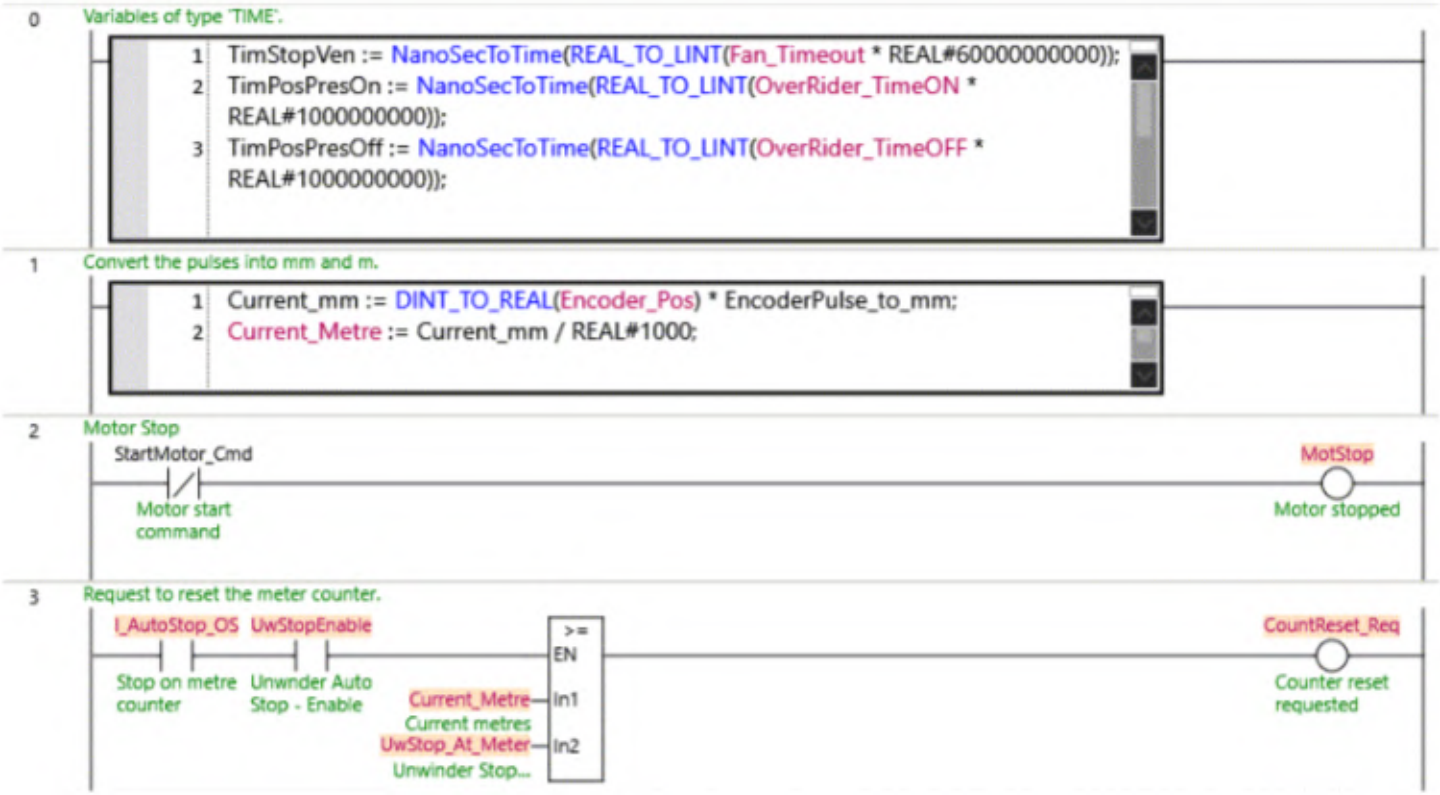
1-8-1-1-4.OverRider_Manager

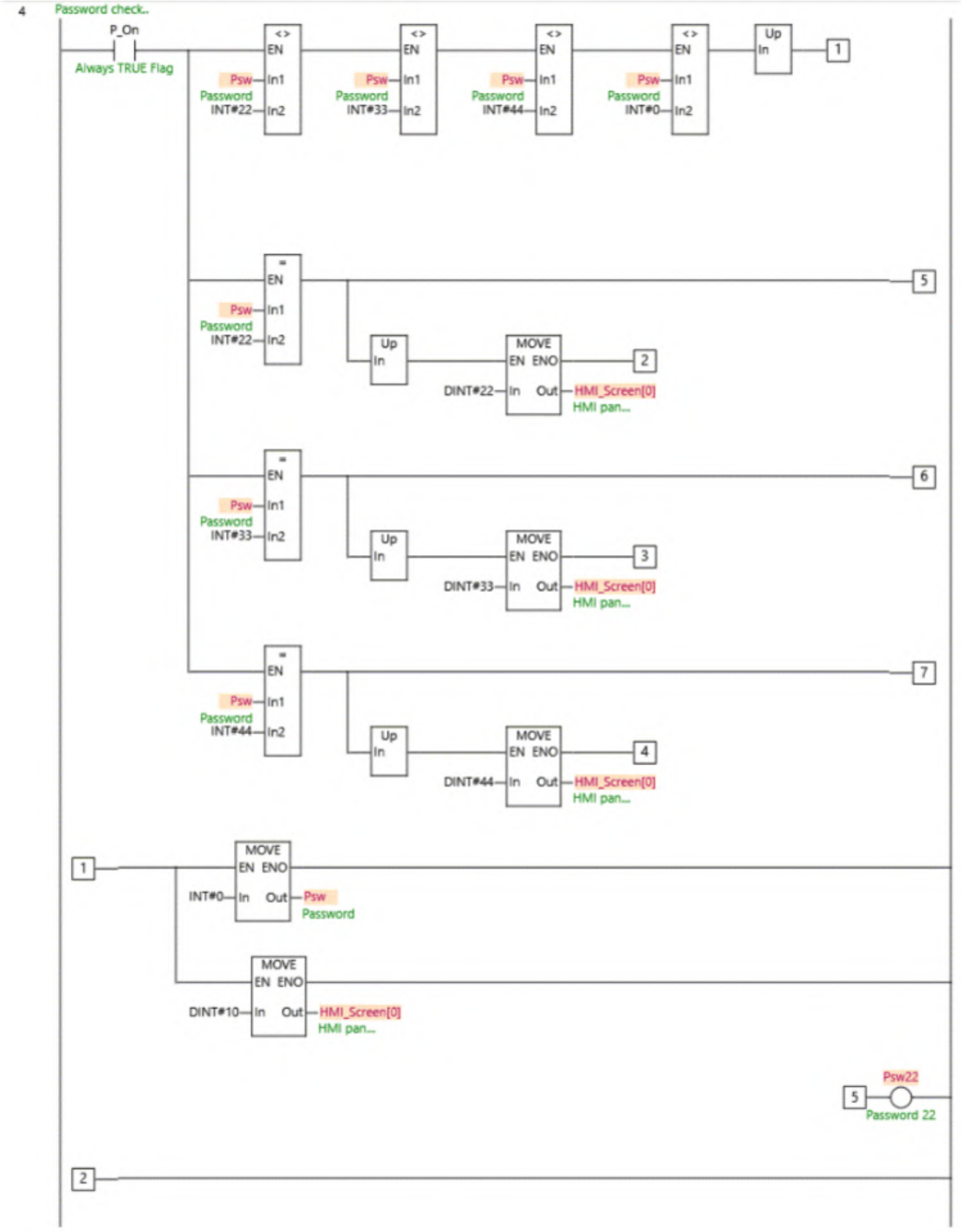


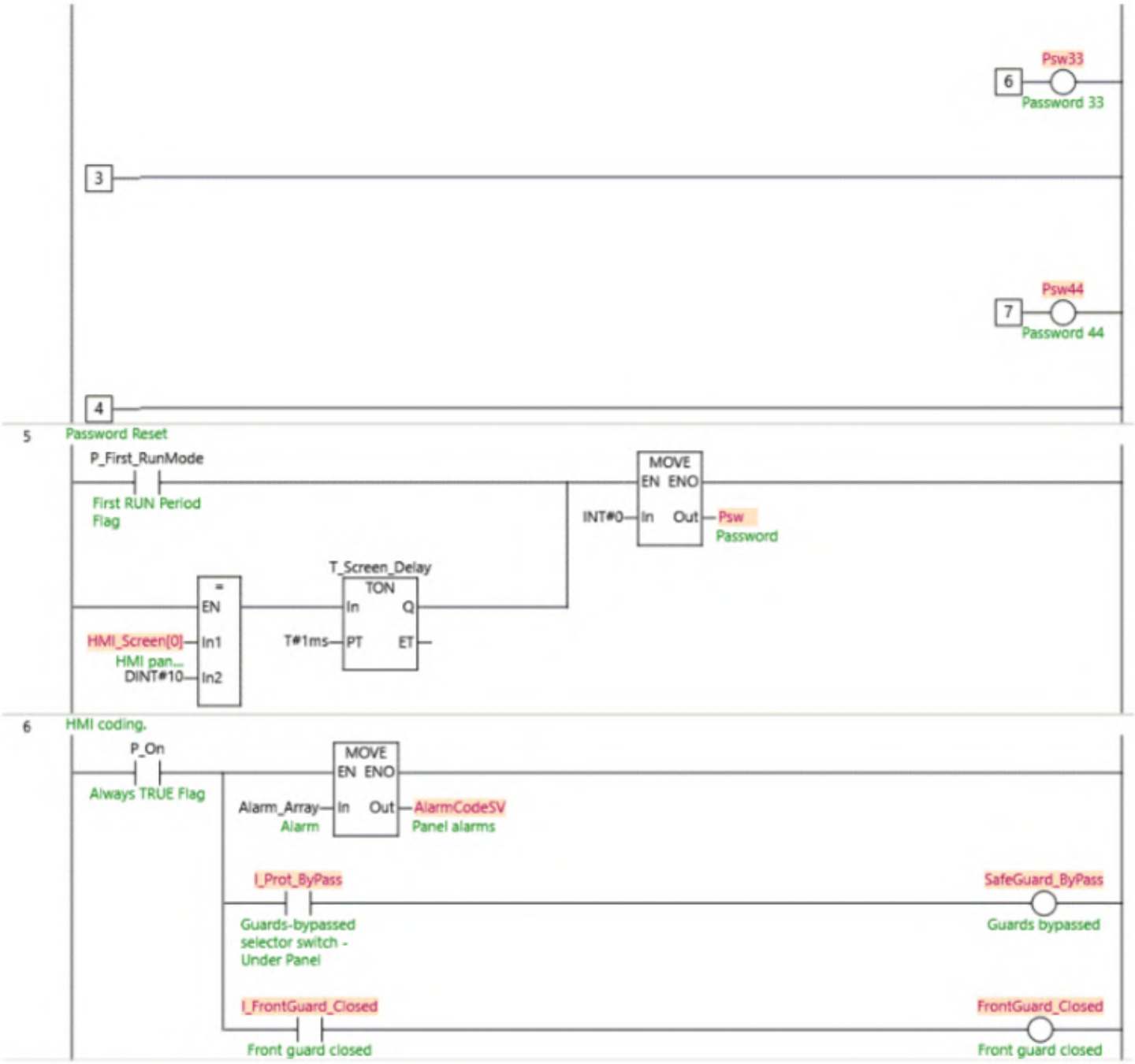

3
Rider roll Hoover control.
StartMotor_Cmd
Motor start command
I_OR_Pressure_Sens
OverRider pressure switch
OverRider_Auto
OverRider in automatic
UwDiamReached
Roll diameter reached
I_OR_Position_Sens
1
OverRider position sens
OverRider_Cont
OverRider in continuous
I_OR_Position_Sens
2
OverRider position sens
T_PosPresOn.Q
OverRider photocell ON timer
3
4
T_PosPresOff.Q
OverRider photocell OFF timer
5
6
T_PosPresOff
TON
In
Q
PT
ET
1
TimPosPresOff
OverRider p...
T_PosPresOn
TON
In
Q
PT
ET
2
TimPosPresOn
OverRider...
OverRider_Lock
3
S
OverRider lock command
OverRider_RaiseUp
4
R
OverRider raise command
OverRider_Lock
5
R
OverRider lock command
OverRider_RaiseUp
6
S
OverRider raise command


4
Rider roll raise command.
ControlVoltage
Auxiliaries OK
OverRider_RaiseUp
S
OverRider raise command
5
I_ControlVoltage
Auxiliaries ON
OverRider_RaiseUp
R
OverRider raise command

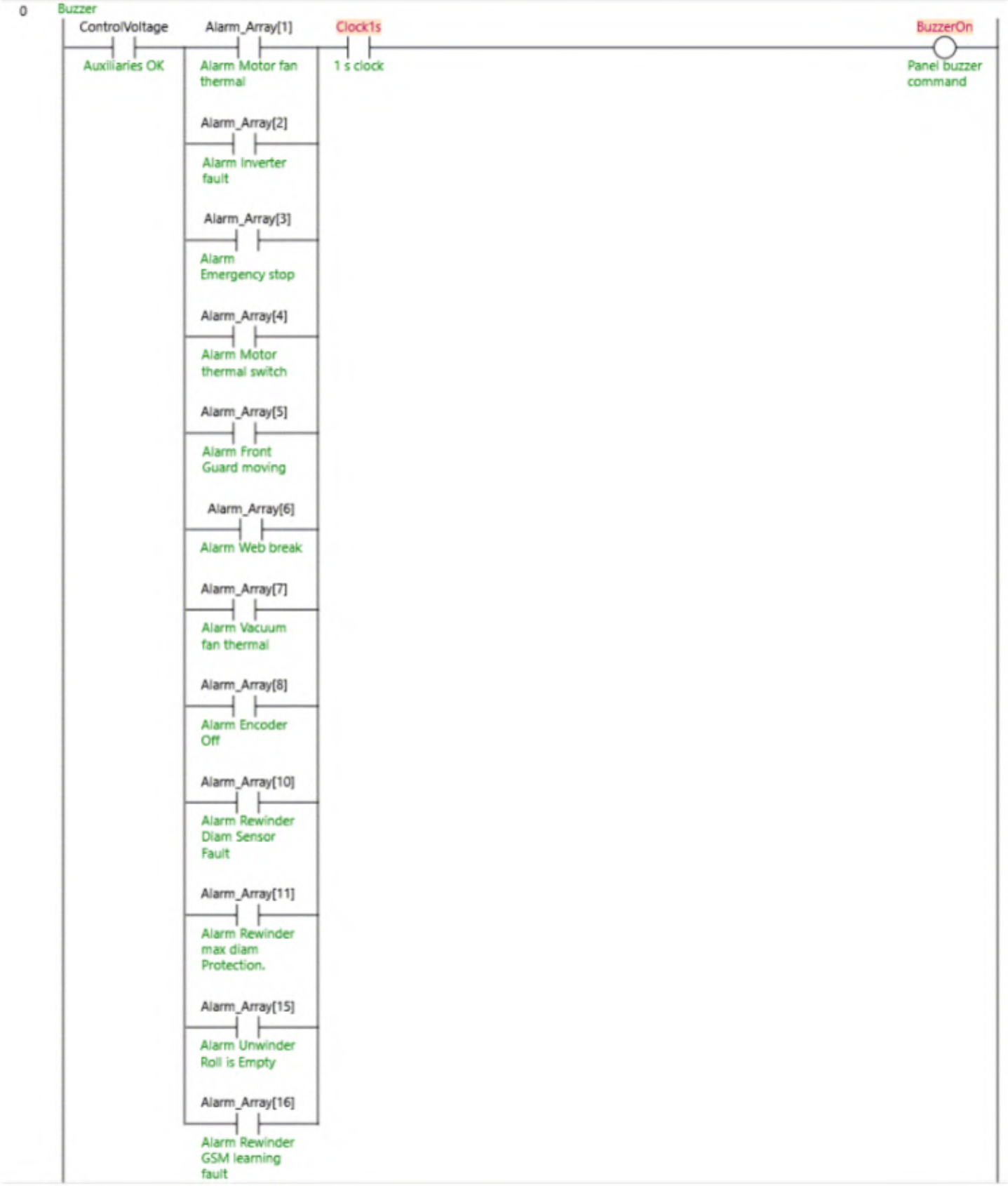
1-8-1-1-5.Console



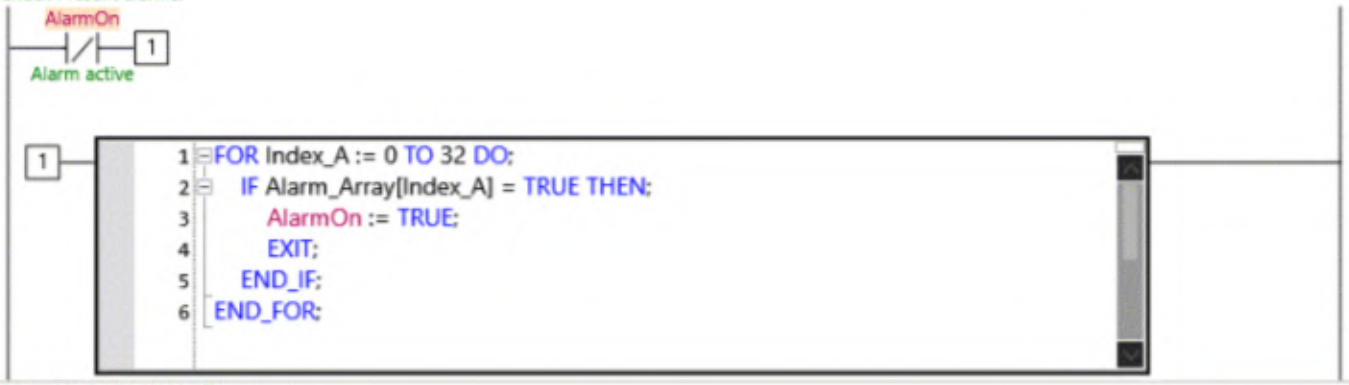

4
Password check..
P_On
Always TRUE Flag
<>
EN
In1
In2
Psw
Password
INT#22
INT#33
INT#44
INT#0
Up
In
1
=
5
6
7
2
3
4
MOVE
EN ENO
DINT#22
DINT#33
DINT#44
Out
HMI_Screen[0]
HMI pan...
INT#0
DINT#10
Psw22
Password 22


6
Psw33
Password 33
3
7
Psw44
Password 44
4
5
Password Reset
P_First_RunMode
First RUN Period Flag
MOVE
EN ENO
INT#0
In
Out
Psw
Password
T_Screen_Delay
TON
In
Q
PT
ET
T#1ms
=
EN
HMI_Screen[0]
HMI pan...
In1
DINT#10
In2
6
HMI coding.
P_On
Always TRUE Flag
MOVE
EN ENO
Alarm_Array
Alarm
In
Out
AlarmCodeSV
Panel alarms
I_Prot_ByPass
Guards-bypassed selector switch - Under Panel
SafeGuard_ByPass
Guards bypassed
I_FrontGuard_Closed
Front guard closed
FrontGuard_Closed
Front guard closed

1-8-1-1-6.Alarms



1 Check Present alarms.



2 Alarm has been called off.



3 Motor cooling Fan Alarm.



4 Inverter fault.



5 Machine in emergency mode.



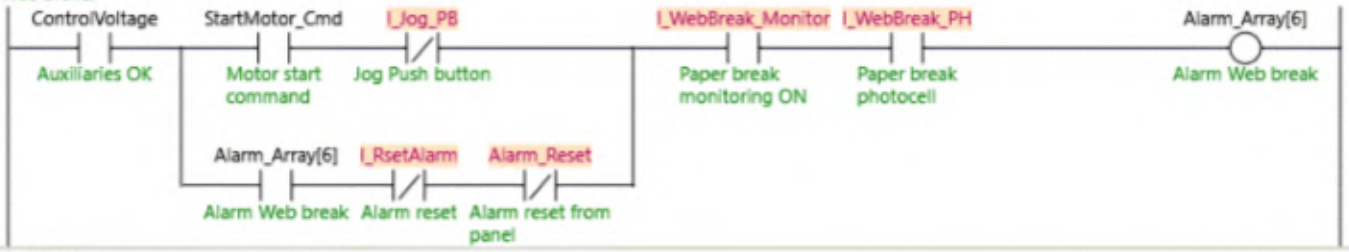
6 Motor thermal cutout.



7 Front Guard Alarm



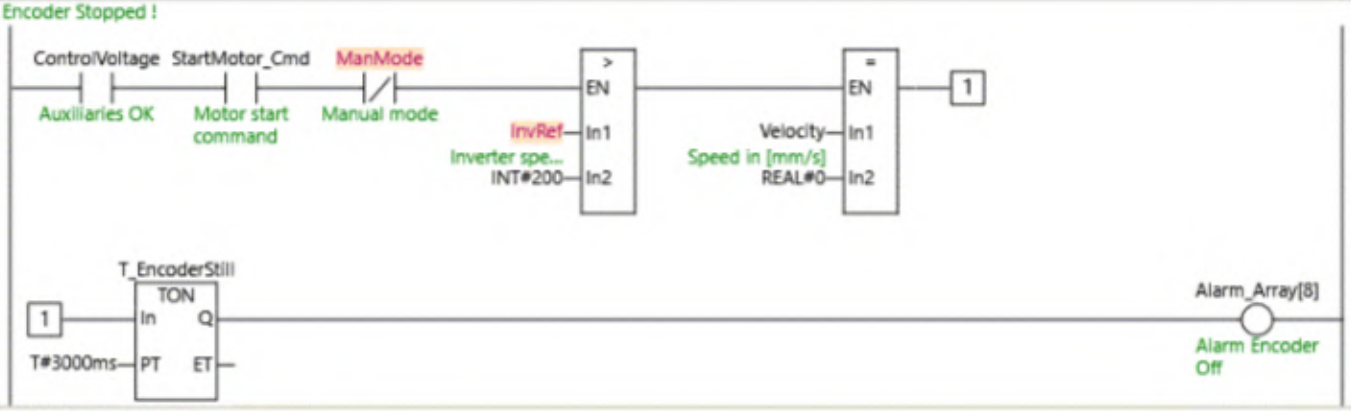
8 Web Brake.



9 Cooling fan.



10 Encoder Stopped !



11 Open-ended roll holder joint.



12 Rewinder Diameter Sensor Fault.



13 Rewinder Diameter Sensor Fault - Reset



14 Rewinder max diameter Machine protection.



15 Rewinder max diameter Machine protection - Reset



16 Rewinder GSM learning fault



17 Rewinder learning fault - Reset



18 Hub not in position.



19 Emergency Stop by Holding Stop PB 2s



20 Slow-down photocell active.



21 End-of-roll Alarm.



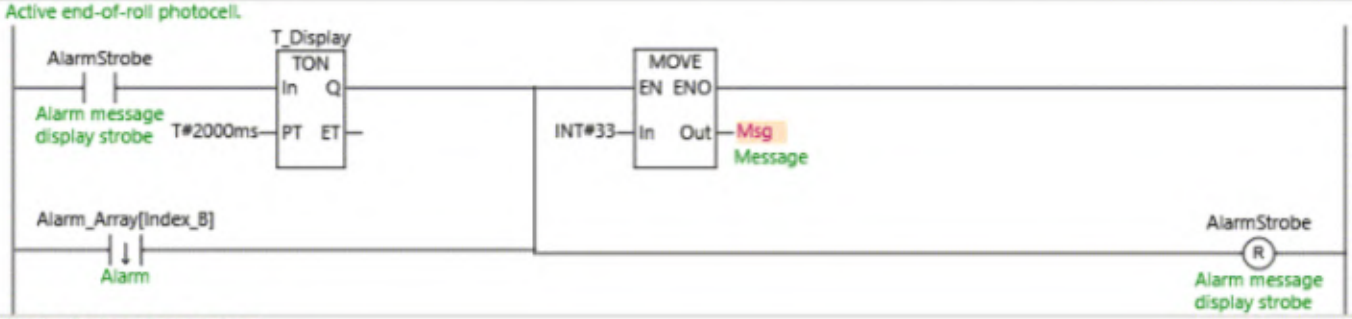
22 Unwinder Roll is Empty



23 Unwinder Roll is Empty - Reset



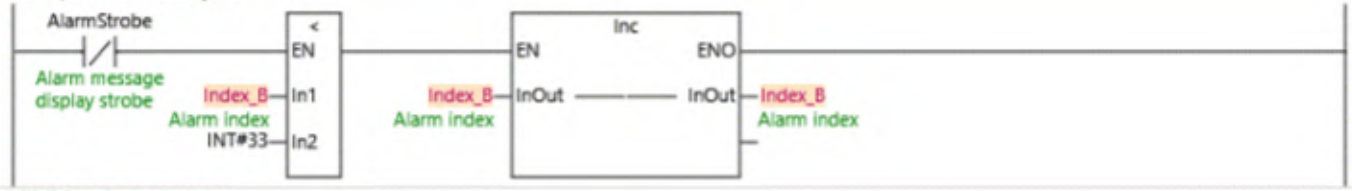
24 Active end-of-roll photocell.



25 Strobe displays alarm message.



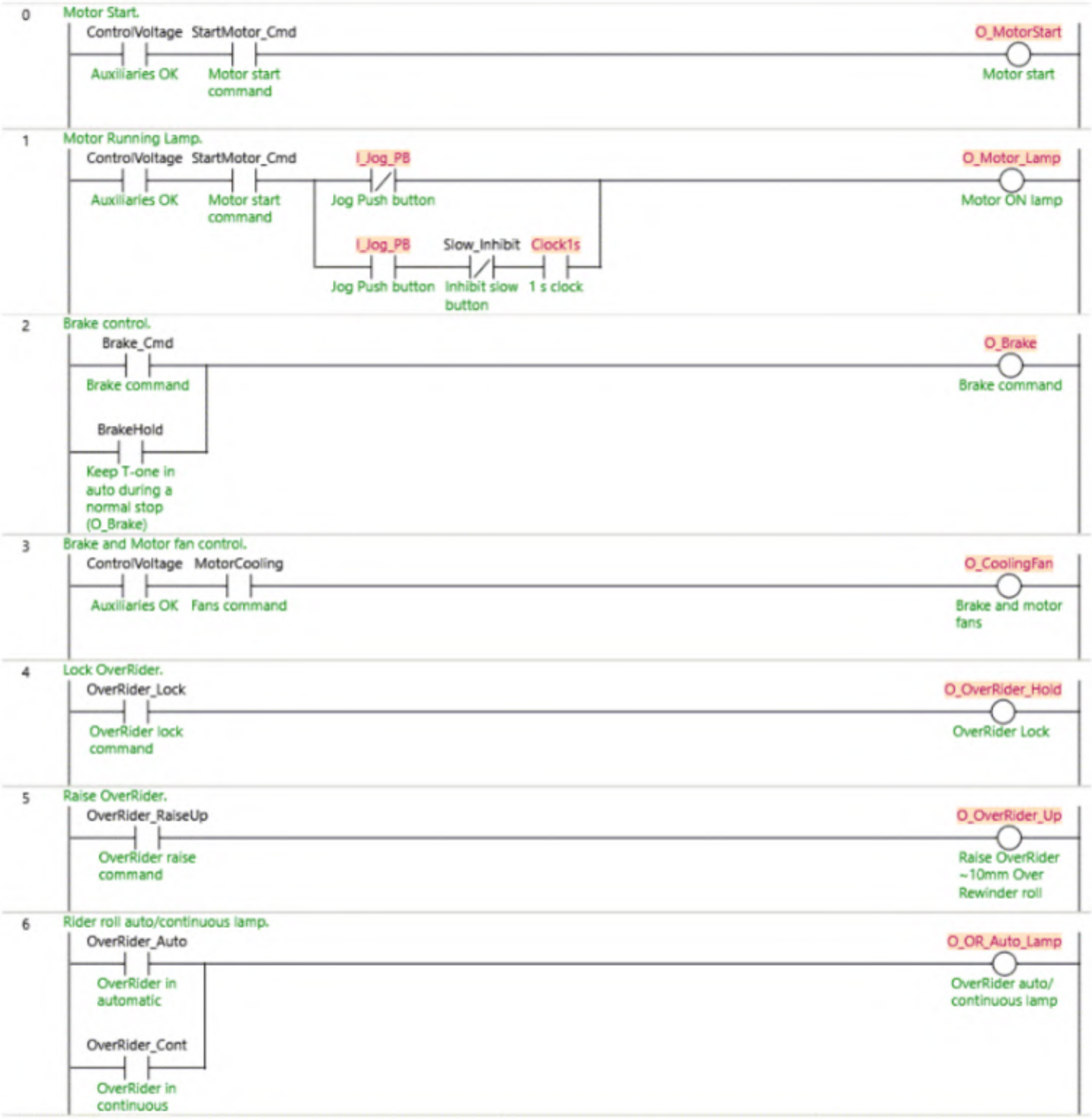
26 Alarm presence check cycle.



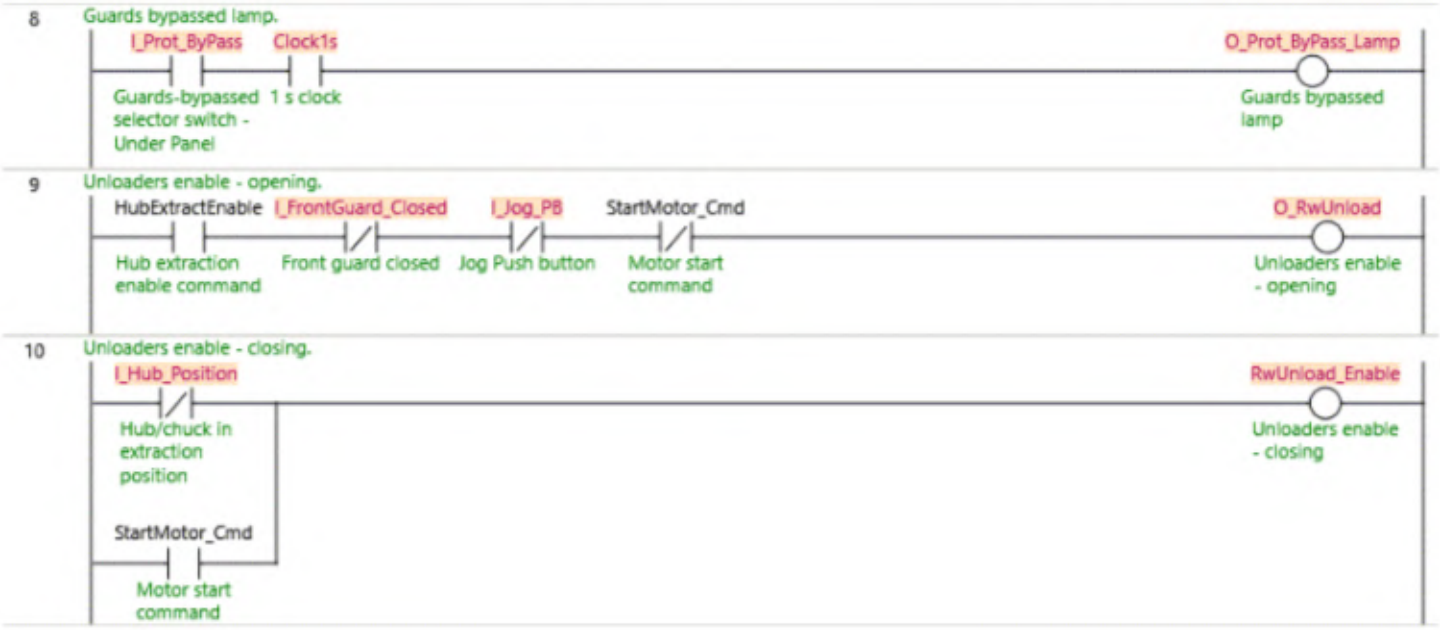
27 Initializing alarm message.



1-8-1-1-7.Output_Manager




7
Alarm lamp.
ControlVoltage
Auxiliaries OK
O_Alarm_Array
Alarm lamp
Alarm_Array[1]
Alarm Motor fan thermal
Clock1s
1 s clock
Alarm_Array[2]
Alarm Inverter fault
Alarm_Array[3]
Alarm Emergency stop
Alarm_Array[4]
Alarm Motor thermal switch
Alarm_Array[5]
Alarm Front Guard moving
Alarm_Array[6]
Alarm Web break
Alarm_Array[7]
Alarm Vacuum fan thermal
Alarm_Array[8]
Alarm Encoder Off
Alarm_Array[10]
Alarm Rewinder Diam Sensor Fault
Alarm_Array[11]
Alarm Rewinder max diam Protection.
Alarm_Array[15]
Alarm Unwinder Roll is Empty
Alarm_Array[16]
Alarm Rewinder GSM learning fault


8
Guards bypassed lamp.
I_Prot_ByPass
Clock1s
O_Prot_ByPass_Lamp
Guards-bypassed selector switch - Under Panel
1 s clock
Guards bypassed lamp
9
Unloaders enable - opening.
HubExtractEnable
I_FrontGuard_Closed
I_Jog_PB
StartMotor_Cmd
O_RwUnload
Hub extraction enable command
Front guard closed
Jog Push button
Motor start command
Unloaders enable - opening
10
Unloaders enable - closing.
I_Hub_Position
RwUnload_Enable
Hub/chuck in extraction position
StartMotor_Cmd
Motor start command
Unloaders enable - closing

11 Horn / acoustic signal.

```
// ==== V9: REPLACE Output_Manager rung 11 ("Horn / acoustic signal.", ladder) by ONE ST rung
// ==== with this code (no contact in front). Comment: "V9: horn, beeps set on the HMI".
// Old: horn blinking with Clock1s (0.5 s ON / 0.5 s OFF) during the whole slow-down
//      and the crawl at final speed -> very annoying.
// New, slow-down / final speed (StartMotor_Cmd AND SlowDown AND NOT Stop):
//      HornStartBeep_s beep when it starts (1 s).
//      then a HornBeepLen_s beep (0.2 s) every HornBeepPeriod (5 s).
//      then ONE HornStopBeep_s beep (3 s) when the machine has stopped (< 2 m/min).
//      Each setting 0 = that beep OFF. Front guard moving: unchanged (Clock1s blinking).
// PrimaryTask 4 ms: 250 scans = 1 s.

// HMI limits
IF HornStartBeep_s < REAL#0 THEN
    HornStartBeep_s := REAL#0;
ELSIF HornStartBeep_s > REAL#5 THEN
    HornStartBeep_s := REAL#5;
END_IF;

IF HornBeepLen_s < REAL#0 THEN
    HornBeepLen_s := REAL#0;
ELSIF HornBeepLen_s > REAL#2 THEN
    HornBeepLen_s := REAL#2;
END_IF;

IF HornBeepPeriod < REAL#0 THEN
    HornBeepPeriod := REAL#0;
ELSIF (HornBeepPeriod > REAL#0) AND (HornBeepPeriod < HornBeepLen_s + REAL#0.5) THEN
    HornBeepPeriod := HornBeepLen_s + REAL#0.5;    // always a pause between beeps
ELSIF HornBeepPeriod > REAL#30 THEN
    HornBeepPeriod := REAL#30;
END_IF;

IF HornStopBeep_s < REAL#0 THEN
    HornStopBeep_s := REAL#0;
ELSIF HornStopBeep_s > REAL#10 THEN
    HornStopBeep_s := REAL#10;
END_IF;

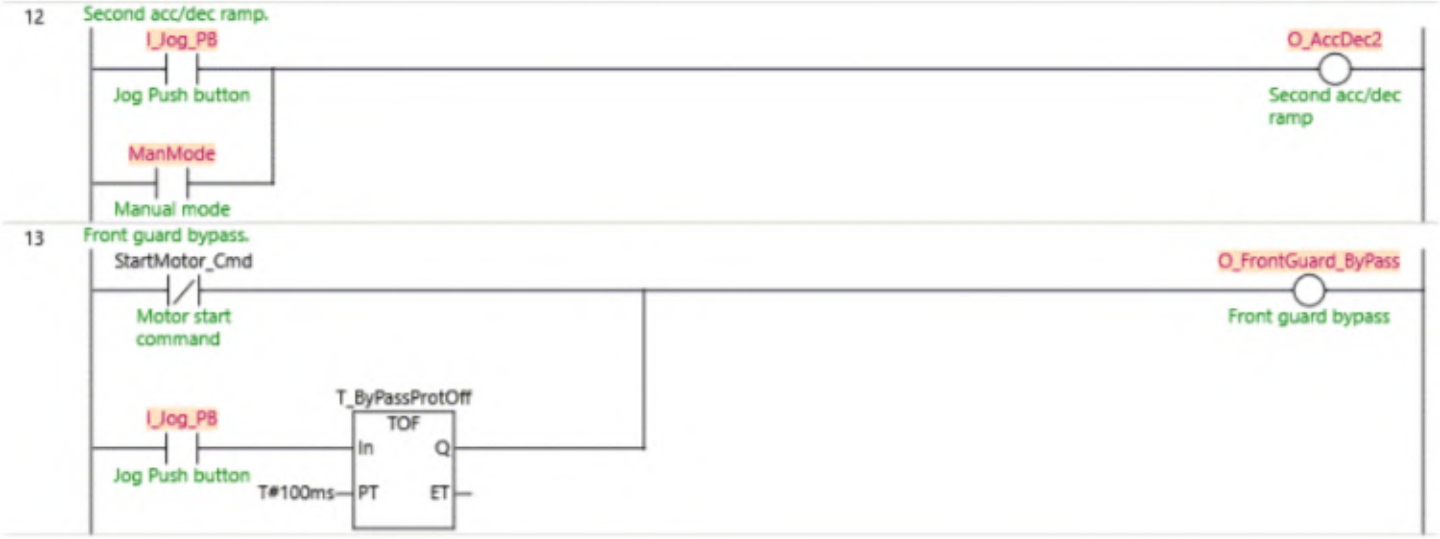
HornStartScans := REAL_TO_INT(HornStartBeep_s * REAL#250);
HornLenScans   := REAL_TO_INT(HornBeepLen_s * REAL#250);
HornPer        := REAL_TO_INT(HornBeepPeriod * REAL#250);

// Slow-down / final speed beeps
HornRun := StartMotor_Cmd AND SlowDown AND NOT Stop;

IF HornRun THEN
    IF NOT HornRunMem THEN
        HornTicks := 0;                    // slow-down starts
    END_IF;

    IF HornTicks < 30000 THEN
```

```
52          HornTicks := HornTicks + 1;
53      END_IF;
54
55      IF HornTicks <= HornStartScans THEN
56          HornSlowBeep := TRUE;                   // start beep
57          HornCnt := 0;
58      ELSIF (HornPer > 0) AND (HornLenScans > 0) THEN
59          HornCnt := HornCnt + 1;
60          IF HornCnt >= HornPer THEN
61              HornCnt := 0;
62          END_IF;
63
64          HornSlowBeep := (HornCnt >= HornPer - HornLenScans);   // beep at the end
    of each period
65      ELSE
66          HornSlowBeep := FALSE;                  // periodic beeps OFF
67      END_IF;
68  ELSE
69      HornTicks := 0;
70      HornCnt := 0;
71      HornSlowBeep := FALSE;
72  END_IF;
73
74  // End of a slow-down by a stop (auto stop, STOP button): one beep at standstill
75  // (Stop is set in Motor_Manager before StartMotor_Cmd drops on the next scan)
76
77  IF HornRunMem AND NOT HornRun AND (Stop OR NOT StartMotor_Cmd) THEN
78      HornStopPend := TRUE;
79  END_IF;
80
81  IF StartMotor_Cmd AND NOT HornStartMem THEN
82      HornStopPend := FALSE;                      // restarted: no stop beep
83      HornStopTicks := 0;
84  END_IF;
85
86  HornStartMem := StartMotor_Cmd;
87
88  IF HornStopPend AND (Velocity_M_min < REAL#2) THEN
89      HornStopPend := FALSE;
90      HornStopTicks := REAL_TO_INT(HornStopBeep_s * REAL#250);
91  END_IF;
92
93  IF HornStopTicks > 0 THEN
94      HornStopTicks := HornStopTicks - 1;
95  END_IF;
96
97  HornRunMem := HornRun;
98
99  // HMI test button: horn while pressed (machine stopped only)
100
101 O_AcousticSignal := (NOT I_FrontGuard_Closed AND NOT I_FrontGuard_Open AND
    Clock1s)
102                 OR HornSlowBeep OR (HornStopTicks > 0)
103                 OR (HornTest AND NOT StartMotor_Cmd);
```


12
Second acc/dec ramp.
I_Jog_PB
Jog Push button
ManMode
Manual mode
O_AccDec2
Second acc/dec ramp
13
Front guard bypass.
StartMotor_Cmd
Motor start command
I_Jog_PB
Jog Push button
T_ByPassProtOff
TOF
In
Q
T#100ms
PT
ET
O_FrontGuard_ByPass
Front guard bypass

1-8-1-1-8.AutoStop_Control

0 Auto Stop Process 1-5

```
(*
=============================================================
====================
  AutoStop_Control  V9   -  Automatic slow-down / stop
  Project : 04820KA001 Mono Canada II  (NX1P2, PrimaryTask 4 ms)
  Rewinder (Rw) : SICK DT35 0-10 V -> AI_Diam_Rewinder (AD2603 Ch2)
  Unwinder (Uw) : meter counter mode + sonar diameter (UwDiam_Act)

  NAMING  Uw_ = unwinder   Rw_ = rewinder
        _mm = millimeters   _m = meters
        _Pls = encoder pulses (counts)   _Pps = pulses per second
        _Pps2 = pulses per second squared

  V9 : RwDiamRev = DT35 averaged over exactly one revolution (2b), used by
       the thickness learning (removes the out-of-round wave), RwWave_mm.
     GSM popup: once per real new roll (empty 3 s, then > stop + 100 mm
       for 3 s); NEW PAPER closes it (section 10).
     paper thickness: long-baseline learning (D^2 growth / length) from the
       start point, LOCKED at RwLockDiam (300 mm): no more change, no
       paper-change detection, until NEW PAPER / GSM popup YES (4b).
  V8 : = V7 + V7c unwinder DIAMETER slow-down follows the speed (6c),
       V7e ManMode slow-down model as before V7 (section 6),
       V7f S-curve extra length in the slow-down (section 6)
  V7 : FEED-FORWARD speed loop (Motor_Manager rung B): reference = set speed
     x RwRatioD / diameter along a ramp in real web mm/s2 (DecelWeb /
     AccelWeb), RwRatioD adapted slowly -> no hunting on big rolls.
     Section 6: slow-down length = one ramp at DecelWeb (FFEnable), else
     web coarse ramp + trim tail; learning only as fallback.
  v6y: rewinder AUTO stop (RwStopEnable) limits the work speed to
     RwAutoMaxSpd (500 m/min), taken over at standstill only; section 6
     uses the limited speed.
  v6x: new parent roll on the unwinder -> GsmAsk (HMI popup "did you change
     the paper?"); GsmAskYes forgets the learned thickness.
  v6w: paper thickness window is LENGTH-based instead of 10 s, modulated by
     the diameter (RwWinGrow_mm 2.5 mm of growth, 4 m .. RwWinLen_m 30 m):
     same accuracy at every speed and roll size; stopped window = dropped.
     ALARM RESET also clears LearnFailCnt / StopFailCnt (alarm 16).
  v6v: BrakeHold released on the SIGNED web speed (Velocity_M_min < 5),
     so a web pulled back after the stop releases the T-one at once.
  v6u: stop learning: a stop shorter than the reaction distance or faster
     than RwDecelMax_mm is ignored (not a fault). Only a too slow stop
     (< RwDecelMin_mm) counts toward alarm 16.
  v6t: rewinder slow-down fallback while the paper thickness is not
     learned: RwSlowReq at RwTargetDiam - RwSlowFallbackMm (30 mm).
  v6s: paper change: a learning window more than RwMatChangePct (25 %) away
     from RwMat_K is ignored once, taken directly the 2nd time in a row
     (automatic). RwMatReset (HMI "NEW PAPER") forgets RwMat_K.
  v6p: section 5c: RwLearnReset clears RwStopDecelK, RwStopDecel and
     RwSlowDecelK (the v6f seed that refilled RwStopDecelK is removed).
  v6l: previous value saved before every TEACH CORE (RwAxisDist_Prev,
       UwCoreDist_Prev, Retain) + UNDO buttons (RwTeachUndo,
       UwTeachUndo): swap current <-> previous. State 4 = RESTORED.
  v6k: unwinder TEACH CORE (Core_Measure) works like the rewinder one:
       same checks, 2 s stable with sonar AVERAGE, status UwTeachState.
       Replaces General rung 15 (one immediate sonar reading).
  v6j: TEACH CORE (section 14): HMI button RwTeachCore, empty core on the
```

```
55          rewinder, machine stopped -> after 2 s stable:
56          RwAxisDist := RwDist + RwCoreDiameter / 2. Status in RwTeachState.
57      v6i: auto: pots scaled to WorkSpd_Max / FinalSpd_Max (Motor_Manager rung 6.
58          Pot1_Mem / Pot2_Mem). ManMode keeps the raw pots (percent),
59          so section 6 uses the raw pots in ManMode.
60      v6h: BrakeHold keeps O_Brake (T-one "reel brake automatic") ON during a
61          normal stop until the web is almost stopped (< 5 m/min, max 15 s).
62          Before: T-one switched to "stop" with the run command, while the
63          rewinder was decelerating hard -> unwinder overran, web on the
64          floor (field 10/20/2026). Used in Output_Manager rung 2.
65      v6g: slow-down AND stop decel proportional to the rewinder diameter.
66          The ATV930 ramp is in Hz/s (motor = rewind shaft), so the web
67          decel [mm/s2] = shaft decel x radius. Field data 10/20/2026:
68          24 mm/s2 at 120 mm -> 90 mm/s2 at 600 mm.
69          Learned: RwSlowDecelK, RwStopDecelK [mm/s2 per mm of diameter].
70          Live:    RwSlowDecelLrn [mm/s2], RwStopDecel [pulses/s2] = K x DecelDiam.
71      v6f: learn the REAL slow-down deceleration (section 5b) -> RwSlowDecelLrn
72          (mm/s2). The ATV930 stretches its ramp with a heavy roll (no
73          braking resistor), so the PLC ramp (DecelCoarse) was too
74          optimistic: the stop came before final speed was reached.
75          Priority in section 6: RwSlowDecel (manual) > RwSlowDecelLrn
76          (learned) > DecelCoarse/DecelTrim model.
77      v6e: GSM_Learned_OK (replaces global RwLearnFault): TRUE only when
78          RwMat_K is learned, RwStopDecel is learned (if RwLearnStop) and
79          there is no learning fault. Alarm 16 uses GSM_LearnFault.
80      v6d: exact 100 ms tick (25 x 4 ms scan counter, the self-resetting
81          TON ran ~104-108 ms -> speed ~4-8 % too high)
82          standstill = <= 2 counts / 100 ms (encoder jitter)
83          stop learning skipped after jog / manual / slow-button runs and
84            when the auxiliaries drop (E-stop): RunWasJog, StopLearnValid
85          work / final speed from the potentiometers in BOTH modes
86            (Speed_Display / LowSpd_Display are not m/min in ManMode)
87          slow-down length includes the DecelTrim tail inside the
88            SpdDelta_Percent band (Motor_Manager rungs 14/15)
89      v6c: learning supervision -> RwLearnFault (alarm 16), learned paper
90          thickness RwThick_mm, plausibility limits for K and stop decel
91      v6b: RwMaxSlowAtDiam display + RwMaxSlowFallback (slow-down warning
92          while RwMat_K is not learned yet)
93      v6 : consistent internal names, units in names.
94          Globals renamed: RwRemain_m, RwStopLen_m, RwSlowLen_m,
95                      RwSlowMargin_m, UwSlowMargin_m
96      v5 : max diameter protection (always active), sensor plausibility,
97          12SA5 master (section 9), alarms in section Alarms (ladder)
98      v4b: AUTO interlock + single HMI display (SlowAtDisp / SlowAtMode)
99
        ================================================================
        ==================== *)
100
101 (* ---- 1. 100 ms tick, mm per encoder pulse ------------------------------ *)
102 (* v6d: 25 scans x 4 ms PrimaryTask = exactly 100 ms.                    *)
103 (*      Change 25 if the PrimaryTask period is changed.                  *)
104
105 TickCnt := TickCnt + 1;
106
107 IF TickCnt >= 25 THEN
108     TickCnt := 0;
109     Tick := TRUE;
110 ELSE
111     Tick := FALSE;
```

```
112 END_IF;
113
114 IF (Encoder_Pulse > 0) AND (EncoderWheel_Diam > 0.0) THEN
115     MmPerPulse := EncoderWheel_Diam / (INT_TO_REAL(Encoder_Pulse) * 4.0);
116 ELSE
117     MmPerPulse := 0.0;
118 END_IF;
119
120 (* ---- 2. Rewinder DT35 -> filtered distance -> diameter ---------- *)
121
122 RwDistRaw_mm := RwDist0V + (INT_TO_REAL(AI_Diam_Rewinder) / 4000.0)
123             * (RwDist10V - RwDist0V);
124
125 IF Tick THEN
126     IF NOT RwFiltInit THEN
127         RwDist := RwDistRaw_mm;
128         RwFiltInit := TRUE;
129     ELSE
130         RwDist := RwDist + RwFilter_K * (RwDistRaw_mm - RwDist);
131     END_IF;
132 END_IF;
133
134 RwSensFault := (AI_Diam_Rewinder < RwRawMin) OR (AI_Diam_Rewinder >
    RwRawMax)
135             OR (RwDist > RwAxisDist);
136
137 RwDiam := 2.0 * (RwAxisDist - RwDist);
138
139 IF RwDiam < 0.0 THEN
140     RwDiam := 0.0;
141 END_IF;
142
143 (* ---- 2b. V9: diameter averaged over exactly ONE revolution ---------- *)
144
145 (* The out-of-round roll gives a wave at the rotation frequency (seen in   *)
146 (* the 10/29 traces: 2.4 Hz at 444 m/min / 973 mm). Averaging the raw     *)
147 (* DT35 distance over exactly one revolution cancels it (and all its      *)
148 (* harmonics), whatever the phase -> no shaft position sensor needed:     *)
149 (* one revolution = PI x D of web on the encoder. Each scan is weighted   *)
150 (* by its encoder pulses = by roll ANGLE, so ramps and standstill do not  *)
151 (* bias it (stopped: nothing added). Used only by the thickness learning. *)
152 (* RwWave_mm = peak-peak diameter wave in the last revolution (HMI).      *)
153
154 RevDP := ABS(Encoder_Pos - RevPrevPos);
155 RevPrevPos := Encoder_Pos;
156
157 IF RwSensFault OR (MmPerPulse <= 0.0) OR (RwDiam < 50.0) OR (RevDP > 100000)
    THEN
158     RevW := 0.0;
159     RevSum := 0.0;
160     RwDiamRevOk := RwDiamRevOk AND NOT RwSensFault;
161 ELSIF RevDP > 0 THEN
162
163     IF RevW <= 0.0 THEN
164         RevMin := RwDistRaw_mm;
165         RevMax := RwDistRaw_mm;
166     END_IF;
167
168     RevSum := RevSum + RwDistRaw_mm * DINT_TO_REAL(RevDP);
```

```
169     RevW := RevW + DINT_TO_REAL(RevDP);
170
171     IF RwDistRaw_mm < RevMin THEN
172         RevMin := RwDistRaw_mm;
173     END_IF;
174
175     IF RwDistRaw_mm > RevMax THEN
176         RevMax := RwDistRaw_mm;
177     END_IF;
178
179     IF RevW >= 3.14159 * RwDiam / MmPerPulse THEN
180         RwDiamRev := 2.0 * (RwAxisDist - RevSum / RevW);
181         RwWave_mm := 2.0 * (RevMax - RevMin);
182         RwDiamRevOk := TRUE;
183         RevW := 0.0;
184         RevSum := 0.0;
185     END_IF;
186 END_IF;
187
188 (* v6g: diameter used by the decel model (last valid on sensor fault)      *)
189
190 IF NOT RwSensFault THEN
191     DecelDiam := RwDiam;
192 ELSE
193     DecelDiam := RwLastValidDiam;
194 END_IF;
195
196 IF DecelDiam < 100.0 THEN
197     DecelDiam := 100.0;                    (* core *)
198 END_IF;
199
200 (* ---- 3. Actual speed from encoder, 100 ms window ---------------------- *)
201
202 IF Tick THEN
203
204     IF NOT SpdInit THEN
205         EncPrev := Encoder_Pos;
206         SpdInit := TRUE;
207     END_IF;
208
209     SpdAct_Pps := DINT_TO_REAL(ABS(Encoder_Pos - EncPrev)) * 10.0;
210     EncPrev := Encoder_Pos;
211 END_IF;
212
213 RwSpeed := SpdAct_Pps * MmPerPulse * 0.06;         (* m/min *)
214
215 (* ---- 4a. v6s: NEW PAPER button: forget the learned thickness ---------- *)
216
217 (* RwMat_K = 0 -> the first valid 10 s window sets it directly.           *)
218 (* HMI momentary button: must be HELD 3 s, then the reset is done once and *)
219 (* the PLC clears the bit (HMI lamp goes off = done).                     *)
220
221 T_MatReset(In := RwMatReset, PT := T#3s);
222
223 IF T_MatReset.Q THEN
224     RwMat_K := 0.0;
225     LearnFailCnt := 0;
226     MatChangeCnt := 0;
227     RwMatLocked := FALSE;                  (* V9 *)
```

```
    LongActive := FALSE;
    GsmAsk := FALSE;                    (* V9: NEW PAPER answers the popup *)
    GsmArmed := FALSE;
    RwMatReset := FALSE;
END_IF;

(* ---- 4b. V9: long-baseline thickness learning + LOCK ---------- *)

(* From the start point (first run after NEW PAPER / GSM YES / power-on    *)
(* while not locked): K = (D^2 - D0^2) / (2 x pulses). The waving of the   *)
(* roll (+/-0.4 mm) only matters at the two ends, so the error falls with  *)
(* the growth (50 mm growth: ~1 %). After 20 mm growth it replaces the     *)
(* 30 m windows. At RwLockDiam (and >= RwLockMinGrow_mm growth) the value
   *)
(* is LOCKED (Retain): windows and paper-change detection stop.            *)
(* RwLockDiam = 0 -> never locked (windows + long baseline as learning).   *)

IF Tick AND NOT RwMatLocked THEN

    IF LongActive AND (RwDiamRev < LongStartD - 30.0) AND RwDiamRevOk THEN
        LongActive := FALSE;            (* new core: restart *)
    END_IF;

    IF Encoder_Reset THEN
        LongActive := FALSE;            (* metre counter reset *)
    END_IF;

    IF NOT LongActive AND O_MotorStart AND RwDiamRevOk
      AND (RwDiamRev > 0.0) THEN
        LongStartPos := Encoder_Pos;
        LongStartD := RwDiamRev;
        LongActive := TRUE;
    END_IF;

    LongValid := FALSE;

    IF LongActive AND RwDiamRevOk AND (MmPerPulse > 0.0) THEN
        LongGrow_mm := RwDiamRev - LongStartD;
        LongPls := DINT_TO_REAL(ABS(Encoder_Pos - LongStartPos));

        IF (LongGrow_mm >= 20.0) AND (LongPls > 0.0) THEN
            LongK := (RwDiamRev * RwDiamRev - LongStartD * LongStartD) / (2.0 *
LongPls);
            LongThick_mm := LongK * 3.14159 / (2.0 * MmPerPulse);

            IF (LongThick_mm >= RwThickMin_mm) AND (LongThick_mm <=
RwThickMax_mm) THEN
                LongValid := TRUE;
                RwMat_K := LongK;
                MatChangeCnt := 0;
                LearnFailCnt := 0;

                IF (RwLockDiam > 0.0) AND (RwDiamRev >= RwLockDiam)
                  AND (LongGrow_mm >= RwLockMinGrow_mm) THEN
                    RwMatLocked := TRUE;        (* solid from now on *)
                    LongActive := FALSE;
                END_IF;
            END_IF;
        END_IF;
```

```
284     END_IF;
285   END_IF;
286
287   IF RwMatLocked THEN
288     LongValid := FALSE;
289     LearnFailCnt := 0;                    (* no thickness alarm *)
290   END_IF;
291
292   (* ---- 4. Learn material factor  K = dDiam/dPulse * Diam ---------- *)
293
294   IF Tick THEN
295     LearnWinTicks := LearnWinTicks + 1;
296
297     IF NOT LearnInit THEN
298       LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiamRev;  (* V9 *)
299       LearnWinTicks := 0;
300       LearnWinRunning := O_MotorStart;
301       LearnInit := TRUE;
302     ELSE
303       LearnWinRunning := LearnWinRunning AND O_MotorStart;
304       (* v6w: LENGTH-based window, modulated by the diameter: closes    *)
305       (* when the roll should have grown RwWinGrow_mm (2.5 mm), so the  *)
306       (* accuracy vs DT35 noise is the same at every speed and size.    *)
307       (* Pulses = growth x diameter / K (K = learned RwMat_K).          *)
308       (* Limits: min 4 m, max RwWinLen_m (30 m). K not learned: max.    *)
309       (* Motor stopped during the window -> window dropped (no fault).  *)
310
311       IF MmPerPulse > 0.0 THEN
312         IF (RwMat_K > 0.0) AND (RwWinGrow_mm > 0.0) THEN
313           LearnWinLen_Pls := RwWinGrow_mm * MAX(RwDiam, REAL#100.0) /
    RwMat_K;
314         ELSE
315           LearnWinLen_Pls := RwWinLen_m * 1000.0 / MmPerPulse;
316         END_IF;
317
318         IF LearnWinLen_Pls < 4000.0 / MmPerPulse THEN
319           LearnWinLen_Pls := 4000.0 / MmPerPulse;          (* 4 m *)
320         END_IF;
321
322         IF (RwWinLen_m > 0.0)
323           AND (LearnWinLen_Pls > RwWinLen_m * 1000.0 / MmPerPulse) THEN
324           LearnWinLen_Pls := RwWinLen_m * 1000.0 / MmPerPulse;
325         END_IF;
326       ELSE
327
328         LearnWinLen_Pls := 300000.0;
329       END_IF;
330
331       IF (DINT_TO_REAL(ABS(Encoder_Pos - LearnWinStartPos)) >= LearnWinLen_Pls)
332         OR NOT LearnWinRunning THEN
333         LearnWin_Pls := ABS(Encoder_Pos - LearnWinStartPos);
334         LearnWinDiamDelta := RwDiamRev - LearnWinStartDiam;  (* V9 *)
335         (* A window SHOULD learn: ran all the time, enough length,  *)
336         (* DT35 OK. Then it must give a plausible paper thickness. *)
337
338         IF LearnWinRunning AND (LearnWin_Pls >= RwMinPulses)
339           AND NOT RwSensFault
340           AND NOT RwMatLocked AND NOT LongValid      (* V9 *)
341           AND RwDiamRevOk AND (LearnWinStartDiam > 0.0) THEN
```

```
342             LearnWinOk := FALSE;
343
344             IF (LearnWinDiamDelta > 0.0) AND (MmPerPulse > 0.0) THEN
345                 MatK_Window := (LearnWinDiamDelta / DINT_TO_REAL(LearnWin_Pls))
346                             * ((RwDiamRev + LearnWinStartDiam) / 2.0);
347
348                 (* K = 2 * thickness * mm-per-pulse / PI         *)
349
350                 ThickWin_mm := MatK_Window * 3.14159 / (2.0 * MmPerPulse);
351
352                 IF (ThickWin_mm >= RwThickMin_mm)
353                   AND (ThickWin_mm <= RwThickMax_mm) THEN
354                     LearnWinOk := TRUE;
355
356                     IF RwMat_K <= 0.0 THEN
357                         RwMat_K := MatK_Window;
358                         MatChangeCnt := 0;
359                     ELSIF (RwMatChangePct > 0.0)
360                         AND (ABS(MatK_Window - RwMat_K)
361                             > RwMat_K * RwMatChangePct / 100.0) THEN
362
363                         (* v6s: far from the learned value: 1 window = ignored *)
364                         (* (outlier). 2 in a row = paper changed -> take it.  *)
365
366                         MatChangeCnt := MatChangeCnt + 1;
367
368                         IF MatChangeCnt >= 2 THEN
369                             RwMat_K := MatK_Window;
370                             MatChangeCnt := 0;
371                         END_IF;
372                     ELSE
373                         MatChangeCnt := 0;
374                         RwMat_K := RwMat_K + RwLearn_K * (MatK_Window - RwMat_K);
375                     END_IF;
376                 END_IF;
377             END_IF;
378
379             IF LearnWinOk THEN
380                 LearnFailCnt := 0;
381             ELSIF LearnFailCnt < 1000 THEN
382                 LearnFailCnt := LearnFailCnt + 1;  (* no growth / implausible *)
383             END_IF;
384         END_IF;
385
386         LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiamRev;  (* V9
    *)
387         LearnWinTicks := 0;
388         LearnWinRunning := O_MotorStart;
389       END_IF;
390     END_IF;
391 END_IF;
392
393 (* ---- 5. Learn real stop deceleration on every stop ---------- *)
394
395 (* v6d: a run with jog / manual / slow button uses the 2nd ramp (O_AccDec2) *)
396 (*      -> its stop must not be learned. Latched for the whole run because  *)
397 (*      I_LowSpd_OS is already released on the stop edge.                   *)
```

```
399 IF O_MotorStart AND NOT MotorRunMem THEN          (* start edge *)
400     RunWasJog := FALSE;
401 END_IF;
402
403 IF O_MotorStart AND (ManMode OR I_Jog_PB) THEN
404     RunWasJog := TRUE;
405 END_IF;
406
407 IF MotorRunMem AND NOT O_MotorStart THEN          (* stop command edge *)
408     StopMeasActive := TRUE;
409     StopStartPos := Encoder_Pos;
410     StopStartSpd_Pps := SpdAct_Pps;
411     StopStartDiam := DecelDiam;
412     StandstillTicks := 0;
413     (* STOP button and automatic stop take the same path (drive DI1 and    *)
414     (* O_Brake -> T-one drop together, electrical plan sheets 1/9/15),     *)
415     (* so both are learned.                                                *)
416     StopLearnValid := ControlVoltage AND NOT RunWasJog AND NOT ManMode;
417 END_IF;
418
419 MotorRunMem := O_MotorStart;
420
421 IF O_MotorStart THEN
422     StopMeasActive := FALSE;                  (* restarted: abort *)
423 ELSIF StopMeasActive AND Tick THEN
424
425     IF NOT ControlVoltage THEN
426         StopLearnValid := FALSE;              (* E-stop during the stop *)
427     END_IF;
428
429     IF SpdAct_Pps <= 20.0 THEN                (* <= 2 counts / 100 ms *)
430         StandstillTicks := StandstillTicks + 1;
431     ELSE
432         StandstillTicks := 0;
433     END_IF;
434
435     IF StandstillTicks >= 3 THEN              (* standstill 300 ms *)
436         StopMeasActive := FALSE;
437         StopMeas_Pls := DINT_TO_REAL(ABS(Encoder_Pos - StopStartPos));
438
439         IF RwLearnStop AND StopLearnValid
440           AND (StopStartSpd_Pps * MmPerPulse * 0.06 >= RwMinLearnSpd) THEN
441             (* This stop SHOULD learn: result must be plausible       *)
442             (* v6u: a stop shorter than the reaction distance, or     *)
443             (* faster than RwDecelMax_mm, is NOT a fault: the web     *)
444             (* stopped within a few mm, nothing to learn (trace       *)
445             (* 10/28/2026: 23 m/min -> stopped in 102 pulses = 10 mm). *)
446             (* Only a too SLOW stop (< RwDecelMin_mm) counts as fail.  *)
447
448             StopLearnOk := FALSE;
449
450             IF StopMeas_Pls > StopStartSpd_Pps * RwTreact THEN
451                 StopDecelMeas := (StopStartSpd_Pps * StopStartSpd_Pps)
452                                  / (2.0 * (StopMeas_Pls - StopStartSpd_Pps * RwTreact));
453                 StopDecelMeas_mm := StopDecelMeas * MmPerPulse;  (* mm/s2 *)
454
455                 IF (StopDecelMeas_mm >= RwDecelMin_mm)
456                   AND (StopDecelMeas_mm <= RwDecelMax_mm) THEN
457                     StopLearnOk := TRUE;
```

```
457                 StopLearnOK := TRUE;
458
459                 (* v6g: learn per mm of diameter *)
460                 DecelK_Meas := StopDecelMeas_mm / ((StopStartDiam + DecelDiam) /
    2.0);
461
462                 IF RwStopDecelK <= 0.0 THEN
463                    RwStopDecelK := DecelK_Meas;
464                 ELSE
465                    RwStopDecelK := RwStopDecelK + 0.3 * (DecelK_Meas -
    RwStopDecelK);
466                 END_IF;
467
468                 StopFailCnt := 0;
469              ELSIF StopDecelMeas_mm < RwDecelMin_mm THEN
470
471                 IF StopFailCnt < 1000 THEN
472                    StopFailCnt := StopFailCnt + 1;   (* stop too long *)
473                 END_IF;
474              END_IF;                              (* too fast: ignored *)
475           END_IF;                                 (* too short: ignored *)
476        END_IF;
477     END_IF;
478  END_IF;
479
480  (* ---- 5b. v6f: learn real slow-down deceleration ---------------------- *)
481
482  (* From the SlowDown edge (at speed) until final speed is reached, or     *)
483  (* until the stop comes first. a = (v0^2 - v1^2) / (2 * distance).        *)
484  (* Not learned in jog / manual / slow button (2nd ramp).                  *)
485
486  IF SlowDown AND NOT SlowMem AND O_MotorStart
487     AND NOT ManMode AND NOT I_Jog_PB
488     AND (SpdAct_Pps > SpdFinal_Pps + SpdBand_Pps) THEN
489     SlowMeasActive := TRUE;
490     SlowStartPos := Encoder_Pos;
491     SlowStartSpd_Pps := SpdAct_Pps;
492     SlowStartDiam := DecelDiam;
493  END_IF;
494
495  SlowMem := SlowDown;
496
497  IF SlowMeasActive THEN
498
499     IF NOT SlowDown OR ManMode OR I_Jog_PB THEN
500        SlowMeasActive := FALSE;                  (* aborted *)
501     ELSIF Tick AND ((SpdAct_Pps <= SpdFinal_Pps + 0.25 * SpdBand_Pps)
502                OR NOT O_MotorStart) THEN       (* final reached or stopped *)
503        SlowMeasActive := FALSE;
504        SlowMeas_Pls := DINT_TO_REAL(ABS(Encoder_Pos - SlowStartPos));
505
506        IF (SlowMeas_Pls > 0.0) AND (MmPerPulse > 0.0)
507          AND ((SlowStartSpd_Pps - SpdAct_Pps) * MmPerPulse * 0.06 >= 10.0) THEN
508           SlowDecelMeas_mm := (SlowStartSpd_Pps * SlowStartSpd_Pps
509                              - SpdAct_Pps * SpdAct_Pps)
510                             / (2.0 * SlowMeas_Pls) * MmPerPulse;  (* mm/s2 *)
511
512           IF (SlowDecelMeas_mm >= RwDecelMin_mm)
513             AND (SlowDecelMeas_mm <= RwDecelMax_mm) THEN
```

```
514             (* v6g: learn per mm of diameter *)
515             DecelK_Meas := SlowDecelMeas_mm / ((SlowStartDiam + DecelDiam) /
    2.0);
516
517             IF RwSlowDecelK <= 0.0 THEN
518                 RwSlowDecelK := DecelK_Meas;
519             ELSE
520                 RwSlowDecelK := RwSlowDecelK + 0.5 * (DecelK_Meas -
    RwSlowDecelK);
521             END_IF;
522         END_IF;
523     END_IF;
524 END_IF;
525 END_IF;
526
527 (* ---- 5c. v6g: live decels = learned K x current diameter ---------- *)
528
529 (* v6p: the one-time v6f seed is removed (it re-filled RwStopDecelK from    *)
530 (* RwStopDecel, so the learning could never be reset).                     *)
531 (* RwLearnReset (HMI / Watch tab, PLC resets it): clear the learned decels. *)
532 (* Both are re-learned at the next auto slow-down / stop >= RwMinLearnSpd.  *)
533
534 IF RwLearnReset THEN
535     RwStopDecelK := 0.0;
536     RwStopDecel := 0.0;
537     RwSlowDecelK := 0.0;
538     StopFailCnt := 0;
539     RwLearnReset := FALSE;
540 END_IF;
541
542 IF (RwStopDecelK > 0.0) AND (MmPerPulse > 0.0) THEN
543     RwStopDecel := RwStopDecelK * DecelDiam / MmPerPulse;     (* pulses/s2 *)
544 END_IF;
545
546 IF RwSlowDecelK > 0.0 THEN
547     RwSlowDecelLrn := RwSlowDecelK * DecelDiam;               (* mm/s2 *)
548 ELSE
549     RwSlowDecelLrn := 0.0;
550 END_IF;
551
```

1 Auto Stop Process 6-14

```
(* ---- 6. Lengths in pulses: stop, slow-down, remaining ------------------ *)

(* Stop length uses the ACTUAL speed (precision)                          *)

IF RwStopDecel > 0.0 THEN
   StopLen_Pls := SpdAct_Pps * RwTreact
                + (SpdAct_Pps * SpdAct_Pps) / (2.0 * RwStopDecel);
ELSE
   StopLen_Pls := SpdAct_Pps * (RwTreact + RwTstopDef / 2.0);
END_IF;
StopLen_Pls := StopLen_Pls * RwTrim;

(* Slow-down uses SpdRef = MAX(actual, work-speed setpoint):              *)
(* stable value, shown to the operator even before start                  *)

SlowLenBase_Pls := 0.0;  RwSlowLen_Pls := 0.0;  UwSlowLen_Pls := 0.0;

IF MmPerPulse > 0.0 THEN
   (* v6f: manual > learned > PLC ramp model                               *)
   (* V7: manual > feed-forward (one ramp at DecelWeb, real web mm/s2)  *)
   (*     > old loop with web ramps (coarse DecelWeb + trim tail)       *)
   (*     > learned > old PLC ramp model                                 *)

   IF RwSlowDecel > 0.0 THEN
      SlowDecel_Pps2 := RwSlowDecel / MmPerPulse;
      SlowOneRamp := TRUE;
   ELSIF FFEnable AND (DecelWeb > 0.0) AND (WebRatio > 0.0) AND NOT
ManMode THEN
      SlowDecel_Pps2 := DecelWeb / MmPerPulse;
      SlowOneRamp := TRUE;
   ELSIF (DecelWeb > 0.0) AND (WebRatio > 0.0) AND NOT ManMode THEN  (* V7e
*)
      SlowDecel_Pps2 := DecelWeb / MmPerPulse;
      SlowOneRamp := FALSE;
   ELSIF RwSlowDecelLrn > 0.0 THEN
      SlowDecel_Pps2 := RwSlowDecelLrn / MmPerPulse;  (* real, incl. trim tail *)
      SlowOneRamp := TRUE;
   ELSE
      SlowDecel_Pps2 := DecelCoarse * 250.0 / MmPerPulse; (* 1 = 250 mm/s2 *)
      SlowOneRamp := FALSE;
   END_IF;

   (* v6d: DecelTrim ramp used inside the +/- SpdDelta_Percent band *)

   IF (DecelTrim > 0.0) AND (WebRatio > 0.0) AND (Vel_Max_mms > 0.0) THEN
      (* V7: trim ramp (reference units) converted to the real web decel *)
      TrimDecel_Pps2 := DecelTrim * 1000000.0 / (0.06 * WebRatio * Vel_Max_mms)
                      / MmPerPulse;
   ELSIF DecelTrim > 0.0 THEN
      TrimDecel_Pps2 := DecelTrim * 250.0 / MmPerPulse;
   ELSE
      TrimDecel_Pps2 := SlowDecel_Pps2;
   END_IF;

   SpdBand_Pps := SpdDelta_Percent * 1000.0 / 60.0 / MmPerPulse;

   (* v6d: speeds from the potentiometers, valid in auto AND ManMode     *)
```

```
57
58    (*     (m/min = Pot / Input_Ratio, same as Motor_Manager rung 8)     *)
59
60    IF (Input_Ratio > 0.0) AND ManMode THEN
61       (* v6i: ManMode drives the inverter with the raw pots (rung 17) *)
62       SpdFinal_Pps := INT_TO_REAL(Pot2_Slow_Velocity) / Input_Ratio * 1000.0 /
   60.0 / MmPerPulse;
63       SpdWork_Pps  := INT_TO_REAL(Pot1_Work_Velocity) / Input_Ratio * 1000.0 /
   60.0 / MmPerPulse;
64    ELSIF Input_Ratio > 0.0 THEN
65       SpdFinal_Pps := INT_TO_REAL(Pot2_Mem) / Input_Ratio * 1000.0 / 60.0 /
   MmPerPulse;
66       SpdWork_Pps  := INT_TO_REAL(Pot1_Mem) / Input_Ratio * 1000.0 / 60.0 /
   MmPerPulse;
67    ELSE
68       SpdFinal_Pps := LowSpd_Display * 1000.0 / 60.0 / MmPerPulse;
69       SpdWork_Pps  := Speed_Display  * 1000.0 / 60.0 / MmPerPulse;
70    END_IF;
71
72    (* v6y: rewinder AUTO stop -> work speed limited to RwAutoMaxSpdAct *)
73    (* (limit + value taken over only at standstill, Motor_Manager rung 5) *)
74
75    IF RwAutoLimAct AND NOT ManMode
76
77      AND (SpdWork_Pps > RwAutoMaxSpdAct * 1000.0 / 60.0 / MmPerPulse) THEN
78       SpdWork_Pps := RwAutoMaxSpdAct * 1000.0 / 60.0 / MmPerPulse;
79    END_IF;
80
81    SpdRef_Pps := MAX(SpdAct_Pps, SpdWork_Pps);
82
83    IF RwStopDecel > 0.0 THEN
84       StopLenFinal_Pls := SpdFinal_Pps * RwTreact
85                         + (SpdFinal_Pps * SpdFinal_Pps) / (2.0 * RwStopDecel);
86    ELSE
87       StopLenFinal_Pls := SpdFinal_Pps * (RwTreact + RwTstopDef / 2.0);
88    END_IF;
89
90    SlowLenBase_Pls := StopLenFinal_Pls * RwTrim;
91
92    IF (SpdRef_Pps > SpdFinal_Pps) AND (SlowDecel_Pps2 > 0.0) THEN
93
94       IF SlowOneRamp OR (TrimDecel_Pps2 <= 0.0) THEN
95          (* manual or learned decel: one constant ramp *)
96          SlowLenBase_Pls := SlowLenBase_Pls
97             + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
98              / (2.0 * SlowDecel_Pps2);
99       ELSE
100         (* auto: coarse ramp down to the band, trim ramp inside it *)
101         SpdKnee_Pps := SpdFinal_Pps + SpdBand_Pps;
102
103         IF SpdRef_Pps > SpdKnee_Pps THEN
104            SlowLenBase_Pls := SlowLenBase_Pls
105               + (SpdRef_Pps * SpdRef_Pps - SpdKnee_Pps * SpdKnee_Pps)
106                / (2.0 * SlowDecel_Pps2)
107               + (SpdKnee_Pps * SpdKnee_Pps - SpdFinal_Pps * SpdFinal_Pps)
108                / (2.0 * TrimDecel_Pps2);
109         ELSE
110            SlowLenBase_Pls := SlowLenBase_Pls
111               + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
```

```
                / (2.0 * TrimDecel_Pps2);
            END_IF;
        END_IF;
    END_IF;

    (* V7f: S-curve rounding: the slow-down ends SCurveTime later ->      *)

    (*      extra length = (SpdRef - SpdFinal) / 2 x SCurveTime           *)

    IF FFEnable AND NOT ManMode AND (RwSlowDecel <= 0.0) AND (DecelWeb >
0.0)
       AND (WebRatio > 0.0) AND (SCurveTime > 0.0)
       AND (SpdRef_Pps > SpdFinal_Pps) THEN
        SlowLenBase_Pls := SlowLenBase_Pls
                    + (SpdRef_Pps - SpdFinal_Pps) * 0.5 * SCurveTime;
    END_IF;

    RwSlowLen_Pls := SlowLenBase_Pls + RwSlowMargin_m * 1000.0 / MmPerPulse;

    IF UwSlowMargin_m > 0.0 THEN
        UwMarginUsed_m := UwSlowMargin_m;
    ELSE
        UwMarginUsed_m := RwSlowMargin_m;
    END_IF;
    UwSlowLen_Pls := SlowLenBase_Pls + UwMarginUsed_m * 1000.0 / MmPerPulse;
END_IF;

IF RwDiam >= RwTargetDiam THEN
    RwRemain_Pls := 0.0;
ELSIF RwMat_K > 0.0 THEN
    RwRemain_Pls := (RwTargetDiam * RwTargetDiam - RwDiam * RwDiam)
                / (2.0 * RwMat_K);
ELSE
    RwRemain_Pls := 1.0E9;                    (* K not learned: plain target *)
END_IF;

RwRemain_m  := RwRemain_Pls  * MmPerPulse / 1000.0;
RwStopLen_m := StopLen_Pls   * MmPerPulse / 1000.0;
RwSlowLen_m := RwSlowLen_Pls * MmPerPulse / 1000.0;

(* ---- 6b. Live operator displays ---------------------------------------- *)

(* Rewinder: diameter where slow-down / stop command will happen          *)

IF RwMat_K > 0.0 THEN
    SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * RwSlowLen_Pls;

    IF SqrtArg > 0.0 THEN
        RwSlowAtDiam := SQRT(SqrtArg);
    ELSE
        RwSlowAtDiam := 0.0;
    END_IF;
    SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * StopLen_Pls;

    IF SqrtArg > 0.0 THEN
        RwStopAtDiam := SQRT(SqrtArg);
    ELSE
        RwStopAtDiam := 0.0;
    END_IF;
```

```
170 ELSE
171     (* v6t: thickness not learned: fixed slow-down point before the target *)
172
173     IF RwUseSlow AND (RwSlowFallbackMm > 0.0) THEN
174         RwSlowAtDiam := RwTargetDiam - RwSlowFallbackMm;
175     ELSE
176         RwSlowAtDiam := RwTargetDiam;
177     END_IF;
178     RwStopAtDiam := RwTargetDiam;
179 END_IF;
180
181 (* Unwinder: meter count where the slow-down will start                *)
182
183 IF MmPerPulse > 0.0 THEN
184     UwSlow_at_Metre := UwStop_At_Meter - UwSlowLen_Pls * MmPerPulse / 1000.0;
185 ELSE
186     UwSlow_at_Metre := UwStop_At_Meter;
187 END_IF;
188
189 IF UwSlow_at_Metre < 0.0 THEN
190     UwSlow_at_Metre := 0.0;
191 END_IF;
192
193 (* Unwinder slow-down request, latched                                *)
194
195 IF (Current_Metre < UwSlow_at_Metre - 50.0) OR (UwStop_At_Meter <= 0.0) THEN
196     UwSlowReq := FALSE;                    (* counter reset / longer roll *)
197 END_IF;
198
199 IF (UwStop_At_Meter > 0.0) AND (Current_Metre >= UwSlow_at_Metre) THEN
200     UwSlowReq := TRUE;
201 END_IF;
202
203 (* ==== "Unwinder slow-down request, latched" (the 2 IF ... UwSlowReq).    *)
204 (* ---- 6c. V7c: unwinder DIAMETER slow-down follows the speed ---------- *)
205 (* Works without AutoStop (only Measure_ON, like General rung 16).        *)
206 (* Paper left above the stop diameter, in pulses:                         *)
207 (*     (UwDiam_Act^2 - UwDiamStop^2) / (2 x RwMat_K)                      *)
208 (* (same paper as the rewinder -> same learned K). Slow down when it is   *)
209 (* shorter than UwSlowLen_Pls = decel from the speed (DecelWeb) + final   *)
210 (* stop + UwSlowMargin_m. Solved for the diameter:                        *)
211 (*     UwSlowDiamCalc = SQRT(UwDiamStop^2 + 2 x RwMat_K x UwSlowLen_Pls)
    *)
212 (* K not learned / UwSlowDiamAuto OFF -> fixed UwSlowDown_Diam as before. *)
213 (* Latched: once slowed down, the point is frozen (+10 mm hysteresis), so *)
214 (* the lower speed does not move it back and speed up again. Released by  *)
215 (* a new roll (diameter 30 mm above the frozen point).                    *)
216
217 IF UwSlowDiamAuto AND (RwMat_K > 0.0) AND (UwDiamStop > 0.0)
218    AND (UwSlowLen_Pls > 0.0) THEN
219     SqrtArg := UwDiamStop * UwDiamStop + 2.0 * RwMat_K * UwSlowLen_Pls;
220     UwSlowDiamCalc := SQRT(SqrtArg);          (* same pattern as 6b *)
221 ELSE
222     UwSlowDiamCalc := UwSlowDown_Diam;
223 END_IF;
224
225 IF UwSlowDiamCalc < UwDiamStop + 10.0 THEN
226     UwSlowDiamCalc := UwDiamStop + 10.0;
```

```
227    ELSIF UwSlowDiamCalc > 1500.0 THEN
228        UwSlowDiamCalc := 1500.0;
229    END_IF;
230
231    IF UwSlowDiamLatch AND (UwDiam_Act > UwSlowDiamAct + 30.0) THEN
232        UwSlowDiamLatch := FALSE;                    (* new roll *)
233    END_IF;
234
235    IF NOT UwSlowDiamLatch THEN
236        UwSlowDiamAct := UwSlowDiamCalc;             (* live point, HMI *)
237
238        IF Measure_ON AND (UwDiam_Act > 0.0) AND (UwDiam_Act <=
       UwSlowDiamCalc) THEN
239            UwSlowDiamLatch := TRUE;
240            UwSlowDiamAct := UwSlowDiamCalc + 10.0;    (* frozen + hysteresis *)
241        END_IF;
242    END_IF;
243
244    (* ---- 7. Rewinder requests: latched, reset when a new core is loaded ---- *)
245
246    IF NOT RwStopEnable
247       OR ((RwDiam < RwTargetDiam - RwResetHyst) AND NOT RwSensFault) THEN
248        RwStopReq := FALSE;
249        RwSlowReq := FALSE;
250    END_IF;
251
252    IF RwStopEnable THEN
253
254        IF (RwRemain_Pls <= StopLen_Pls) OR (RwSensFault AND RwStopOnFault) THEN
255            RwStopReq := TRUE;
256        END_IF;
257
258        IF RwUseSlow AND (RwRemain_Pls <= RwSlowLen_Pls) THEN
259            RwSlowReq := TRUE;
260        END_IF;
261
262        (* v6t: thickness not learned (RwMat_K = 0): slow down at a fixed      *)
263
264        (* distance before the target instead of stopping from full speed.    *)
265
266        IF RwUseSlow AND (RwMat_K <= 0.0) AND (RwSlowFallbackMm > 0.0)
267           AND NOT RwSensFault
268           AND (RwDiam >= RwTargetDiam - RwSlowFallbackMm) THEN
269            RwSlowReq := TRUE;
270        END_IF;
271    END_IF;
272
273    (* ---- 8. One AUTO at a time + single "SLOW DOWN AT" display ---------- *)
274
275    IF UwStopEnable AND NOT UwEnMem THEN             (* unwinder AUTO switched
       ON *)
276        RwStopEnable := FALSE;
277    END_IF;
278
279    IF RwStopEnable AND NOT RwEnMem THEN             (* rewinder AUTO switched
       ON *)
280        UwStopEnable := FALSE;
281    END_IF;
282
```

```
283   UwEnMem := UwStopEnable;
284   RwEnMem := RwStopEnable;
285
286 IF RwStopEnable THEN
287     SlowAtDisp := RwSlowAtDiam;                (* mm *)
288     SlowAtMode := 2;
289   ELSIF UwStopEnable THEN
290     SlowAtDisp := UwSlow_at_Metre;             (* m  *)
291     SlowAtMode := 1;
292   ELSE
293     SlowAtDisp := 0.0;
294     SlowAtMode := 0;                           (* hide display *)
295   END_IF;
296
297   SlowAtShow := (SlowAtMode <> 0);
298
299 (* V9: HMI main page, unwinder column, same place:                    *)
300
301 (*  UwShowStopLen  -> "STOP LENGTH [m]" (UwStop_At_Meter): AUTO STOP    *)
302 (*                    UNWINDER active (selector 12SA5 + unwinder AUTO).  *)
303 (*  UwShowSlowDiam -> "SLOW AT Ø [mm]" (UwSlowDiamAct): normal operation,
      *)
304 (*                    sonar ON (diameter slow-down, General rung 16).    *)
305
306   UwShowStopLen  := I_AutoStop_OS AND UwStopEnable;
307   UwShowSlowDiam := NOT UwShowStopLen AND Measure_ON;
308
309 (* ---- 9. Automatic stop master = selector 12SA5 (I_AutoStop_OS) -------- *)
310
311   AutoSlowCmd := I_AutoStop_OS AND
312               ((UwStopEnable AND UwSlowReq) OR (RwStopEnable AND RwSlowReq));
313
314   AutoStopCmd := I_AutoStop_OS AND
315               ((UwStopEnable AND (UwStop_At_Meter > 0.0)
316                 AND (Current_Metre >= UwStop_At_Meter))
317                OR (RwStopEnable AND RwStopReq));
318
319 (* ---- 10. Unwinder roll empty (for alarm 15) ---------------------------- *)
320
321 IF UwDiam_Act > (UwDiamStop + UwEmptyHyst) THEN
322     UnWinder_Empty := FALSE;                   (* new roll loaded *)
323   END_IF;
324
325 IF (UwDiam_Act > 0.0) AND (UwDiam_Act <= UwDiamStop) THEN
326     UnWinder_Empty := TRUE;                    (* roll empty *)
327   END_IF;
328
329 (* v6x: new parent roll loaded (empty -> not empty) -> HMI popup       *)
330
331 (* "DID YOU CHANGE THE PAPER (GSM)?". YES = forget the learned thickness *)
332 (* (same as NEW PAPER), NO = the HMI resets GsmAsk.                    *)
333 (* V9: ask only ONCE per REAL new roll: the roll must have been empty for *)
334 (* 3 s (arms the question), then the new roll must read more than      *)
335 (* UwDiamStop + 100 mm for 3 s. The sonar bouncing around UwDiamStop    *)
336 (* (empty core, hands, arms) no longer re-opens the popup.             *)
337
338 IF Tick THEN
339     IF UnWinder_Empty THEN
340
```

```
341         IF UwEmptyTicks < 1000 THEN
342             UwEmptyTicks := UwEmptyTicks + 1;
343         END_IF;
344
345         IF UwEmptyTicks >= 30 THEN
346             GsmArmed := TRUE;
347         END_IF;
348
349         UwFullTicks := 0;
350     ELSE
351         UwEmptyTicks := 0;
352         IF GsmArmed AND Measure_ON AND (UwDiam_Act > UwDiamStop + 100.0)
    THEN
353             UwFullTicks := UwFullTicks + 1;
354
355             IF UwFullTicks >= 30 THEN
356                 GsmAsk := TRUE;
357                 GsmArmed := FALSE;
358                 UwFullTicks := 0;
359             END_IF;
360         ELSE
361             UwFullTicks := 0;
362         END_IF;
363     END_IF;
364 END_IF;
365
366 UwEmptyMem := UnWinder_Empty;
367
368 IF GsmAskYes THEN
369     RwMat_K := 0.0;
370     RwMatLocked := FALSE;                    (* V9 *)
371     LongActive := FALSE;
372     LearnFailCnt := 0;
373     MatChangeCnt := 0;
374     GsmAskYes := FALSE;
375     GsmAsk := FALSE;
376 END_IF;
377
378 (* ---- 11. Rewinder max diameter protection - ALWAYS active ---------- *)
379 (* Independent of selector 12SA5 and of the AUTO buttons.              *)
380 (* Mechanical limit 680 mm radius = 1360 mm                           *)
381
382 IF NOT RwSensFault THEN
383     RwLastValidDiam := RwDiam;
384 END_IF;
385
386 IF RwDiam >= RwMaxDiam THEN
387     RwRemainMax_Pls := 0.0;
388 ELSIF RwMat_K > 0.0 THEN
389     RwRemainMax_Pls := (RwMaxDiam * RwMaxDiam - RwDiam * RwDiam)
390                        / (2.0 * RwMat_K);
391 ELSE
392     RwRemainMax_Pls := 1.0E9;                (* K not learned: plain limit *)
393 END_IF;
394
395 (* Display: diameter where the max-protection slow-down will start       *)
396
397 IF RwMat_K > 0.0 THEN
398     SqrtArg := RwMaxDiam * RwMaxDiam - 2.0 * RwMat_K * RwSlowLen_Pls;
```

```
399
400     IF SqrtArg > 0.0 THEN
401         RwMaxSlowAtDiam := SQRT(SqrtArg);
402     ELSE
403         RwMaxSlowAtDiam := 0.0;
404     END_IF;
405   ELSIF RwMaxSlowFallback > 0.0 THEN
406     RwMaxSlowAtDiam := RwMaxSlowFallback;          (* fallback in use *)
407   ELSE
408     RwMaxSlowAtDiam := RwMaxDiam;
409   END_IF;
410
411 (* Reset only after the roll is removed (diameter well below the limit)    *)
412
413 IF (RwDiam < RwMaxDiam - RwResetHyst) AND NOT RwSensFault THEN
414     RwMaxStop := FALSE;
415     RwMaxSlow := FALSE;
416   END_IF;
417
418 IF NOT RwSensFault THEN
419
420     IF RwRemainMax_Pls <= StopLen_Pls THEN
421         RwMaxStop := TRUE;                         (* speed-compensated stop *)
422     END_IF;
423
424     IF RwRemainMax_Pls <= RwSlowLen_Pls THEN
425         RwMaxSlow := TRUE;                         (* warning slow-down + horn *)
426     END_IF;
427
428     IF (RwMat_K <= 0.0) AND (RwMaxSlowFallback > 0.0)
429       AND (RwDiam >= RwMaxSlowFallback) THEN
430         RwMaxSlow := TRUE;                         (* fallback while K not learned *)
431     END_IF;
432
433   ELSIF (AI_Diam_Rewinder < RwRawMin) AND (RwLastValidDiam > RwMaxDiam -
      100.0) THEN
434     RwMaxStop := TRUE;                             (* roll grew into the DT35 near zone *)
435   END_IF;
436
437 (* Max diameter protection added OUTSIDE the 12SA5 master (section 9)      *)
438
439   AutoSlowCmd := AutoSlowCmd OR RwMaxSlow;
440   AutoStopCmd := AutoStopCmd OR RwMaxStop;
441
442 (* ---- 12. Learning supervision -> alarm 16 (section Alarms) ------------ *)
443
444 (* Learned paper thickness, for display / plausibility check              *)
445
446 IF MmPerPulse > 0.0 THEN
447     RwThick_mm := RwMat_K * 3.14159 / (2.0 * MmPerPulse);
448   ELSE
449     RwThick_mm := 0.0;
450   END_IF;
451
452 (* Fault: too many consecutive windows / stops that should have           *)
453 (* learned but did not. Clears by itself on the next good result.         *)
454 (* v6w: ALARM RESET (panel button or HMI) also clears the fail counters, *)
455 (* so alarm 16 resets in one press. It comes back only if the learning  *)
456 (* fails again RwLearnFailMax / RwStopFailMax times in a row.             *)
```

```
457
458 IF I_RsetAlarm OR Alarm_Reset THEN
459     LearnFailCnt := 0;
460     StopFailCnt := 0;
461 END_IF;
462
463 GSM_LearnFault := (LearnFailCnt >= RwLearnFailMax)
464             OR (StopFailCnt >= RwStopFailMax);      (* -> alarm 16 *)
465
466 (* v6e: learning status for the HMI. FALSE until learned.                *)
467
468 GSM_Learned_OK := (RwMat_K > 0.0)
469               AND ((RwStopDecel > 0.0) OR NOT RwLearnStop)
470               AND NOT GSM_LearnFault;
471
472 (* ---- 13. v6h: keep T-one in automatic tension control during the stop - *)
473 (* While running: BrakeHold = Brake_Cmd (no change in running behaviour).  *)
474 (* After the stop: stays ON until the web is below 5 m/min, after 15 s,   *)
475 (* or on E-stop. Already TRUE on the stop scan (Output_Manager runs before *)
476 (* this section), so O_Brake never drops for one scan.                    *)
477 (* Output_Manager rung 2:  O_Brake := Brake_Cmd OR BrakeHold              *)
478
479 IF O_MotorStart THEN
480     BrakeHold := Brake_Cmd AND ControlVoltage;
481     BrakeHoldTicks := 0;
482 ELSIF BrakeHold THEN
483
484     IF Tick THEN
485         BrakeHoldTicks := BrakeHoldTicks + 1;
486     END_IF;
487     (* v6v: signed web speed: a web pulled BACK (negative speed) also     *)
488
489     (* releases the T-one (trace 10/28 15:00: -6.6 m/min, about 20 cm).  *)
490
491     IF NOT ControlVoltage OR (Velocity_M_min < 5.0) OR (BrakeHoldTicks >= 150)
    THEN
492         BrakeHold := FALSE;
493     END_IF;
494 END_IF;
495
496 (* ---- 14. v6j: TEACH CORE - automatic offset with an empty core -------- *)
497
498 (* HMI: set RwTeachCore (button, Set ON). Conditions: motor stopped, web  *)
499 (* not moving, DT35 OK. After 2 s stable (filter settled):                *)
500 (*     RwAxisDist := RwDist + RwCoreDiameter / 2   -> RwDiam = RwCoreDiameter
    *)
501 (* RwTeachState: 0 idle, 1 teaching (wait 2 s), 2 done OK,                *)
502 (*               3 refused (motor running / web moving / DT35 fault).     *)
503 (* Only the offset is taught: the scale is RwDist10V - RwDist0V           *)
504 (* (= real distance between the two DT35 taught points).                  *)
505
506 IF RwTeachCore THEN
507
508     IF O_MotorStart OR (RwSpeed > 1.0) OR RwSensFault
509       OR NOT RwFiltInit OR (RwCoreDiameter <= 0.0) THEN
510         RwTeachState := 3;                  (* refused *)
511         RwTeachCore := FALSE;
512         TeachTicks := 0;
513     ELSE
```

```
514       RwTeachState := 1;                    (* teaching *)
515
516       IF Tick THEN
517          TeachTicks := TeachTicks + 1;
518       END_IF;
519
520       IF TeachTicks >= 20 THEN               (* 2 s stable *)
521          RwAxisDist_Prev := RwAxisDist;      (* v6l: keep previous *)
522          RwAxisDist := RwDist + RwCoreDiameter / 2.0;
523          RwTeachState := 2;                  (* done *)
524          RwTeachCore := FALSE;
525          TeachTicks := 0;
526       END_IF;
527    END_IF;
528 ELSE
529    TeachTicks := 0;
530
531    IF O_MotorStart THEN
532       RwTeachState := 0;                     (* clear status on next run *)
533    END_IF;
534 END_IF;
535
536 (* ---- 14b. v6k: unwinder TEACH CORE (HMI button Core_Measure) ---------- *)
537
538 (* Same behaviour as section 14. Sonar is not filtered, so the 2 s are   *)
539 (* averaged:  UwCoreDistance := average(UwDiam_Measure)  (sonar -> core   *)
540 (* surface). UwDiam_Act = (UwCoreDistance - UwDiam_Measure) * 2          *)
541 (*                      + UwCoreDiameter   (General rung 14, unchanged). *)
542 (* UwTeachState: 0 idle, 1 teaching, 2 done OK, 3 refused.               *)
543
544 IF Core_Measure THEN
545
546    IF O_MotorStart OR (RwSpeed > 1.0) OR (UwCoreDiameter <= 0.0)
547       OR (UwDiam_Measure <= 0.0) THEN
548       UwTeachState := 3;                     (* refused *)
549       Core_Measure := FALSE;
550       UwTeachTicks := 0;
551       UwTeachSum := 0.0;
552    ELSE
553       UwTeachState := 1;                     (* teaching *)
554
555       IF Tick THEN
556          UwTeachTicks := UwTeachTicks + 1;
557          UwTeachSum := UwTeachSum + UwDiam_Measure;
558       END_IF;
559
560       IF UwTeachTicks >= 20 THEN             (* 2 s, 20 samples *)
561          UwCoreDist_Prev := UwCoreDistance;  (* v6l: keep previous *)
562          UwCoreDistance := UwTeachSum / INT_TO_REAL(UwTeachTicks);
563          UwTeachState := 2;                  (* done *)
564          Core_Measure := FALSE;
565          UwTeachTicks := 0;
566          UwTeachSum := 0.0;
567       END_IF;
568    END_IF;
569 ELSE
570    UwTeachTicks := 0;
571    UwTeachSum := 0.0;
572
```

```
573     IF O_MotorStart THEN
574         UwTeachState := 0;                  (* clear status on next run *)
575     END_IF;
576 END_IF;
577
578 (* ---- 14c. v6l: UNDO teach = swap current <-> previous (machine stopped) *)
579
580 (* Pressing UNDO twice goes back to the taught value. State 4 = RESTORED. *)
581
582 IF RwTeachUndo THEN
583
584     IF NOT O_MotorStart AND (RwAxisDist_Prev > 0.0) THEN
585         TeachSwap := RwAxisDist;
586         RwAxisDist := RwAxisDist_Prev;
587         RwAxisDist_Prev := TeachSwap;
588         RwTeachState := 4;                  (* restored *)
589     ELSE
590         RwTeachState := 3;                  (* refused *)
591     END_IF;
592     RwTeachUndo := FALSE;
593 END_IF;
594
595 IF UwTeachUndo THEN
596
597     IF NOT O_MotorStart AND (UwCoreDist_Prev > 0.0) THEN
598         TeachSwap := UwCoreDistance;
599         UwCoreDistance := UwCoreDist_Prev;
600         UwCoreDist_Prev := TeachSwap;
601         UwTeachState := 4;                  (* restored *)
602     ELSE
603         UwTeachState := 3;                  (* refused *)
604     END_IF;
605     UwTeachUndo := FALSE;
```

1-9.Data

1-9-1.Data Types

Name	Type	Comment

1-9-2.Global Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Network Publish	Comment
VAR_GLOBAL							
Pot1_Work_Velocity	INT		BuiltInIO://opt/#0/Ch1_Analog_Input_Value	False	False	Publish Only	Potentiometer 1 value - Work speed -
InvRef	INT		BuiltInIO://opt/#0/Ch2_Analog_Output_Value	False	False	Publish Only	Inverter speed reference
Fix10Vdc	INT		BuiltInIO://opt/#0/Ch1_Analog_Output_Value	False	False	Publish Only	Fixed 10 Vdc generator
VelMax	REAL	800		True	False	Publish Only	Maximum speed
VelSet	REAL			False	False	Publish Only	Set speed
RwSlowDecelK	REAL	0.15		True	False	Publish Only	Learned slow-down decel per mm of rewinder diameter (mm/s2 per mm)
RwMaxSlowFallback	REAL	1360.0		True	False	Publish Only	Rewinder max: slow-down diameter while K not learned (mm)
RwThickMin_mm	REAL	0.02		True	False	Publish Only	Learning: min plausible paper thickness (mm)
RwThickMax_mm	REAL	2.0		True	False	Publish Only	Learning: max plausible paper thickness (mm)
RwDecelMin_mm	REAL	20.0		True	False	Publish Only	Learning: min plausible stop decel (mm/s2)
RwDecelMax_mm	REAL	5000.0		True	False	Publish Only	Learning: max plausible stop decel (mm/s2)
RwLearnFailMax	INT	12		True	False	Publish Only	Learning: failed windows before alarm (12 = 2 min)
RwStopFailMax	INT	3		True	False	Publish Only	Learning: failed stops before alarm
RwThick_mm	REAL			True	False	Publish Only	Learned paper thickness (mm)
OverSpeed	BOOL			False	False	Publish Only	Speed too high
WorkSpd_Max	REAL	400.0		True	False	Publish Only	Work speed pot full scale (m/min), 50..VelMax
SoftStartTime	REAL	3.0		True	False	Publish Only	AUTO start S-curve: time for the accel ramp to rise from 5 % to 100 % (s), 0 = off
WorkSpd_Max_Set	REAL	400.0		True	False	Publish Only	HMI request: work pot full scale (m/min), applied only with motor stopped
UwSlowDiamAuto	BOOL	True		True	False	Publish Only	V7c: unwinder diameter slow-down follows the speed (FALSE = fixed UwSlowDown_Diam)
UwSlowDiamAct	REAL	0.0		False	False	Publish Only	V7c: unwinder slows down at this diameter (mm), HMI display
RwJogMaxDiam	REAL	400.0		True	False	Publish Only	V7d: jog blocked above this rewinder diameter (mm), 0 = no limit
JogInhibit	BOOL	False		False	False	Publish Only	V7d: jog blocked (rewinder roll too big), HMI lamp
SCurveTime	REAL	0.5		True	False	Publish Only	V7f: S-curve rounding of the AUTO ramps (s), 0 = off

							, max 2
TOneDecel	BOOL	False		False	False	Publish Only	V7f: line decelerating (for a T-one stop/boost input)
HornStartBeep_s	REAL	1.0		True	False	Publish Only	V9 horn: beep when the slow-down starts (s), 0 = off, max 5
HornBeepLen_s	REAL	0.2		True	False	Publish Only	V9 horn: length of the short beeps during slow-down (s), 0 = off, max 2
HornBeepPeriod	REAL	5.0		True	False	Publish Only	V9 horn: one short beep every ... s during slow-down, 0 = off, max 30
HornStopBeep_s	REAL	3.0		True	False	Publish Only	V9 horn: beep when stopped after a slow-down (s), 0 = off, max 10
HornTest	BOOL	False		False	False	Publish Only	V9 horn: HMI test button (momentary), machine stopped only
RwLockDiam	REAL	300.0		True	False	Publish Only	V9: paper thickness LOCKED at this rewinder diameter (mm), 0 = never lock
RwLockMinGrow_mm	REAL	50.0		True	False	Publish Only	V9: min diameter growth since the start of learning before locking (mm)
RwMatLocked	BOOL	False		True	False	Publish Only	V9: paper thickness locked (HMI lamp), NEW PAPER / GSM YES unlocks
LongThick_mm	REAL	0.0		False	False	Publish Only	V9: thickness from the long baseline (mm), HMI display
RwDiamRev	REAL	0.0		False	False	Publish Only	V9: rewinder diameter averaged over one revolution (mm)
RwWave_mm	REAL	0.0		False	False	Publish Only	V9: diameter wave peak-peak in the last revolution (mm), HMI
UwShowStopLen	BOOL	False		False	False	Publish Only	V9 HMI: show unwinder STOP LENGTH (AUTO STOP UNWINDER active)
UwShowSlowDiam	BOOL	False		False	False	Publish Only	V9 HMI: show unwinder SLOW AT diameter (normal operation, sonar ON)
SoftCancelAccel	REAL	100.0		True	False	Publish Only	V9: then ramp up at this web accel (mm/s2), 0 = AccelWeb
RwMatReset	BOOL	False		False	False	Publish Only	HMI NEW PAPER: forget the learned paper thickness RwMat_K (PLC resets it)
FinalSpd_Max_Set	REAL	40.0		True	False	Publish Only	HMI request: final pot full scale (m/min), applied only with motor stopped
FinalSpd_Max	REAL	40.0		True	False	Publish Only	Final speed pot full scale (m/min), 5..WorkSpd_Max
UnderSpeed	BOOL			False	False	Publish Only	Speed too low
RwLearnReset	BOOL	False		False	False	Publish Only	HMI / Watch: clear learned decels RwStopDecelK, RwStopDecel, RwSlowDecelK (PLC resets it)
RwCoreDiameter	REAL	100.0		True	False	Publish Only	Rewinder core outside diameter for TEACH COR

							E (mm)
RwTeachCore	BOOL	False		False	False	Publish Only	HMI: TEACH CORE request (PLC resets it)
RwTeachState	INT	0		False	False	Publish Only	TEACH CORE: 0 idle 1 teaching 2 done 3 refused
SpdDelta_Percent	REAL			True	False	Publish Only	Speed difference percentage
Encoder_Pos	DINT		IOBus://unit#3/Ch1 Encoder Present Position	False	False	Publish Only	Encoder counter
UwTeachState	INT	0		False	False	Publish Only	Unwinder TEACH CORE: 0 idle 1 teaching 2 done 3 refused
PulseRate	UDINT		IOBus://unit#3/Ch1 Pulse Rate	False	False	Publish Only	Pulses per 100 ms
Encoder_On	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Counter Enable	False	False	Publish Only	Encoder enable
Encoder_Reset	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Internal Reset Execution	False	False	Publish Only	Encoder reset
HMI_Screen	ARRAY[0..1] OF DINT			False	False	Publish Only	HMI panel Screen
Velocity_M_min	REAL			True	False	Publish Only	Speed in m/min
Current_Metre	REAL			True	False	Publish Only	Current metres
I_RsetCnt	BOOL			False	False	Publish Only	Metre counter reset button
Speed_Display	REAL			True	False	Publish Only	Set speed (displayed)
I_ControlVoltage	BOOL		BuiltInIO://cpu/#0/Input_Bit_00	False	False	Publish Only	Auxiliaries ON
I_Start_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_01	False	False	Publish Only	Start button
I_Stop_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_02	False	False	Publish Only	Stop button
I_Motor_Thermal	BOOL		BuiltInIO://cpu/#0/Input_Bit_03	False	False	Publish Only	Motor thermal switch
I_Prot_ByPass	BOOL		BuiltInIO://cpu/#0/Input_Bit_04	False	False	Publish Only	Guards-bypassed selector switch - Under Panel
I_Hub_Position	BOOL		BuiltInIO://cpu/#0/Input_Bit_05	False	False	Publish Only	Hub/chuck in extraction position

I_FrontGuard_Closed	BOOL		BuiltInIO://cpu/#0/Input_Bit_06	False	False	Publish Only	Front guard closed
I_Jog_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_07	False	False	Publish Only	Jog Push button
I_VFD_Alarm	BOOL		BuiltInIO://cpu/#0/Input_Bit_08	False	False	Publish Only	Inverter in alarm
I_SafeGuard_Closed	BOOL			False	False	Publish Only	Side doors closed
I_OverRider_PH	BOOL		BuiltInIO://cpu/#0/Input_Bit_13	False	False	Publish Only	Roll diameter photocell
I_OR_Position_Sens	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN01 (Pin 1.1) _6000_01	False	False	Publish Only	OverRider position sens
I_OR_Pressure_Sens	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN02 (Pin 2.1) _6000_02	False	False	Publish Only	OverRider pressure switch
I_OR_Auto_OS	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN03 (Pin 1.2) _6000_03	False	False	Publish Only	OverRider in automatic
I_OR_Cont_OS	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN04 (Pin 2.2) _6000_04	False	False	Publish Only	OverRider in continuous
O_MotorStart	BOOL		BuiltInIO://cpu/#0/Output_Bit_00	False	False	Publish Only	Motor start
I_FanTemp_Ok	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN06 (Pin 2.3) _6000_06	False	False	Publish Only	Motor (fan) thermal OK
O_Alarm_Array	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT01 (Pin 1.1) _7000_01	False	False	Publish Only	Alarm lamp
O_Prot_ByPass_Lamp	BOOL			False	False	Publish Only	Guards bypassed lamp
O_Motor_Lamp	BOOL		BuiltInIO://cpu/#0/Output	False	False	Publish Only	Motor ON lamp

			_Bit_01				
O_RwUnload	BOOL		ECAT://node #[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT 03 (Pin 1.2) _7000_03	False	False	Publish Only	Unloaders enable - opening
O_FrontGuard_ByPass	BOOL		ECAT://node #[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT 04 (Pin 2.2) _7000_04	False	False	Publish Only	Front guard bypass
O_Brake	BOOL		BuiltInIO://cpu/#0/Output_Bit_02	False	False	Publish Only	Brake command
O_OR_Auto_Lamp	BOOL		BuiltInIO://cpu/#0/Output_Bit_06	False	False	Publish Only	OverRider auto/continuous lamp
O_OverRider_Up	BOOL		BuiltInIO://cpu/#0/Output_Bit_09	False	False	Publish Only	Raise OverRider ~10mm Over Rewinder roll
O_OverRider_Hold	BOOL		BuiltInIO://cpu/#0/Output_Bit_04	False	False	Publish Only	OverRider Lock
O_CoolingFan	BOOL		BuiltInIO://cpu/#0/Output_Bit_03	False	False	Publish Only	Brake and motor fans
Fan_Timeout	REAL	0.5		True	False	Publish Only	Fans stop timeout [min]
OverRider_TimeOFF	REAL	0.01		True	False	Publish Only	OverRider photocell OFF timeout (panel)
OverRider_TimeON	REAL	0.05		True	False	Publish Only	OverRider photocell ON timeout (panel)
AlarmOn	BOOL			False	False	Publish Only	Alarm active
BuzzerOn	BOOL			False	False	Publish Only	Panel buzzer command
Msg	INT			False	False	Publish Only	Message
Index_B	INT			False	False	Publish Only	Alarm index
MotStop	BOOL			False	False	Publish Only	Motor stopped
Run_Normal_Spd	BOOL			False	False	Publish Only	Motor running - normal speed
Clock1s	BOOL			False	False	Publish Only	1 s clock
ManMode	BOOL			True	False	Publish Only	Manual mode
Speed_Man_SP	INT			True	False	Publish Only	% speed in manual
Counter_Reset	BOOL			False	False	Publish Only	Counter reset from panel
I_FrontGuard_Open	BOOL		BuiltInIO://cpu/#0/Input_Bit_09	False	False	Publish Only	Front guard open
RwUnload_Enable	BOOL		BuiltInIO://cpu/#0/Output_Bit_07	False	False	Publish Only	Unloaders enable - closing
JogSpd_SP	REAL	10.0		True	False	Publish Only	Speed in manual
O_AccDec2	BOOL		BuiltInIO://cpu/#0/Output_Bit_05	False	False	Publish Only	Second acc/dec ramp
AccelTrim	REAL			True	False	Publish Only	Acceleration coefficient in run
DecelTrim	REAL			True	False	Publish Only	Deceleration coefficient in run

DecelCoarse	REAL			True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL			True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests accel)
O_AcousticSignal	BOOL		BuiltInIO://cpu/#0/Output_Bit_08	False	False	Publish Only	Horn / Acoustic signal
Pot2_Slow_Velocity	INT		BuiltInIO://opt/#0/Ch2_Analog_Input_Value	False	False	Publish Only	Potentiometer 2 value - Final speed -
I_AutoStop_OS	BOOL		BuiltInIO://cpu/#0/Input_Bit_10	False	False	Publish Only	Stop on metre counter
I_RollEnd_SlowDown	BOOL		BuiltInIO://cpu/#0/Input_Bit_11	False	False	Publish Only	Slow down at end of roll
I_WebBreak_Monitor	BOOL		BuiltInIO://cpu/#0/Input_Bit_12	False	False	Publish Only	Paper break monitoring ON
I_Vacuum_OK	BOOL		IOBus://unit#3/Ch1 Reset/External Input Status/Ch1 External Input 0 Status	False	False	Publish Only	Extractor fan thermal OK
I_SlowSens_PH	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN05 (Pin 1.3) _6000_05	False	False	Publish Only	Slow-down photocell
I_WebBreak_PH	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN07 (Pin 1.4) _6000_07	False	False	Publish Only	Paper break photocell
UwStop_At_Meter	REAL	5100		True	False	Publish Only	Unwinder Stop at metres
UwSlowMargin_m	REAL	10		True	False	Publish Only	Unwinder Slow-down metres
LowSpd_Display	REAL			True	False	Publish Only	Slow-down speed (displayed)
RwMaxDiam	REAL	1380.0		True	False	Publish Only	Rewinder max diameter protection (mm)
RwMaxStop	BOOL			False	False	Publish Only	Rewinder max diameter STOP (latched)
Encoder_Pulse	INT	1000		True	False	Publish Only	Encoder pulses/Rotation
RwSlowDecelLrn	REAL	0.0		True	False	Publish Only	Learned real slow-down decel (mm/s2) - 0 = not learned
LowSpd_Man_SP	INT			False	False	Publish Only	% slow-down speed in manual
CountReset_	BOOL			False	False	Publish Only	Counter reset requested

Req							
Psw	INT			False	False	Publish Only	Password
Language	DINT			True	False	Publish Only	Language
I_RsetAlarm	BOOL		ECAT://node #[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN08 (Pin 2.4) _6000_08	False	False	Publish Only	Alarm reset
Fix10Vdc_1	UINT			False	False	Publish Only	Fixed 10 Vdc generator
EncoderWheel_Diam	REAL	400		True	False	Publish Only	Encoder wheel diameter
AlarmCodeSV	ARRAY[0..32] OF BOOL			False	False	Publish Only	Panel alarms
SafeGuard_ByPass	BOOL			False	False	Publish Only	Guards bypassed
FrontGuard_Closed	BOOL			False	False	Publish Only	Front guard closed
Anal3	UINT			False	False	Publish Only	(?) Analog 3
Anal4	UINT			False	False	Publish Only	(?) Analog 4
SoftStop	BOOL	False		False	False	Publish Only	Soft stop in progress (HMI)
SoftStopEnable	BOOL	True		True	False	Publish Only	STOP button = soft stop (False = immediate as before)
SoftStopEndSpd	REAL	5.0		True	False	Publish Only	Soft stop: run command drops below this speed (m/min)
UwDiam_Measure	REAL			True	False	Publish Only	Measured distance
UwCoreDiameter	REAL	100		True	False	Publish Only	Core Outside diameter [mm]
Core_Measure	BOOL	False		True	False	Publish Only	Stored core value
UwCoreDistance	REAL	785.56		True	False	Publish Only	Sonar distance to empty core. [mm]
AI_Diam_Unwinder	INT		IOBus://unit#2/Ch1 Analog Input Value	False	False	Publish Only	Analog value - UnWinder Diameter
Measure_ON	BOOL	True		True	False	Publish Only	Measurement enabled
UwDiam_Act	REAL			True	False	Publish Only	Measured roll diameter
UwSlowDown_Diam	REAL	200		True	False	Publish Only	Unwinder Slow-down diameter [mm]
UwDiamStop	REAL	110		True	False	Publish Only	Stop diameter
Alarm_Reset	BOOL			False	False	Publish Only	Alarm reset from panel
Psw22	BOOL			False	False	Publish Only	Password 22
Psw44	BOOL			False	False	Publish Only	Password 44
I_Roll_OK	BOOL			False	False	Publish Only	Roll OK / Broken Holder Joint
I_RollEnd_PH	BOOL			False	False	Publish Only	End-of-roll photocell
AI_Diam_Rewinder	INT		IOBus://unit#2/Ch2 Analog Input Value	False	False	Publish Only	Analog value - ReWinder Diameter
RwStopEnable	BOOL	False		True	False	Publish Only	Rewinder diameter stop enabled
RwTargetDiam	REAL	300.0		True	False	Publish Only	Rewinder final diameter (mm)
RwUseSlow	BOOL	True		True	False	Publish Only	Slow down before target

RwSlowMargin_m	REAL	10.0		True	False	Publish Only	Rewinder Metres at slow speed before stop
RwSlowDecel	REAL	0.0		True	False	Publish Only	Rewinder Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL	100.0		True	False	Publish Only	Reset requests below target - this [mm]
RwDist0V	REAL	0.0		True	False	Publish Only	DT35 distance at 0 V (mm)
RwDist10V	REAL	750.0		True	False	Publish Only	DT35 distance at 10 V (mm)
RwRawMin	INT	10		True	False	Publish Only	Raw below = sensor fault
RwRawMax	INT	3900		True	False	Publish Only	Raw above = sensor fault
RwStopOnFault	BOOL	True		False	False	Publish Only	Stop on sensor fault
RwAxisDist	REAL	675.3		True	False	Publish Only	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL	0.2		True	False	Publish Only	Distance filter per 100 ms (0..1)
RwWinTicks	INT	100		True	False	Publish Only	Material learn window (100 = 10 s)
RwMinPulses	DINT	1000		True	False	Publish Only	Min pulses in window to learn
RwLearn_K	REAL	0.3		True	False	Publish Only	Material factor filter weight
RwMat_K	REAL	0.0		True	False	Publish Only	Learned material factor
RwLearnStop	BOOL	True		True	False	Publish Only	Learn stop decel on every stop
RwMinLearnSpd	REAL	50.0		True	False	Publish Only	Min speed to learn a stop (m/min)
RwStopDecel	REAL	0.0		True	False	Publish Only	Learned stop decel (pulses/s2)
RwTreact	REAL	0.1		True	False	Publish Only	Stop reaction time (s)
RwTstopDef	REAL	3.0		True	False	Publish Only	Stop time until decel learned (s)
RwTrim	REAL	1.0		True	False	Publish Only	Stop length trim (>1 = earlier)
RwDist	REAL			False	False	Publish Only	Filtered distance (mm)
RwDiam	REAL			False	False	Publish Only	Rewinder diameter (mm)
RwSpeed	REAL			False	False	Publish Only	Line speed from encoder (m/min)
RwRemain_m	REAL			False	False	Publish Only	Metres to target diameter
RwStopLen_m	REAL			False	False	Publish Only	Stopping length now (m)
RwSlowLen_m	REAL			False	False	Publish Only	Rewinder Slow-down length [m]
RwStopReq	BOOL			False	False	Publish Only	Rewinder diameter STOP request
RwSlowReq	BOOL			False	False	Publish Only	Rewinder diameter SLOW request
RwSensFault	BOOL			False	False	Publish Only	Rewinder DT35 sensor fault
UwStopEnable	BOOL	False		True	False	Publish Only	Unwnder Auto Stop - Enable
SlowAtDisp	REAL			False	False	Publish Only	HMI: slow-down starts at (m or mm, see SlowAtMode)
SlowAtMode	INT			False	False	Publish Only	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL			False	False	Publish Only	HMI: show slow-down display
UwSlow_at_Metre	REAL			False	False	Publish Only	Unwinder: slow-down starts at [m]

UwSlowReq	BOOL			False	False	Publish Only	Unwinder metre slow-down request (latched)
RwSlowAtDiam	REAL			False	False	Publish Only	Rewinder slow-down starts at diameter [mm]
RwStopAtDiam	REAL			False	False	Publish Only	Rewinder: stop command at diameter (mm)
RwMaxSlow	BOOL			False	False	Publish Only	Rewinder max diameter SLOW (latched)
UwEmptyHyst	REAL	5.0		True	False	Publish Only	Unwinder Empty Hysteresis
GSM_Learned_OK	BOOL			False	False	Publish Only	Paper [Grams / Square Meter] FALSE until GSM learned
RwMaxSlowAtDiam	REAL	1370.0		False	False	Publish Only	Rewinder max: slow-down starts at diameter (mm)
RwStopDecelK	REAL	0.0		True	False	Publish Only	Learned stop decel per mm of rewinder diameter (mm/s2 per mm)
SoftStopDecel	REAL	0.0		True	False	Publish Only	Soft stop decel (mm/s2), 0 = DecelCoarse
SpdMax_Pending	BOOL	False		False	False	Publish Only	New pot range waiting for the motor to stop
RwAxisDist_Prev	REAL	675.3		True	False	Publish Only	Rewinder: RwAxisDist before the last TEACH (for UNDO)
UwCoreDist_Prev	REAL	785.56		True	False	Publish Only	Unwinder: UwCoreDistance before the last TEACH (for UNDO)
RwTeachUndo	BOOL	False		False	False	Publish Only	HMI: UNDO rewinder TEACH (PLC resets it)
UwTeachUndo	BOOL	False		False	False	Publish Only	HMI: UNDO unwinder TEACH (PLC resets it)
Psw33	BOOL			False	False	Publish Only	Password 33
RwMatChangePct	REAL	25.0		True	False	Publish Only	Paper change detection: window deviation (%), 2 windows in a row = new paper, 0 = off
RwSlowFallbackMm	REAL	30.0		True	False	Publish Only	Rewinder: slow-down this many mm before the target while the paper thickness is not learned (0 = off)
RwAutoMaxSpd	REAL	500.0		True	False	Publish Only	Rewinder AUTO stop ON: maximum work speed (m/min), 0 = no limit
RwWinLen_m	REAL	30.0		True	False	Publish Only	Paper thickness learning: MAX window length (m of paper)
RwWinGrow_mm	REAL	2.5		True	False	Publish Only	Paper thickness learning: window = this diameter growth (mm), 4 m .. RwWinLen_m
GsmAsk	BOOL	False		False	False	Publish Only	HMI popup: new parent roll, did you change the paper? (HMI NO resets it)
GsmAskYes	BOOL	False		False	False	Publish Only	HMI popup YES: forget the learned thickness (PLC resets it)
DecelWeb	REAL	250.0		True	False	Publish Only	V7: AUTO decel ramp, real web mm/s2 (0 = DecelCoarse)
AccelWeb	REAL	200.0		True	False	Publish Only	V7: AUTO accel ramp, real web mm/s2 (0 = AccelCoar

							se)
	RwRatioD	REAL	2900.0		True	False	Publish Only	V7: drive counts per m/min x rewinder diameter (mm), adapted while running
	FFEnable	BOOL	True		True	False	Publish Only	V7: feed-forward speed loop ON (FALSE = old loop of rung 16)
	SpdHold	BOOL	False		False	False	Publish Only	V9: speed held after START during a soft stop, START again = ramp up (HMI lamp)